ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/371351533

libro c++

Book · June 2023

CITATIONS
0

READS
6,442

1 author:



Jorge Domínguez Chávez
Universidad Politécnica Territorial del Estado Aragua
84 PUBLICATIONS **55** CITATIONS

SEE PROFILE

All content following this page was uploaded by Jorge Domínguez Chávez on 07 June 2023.

The user has requested enhancement of the downloaded file.



Universidad Politécnica Territorial del Estado Aragua
Departamento de Informática
Ingeniería en Informática

Algorítmica y Programación

Jorge Domínguez Chávez

Fundamentos de Programación en Lenguaje C++

Venezuela, Mayo, 2023

Copyright ® Jorge Domínguez Chávez.

ORCID: 0000-0002-5018-3242

Publicado por IEASS, Editores, Venezuela

 Este libro se distribuye bajo la licencia Creative Commons: Atribución-NoComercial-SinDerivadas CC BY-NC-ND (la ”Licencia”). Usted puede utilizar este archivo de conformidad con la Licencia. Usted puede obtener una copia de la Licencia en http://creativecommons.org/licenses/by-nc-nd/3.0/ . En particular, esta licencia permite copiado y distribución gratuita, pero no permite venta ni modificaciones de este material. Límite de responsabilidad y exención de garantía: El autor o los autores han hecho su mejor esfuerzo en la preparación de este material. Esta edición se proporciona ”tal cual”. Se distribuye gratuitamente con la esperanza de que sea útil, pero sin ninguna garantía expresa o implícita respecto a la exactitud o completitud del contenido.

Primera impresión, Junio 2023

Este libro fue editado y maquetado en LaTeX y TeXstudio 3.0.4

AGRADECIMIENTOS

por TEX a

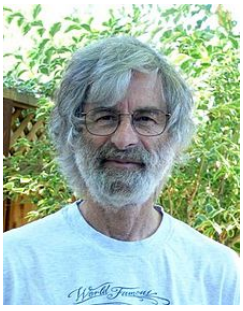
Donald E. Kunth



Donald Knuth en 2005.

por LATEX a

Leslie Lamport



ÍNDICE GENERAL

Prólogo III
Glosario V
Introducción 1

1. La Historia **5**
1.1. Inicio 5
1.2. Lenguajes asociados 7
1.2.1. C++ Builder 7
1.2.2. Visual C++ 8
1.2.3. C# 8
1.2.4. Java 8
1.3. Utilidad de C 8
1.4. El futuro 9
1.5. Estructura básica de un programa 9

2. Tipos de datos **11**
2.1. Tipos de datos 13
2.2. Operaciones básicas 15
2.3. Tipos de datos 17
2.4. Operadores de desplazamiento 20

3. Entrada y salida **21**
3.1. Modificadores de formato 22
3.2. Otras funciones muy útiles 24

4. Estructuras de control **29**
4.1. Bloques de sentencias 29
4.2. Operadores relacionales y lógicos 30
4.3. Jerarquía de operadores 31
4.3.1. ? 31
4.4. Estructuras de selección 32

4.4.1. if . . . 32
4.4.2. switch . . . 34
4.5. Estructuras de iteración . . . 35
4.5.1. while . . . 35
4.5.2. do while . . . 37
4.5.3. for . . . 38

5. Funciones **41**
5.1. Proptotipos . . . 41
5.2. Paso de argumentos . . . 42
5.3. Valores de retorno . . . 42
5.4. Macros . . . 43
5.5. Recursividad . . . 44

6. Arreglos **45**
6.1. Declaración . . . 45
6.2. Asignación de valores . . . 46
6.3. Paso de argumentos . . . 48
6.4. Arreglos multidimensionales . . . 50

7. Estructuras **53**
7.1. Enumeraciones . . . 53
7.2. Uniones . . . 54
7.3. Estructuras . . . 55

8. Clases **61**
8.1. Funciones miembro de una clase . . . 62
8.2. Constructores . . . 64
8.3. El constructor por defecto . . . 64
8.4. Destructores . . . 65
8.5. Asignación de parámetros por defecto . . . 65
8.6. Lista de inicialización . . . 65
8.7. Acceso a campos miembro privados . . . 66
8.8. Modificación de campos miembro privados . . . 66
8.9. Funciones amigas . . . 67
8.10. El puntero this . . . 67
8.11. Miembros estáticos de una clase . . . 67
8.12. Sobrecarga de operadores . . . 68
8.13. Funciones de sobrecarga miembros de la clase . . . 68
8.14. Funciones de sobrecarga no miembros de la clase . . . 68
8.15. Clases derivadas . . . 69
8.16. Archivo .h y .cpp . . . 70
8.17. Clases derivadas . . . 70
8.17.1. Constructor de instancia . . . 73

8.18. Jerarquía de clases . . . 75
8.19. Tipos de campos . . . 76
8.20. Funciones virtuales . . . 76
8.21. Constructores y destructores . . . 79
8.22. Punteros a objetos y funciones virtuales . . . 80
8.23. Referencias a objetos y funciones virtuales . . . 80
8.24. Funciones virtuales puras y clases abstractas . . . 81

9. Punteros **85**
9.1. Detalles al crear y usar punteros . . . 87
9.2. Los punteros y el ampersand . . . 87
9.3. Los apuntadores y el asterisco . . . 87
9.4. Parámetros por referencia . . . 88
9.5. Array dinámico . . . 88
9.6. Matrices dinámicas . . . 89
9.7. Usos de los punteros inteligentes . . . 91
9.8. Clases de punteros inteligentes . . . 93
9.8.1. Punteros inteligentes para objetos COM (programación clásica de Windows) . . . 94
9.8.2. Punteros inteligentes ATL para objetos POCO . . . 94
9.9. Operadores referencia-desreferencia . . . 95
9.10. Aritmética de operadores . . . 96
9.11. Arreglos de apuntadores (arreglos de cadenas) . . . 97
9.12. Paso de argumentos por referencia . . . 99

10. Memoria dinámica (Heap) **101**
10.1. Punteros colgantes (dangling pointers) . . . 102
10.2. Error al reservar la memoria dinámica . . . 103
10.3. Pérdidas de memoria (memory leaks) . . . 103
10.4. Arreglos almacenados dinámicamente . . . 104
10.5. Diferencias entre memoria dinámica y estática . . . 105

11. Apéndice **109**
11.1. Biblioteca estándar de C++ . . . 109
Bibliografía . . . 115

Índice alfabético **116**

ÍNDICE DE FIGURAS

2.1. Desplazamiento de bits. 20

6.1. Mapa de memoria para arreglos 45
6.2. Desplazamiento de punteros 49
6.3. Arreglo multidimensional 50

7.1. Espacio de memoria para Empresa 54

8.1. Relaciones entre cajas. 71
8.2. Herencia simple y múltiple 75

9.1. Entra - salida a memoria 85
9.2. Direccionamiento de apuntadores 95
9.3. Arreglo de cadena de caracteres. 97

ÍNDICE DE CUADROS

1.1. Lugar de C++ en los lenguajes de programación 6

2.1. Palabras reservadas de C++ 12
2.2. Tipos de datos de C++ 14
2.3. Secuencias de escape C++ 15
2.4. Operadores aritméticos de C++ 16
2.5. Palabras reservadas de C++ 18
2.6. Operadores de desplazamiento C++ 20

3.1. Funciones de lectura sin formato 25

4.1. Operadores relacionales en C++ 30

11.1. Entrada/Salida 110
11.2. Cadenas o Strings 110
11.3. Contenedores 111
11.4. Iteradores 111
11.5. Algoritmos 111
11.6. Números 111
11.7. Diagnóticos 112
11.8. Utilidades generales 112
11.9. Localización 112
11.10Soporte del lenguaje 113

PRÓLOGO

El objetivo de este libro es presentar las bases del lenguaje C++ para aprovechar su potencialidad.

Hoy en día son muchas las aplicaciones de la informática. En invernaderos se utilizan computadores con programas que controlan la temperatura, humedad y el riego para conseguir mejores plantas y frutos. En aviación se utilizan simuladores de vuelo para que los pilotos entrenen y programas para controlar el tráfico aéreo. Los vehículos sin conductor, el tratamiento de imágenes por computadora, la arqueología, ganadería, medicina, ciencias e ingenierías donde los usuarios de computadores de todo el mundo manejan programas de distintos tipos: procesadores de texto, hojas de cálculo, gestores de bases de datos, sistemas operativos, navegadores de Internet, juegos, reproductores de audio, de vídeo, etc. También hay computadores en los edificios inteligentes, en cajeros automáticos y en muchos otros lugares.

Pues todos los programas que están en los computadores son hechos por programadores.

La forma en la que se ha estructurado el libro ha sido, precisamente, pensando en lograr ese objetivo.

En este libro se presenta la sintaxis del lenguaje C++ a través de códigos de estudio, su ”lavado” y resaltado de segmentos importantes de éste, para ayudar a que los estudiantes se enfoquen en los conceptos clave que se presentan en cada capítulo.

Durante décadas, el Dr. Jorge Domínguez Chávez ha impartido cursos introductorios a la programación a nivel universitario, con énfasis en el uso efectivo de instrucciones de control y en la funcionalidad dentro del desarrollo de programas claramente escritos y bien estructurados.

El objetivo es claro: producir un libro de texto de programación en C++ para cursos introductorios de programación a nivel universitario, para estudiantes con poca o ninguna experiencia en el tema, con un riguroso y profundo tratamiento de la teoría y práctica que exigen los cursos tradicionales de C++.

Esta edición contiene una gran colección de ejemplos, ejercicios y proyectos sobre muchos campos,

los cuales están diseñados para dar a los alumnos la oportunidad de resolver problemas reales muy interesantes, y el código de los ejemplos del texto fue probado en varios compiladores.

El libro se concentra en los principios de la buena ingeniería de software y hace hincapié en la claridad de los programas. Somos maestros que enseñamos temas de vanguardia en salones de clases de la industria alrededor del mundo y este texto pone énfasis en la buena pedagogía.

El lector comprobará al final de esta obra que el nivel de conocimientos que ha adquirido es elevado.

Este libro posee varias características dignas de resaltar. Es breve en teoría y abundante en casos, lo que le hará aún más fácil el aprendizaje.

Al final del libro se proporciona la bibliografía incluye sólo una breve selección de aquellos textos que han tenido más relevancia para el autor. Además, de la documentación oficial del lenguaje C++.

G. Wolf
Facultad de Economía
UNAM, México

GLOSARIO

ASCII

(Código estándar estadounidense para el intercambio de información): el popular subconjunto de Unicode que representa las letras mayúsculas y minúsculas, los dígitos y algunos caracteres especiales comunes. 127 elementos.

ANSI

(American National Standards Institute) es el juego de caracteres estándar más utilizado por los equipos personales. Como estándar sólo utiliza un byte para representar un carácter, está limitado a un máximo de 256 caracteres.

Campo

Es un identificador para un dato. El campo "FechaNacimientorepresenta la fecha de nacimiento de una persona. Este campo pertenece al tipo de dato Fecha.

Componente de manejo de datos

Comprende las clases y objetos necesarios para almacenar y recuperar otros objetos del sistema. Proporciona mecanismos para la recuperación, almacenamiento de objetos, clases y estados del sistema.

Componente de interfaz humana

Menús, ventanas, control del mouse, iconos, tipo de letra y utilerías de cortar y pegar. Pantallas interactivas y reportes que representan.

Conexiones de ocurrencia

Las asociaciones o relaciones entre objetos deben ser detalladas como conexiones de ocurrencia. Estas relaciones suceden siempre entre objetos y no entre clases.

Clase

Es una categoría de objetos similares. Es un conjunto de objetos relacionados.

Jerarquía de comando

Es una estructura que se planea cuidadosamente: los comandos de alto nivel están dirigidos hacia los usuarios de bajo nivel.

Encapsulacion

Ocultamiento del estado de los datos miembro, de manera que sólo se cambien mediante las operaciones definidas para ese objeto.

Generalización-especializacion

Es un tipo básico de estructura a ser impuesto en las clases y objetos. Siempre conectan clases a clases. Son típicamente una forma jerárquica en el intercambio entre las computadoras del sistema y las personas que lo utilizan.

Hardware

Equipo físico de un computador formado por la unidad central de proceso (CPU) y los periféricos.

Herencia

Habilidad de crear nuevas clases (derivadas) basadas en parte en clases padre.

Herencia múltiple

Crear una clase derivada en forma tal que herede todos los atributos y comportamientos de mas de una clase padre.

Informe

Es un listado ordenado de los campos y registros seleccionados en un formato fácil de leer. Generalmente se usan como peticiones expresas de un tipo de información por parte de un usuario. Por ejemplo, un informe de las facturas impagadas del mes de enero ordenado por nombre de cliente.

Lenguaje C y C++

C es un lenguaje de programación estructurado, mientras que C++ es un lenguaje de programación orientados a objetos, basado en C. En el mercado de los lenguajes de programación, muchos fabricantes combinan C y C++ en un sólo producto, en lugar de ofrecerlos por separado.

Mensaje

Es la forma en que los objetos se comunican entre si.

Metodología

Es un conjunto de procedimientos, técnicas y ayudas a la documentación para el desarrollo de un producto software.

Objeto

Es una representación en computadora de alguna cosa o evento del mundo real.

Orientado a objetos

Un lenguaje orientado a objetos tiene tres características básicas: debe estar basado en objetos, en clases y en herencia de clases. Muchos lenguajes cumplen uno o dos de estos puntos; muchos menos cumplen los tres. La barrera más difícil es usualmente la herencia.

Plantilla de especificación

Proporcionan detalles de análisis adicional para dar soporte al paquete de diagrama en capas. Deben proporcionar un esquema similar al diccionario de datos. La plantilla puede ser luego expandida y modificada conforme continua el análisis.

POO

Programación orientada a objetos.

Polimorfismo

En POO se denomina polimorfismo a la capacidad que tienen los objetos de diferentes clases de responder al mismo mensaje. **Componentes de problema**

Prototipo

Es un modelo (representación, demostración o simulación) fácilmente ampliable y modificable de un sistema planificado, probablemente incluyendo su interfaz y su funcionalidad de entradas y salidas.

RAII

Resource Acquisition Is Initialization.

Servicio

Es el comportamiento de los objetos. Son métodos o procedimientos, que llegan a ser parte de los objetos, en forma similar a los atributos e involucran cambios en el estado de un objeto.

Software

equipo lógico de un computador, compuesto por programas, lenguajes de programación, documentación, etc.

TDA

Tipo abstracto de dato. Son datos alfanuméricos y numéricos combinados para obtener tipos de datos más elaborados.

Tipo de Dato

Indica la naturaleza del campo. Los datos numéricos son aquellos con los que se realizan cálculos aritméticos (sumas, restas, multiplicaciones...) y los alfanuméricos, que son los que contienen caracteres alfabéticos, símbolos nacionales y numéricos.

INTRODUCCIÓN

El objetivo de este libro es presentar los conceptos y mecanismos básicos del lenguaje de programación.

El presente trabajo consta de 10 capítulos, con casos prácticos para explicar los conceptos descritos. Recuerde, la práctica es recomendable conforme avance en la lectura de este texto. Cada caso está explicado en detalle, para entender el funcionamiento de cada sección de código.

I.- Este capítulo presenta el lenguaje C++, cuenta su historia y su relación con otros lenguajes de programación similares. Luego, veremos una descripción de la forma básica de un programa.

II.- En este capítulo entramos en el aprendizaje del lenguaje, definimos la estructura de un programa, los tipos de datos y sus operadores. Aunque en los ejemplos escritos en este texto no los usemos, si los podemos utilizar en cualquier momento para realizar operaciones complejas de una forma rápida.

III.- Este capítulo trata de la entrada y salida básica. Es necesario conocer la forma de comunicamos con el usuario de nuestro programa. Los programas reciben datos del teclado, los procesan y muestran el resultado en la pantalla del computador. Aprenderemos a redactar mensajes sucintos, claros en la pantalla de fácil comprensión.

IV.- En este capítulo estudiamos las estructuras de control. Las cuales gestionan el flujo de proceso en nuestro programa, veremos el uso de los bucles y la toma de decisiones.

En C++ el concepto de bloque de sentencias se utilizar para agrupar un conjunto de sentencias dentro de un ámbito concreto del programa. Un bloque de sentencias es un conjunto de instrucciones englobadas bajo llaves: { y } .

V.- Este capítulo trata de las funciones, un elemento muy utilizado en la programación. Una función es un conjunto de instrucciones que ejecutan una tarea única y devuelven un resultado; además, es un segmento de programa encapsulado en un formato estándar para ser sencillo de manipular, reutilizar, reutilizable y reduce el código a escribir.

Presentamos casos prácticos de su uso, describimos sus datos y comportamiento. También, estudiamos la recursividad y su forma de operar.

VI.- En este capítulo estudiamos los arreglos[1] y las cadenas de texto, por medio de esquemas explicamos el funcionamiento de este tipo de datos.

Un arreglo (matriz) es una colección ordenada de datos (tanto primitivos u objetos dependiendo del lenguaje). Los arreglos (matrices) se emplean para almacenar múltiples valores en una sola variable, frente a las variables que sólo pueden almacenar un valor (por cada variable).

Cada elemento del arreglo (matriz) tiene un número al que está asociado, llamado ”índice numérico”, para acceder a él.

Se proporcionan casos simple para mejor comprensión, como cifrar una frase, una contraseña, para que tenga una idea de lo que puede hacer con su código.

VII.- Este capítulo trata de apuntadores, quizás el tema que más trabajo cuesta entender, para facilitar su comprensión presentamos esquemas de comportamiento, utilización de la memoria. Explicamos los arreglos de apuntadores y el código de un juego simple, pero que puede ser mejorado bastante por el lector.

Un puntero es una variable que almacena la dirección de memoria de un objeto. Los punteros se usan ampliamente en C y C++ para tres propósitos principales: para asignar nuevos objetos en el montón, para pasarlos de unas funciones a otras.

Los punteros en C++ (o apuntadores) son quizá uno de los temas que más confusión causan al momento de aprender a programar en C++, sin embargo verás que no es para tanto y que todo depende de dos elementos: el signo & (ampersand) y el * (asterisco) que los explicaré en breve. Durante este artículo verás entonces que no es para nada difícil hacer y usar punteros y que además son de gran ayuda al momento de necesitar valores y estructuras dinámicas, por ejemplo, para crear un array dinámico, con lo cual me refiero a que su tamaño puede ser establecido en tiempo de ejecución y lo mismo se puede hacer con las matrices (que en realidad son un array multidimensional).

VIII.- Este capítulo explica las características y uso de las estructuras básicas como union, enumeración y estructura. Con ejemplos interesantes y explicados detalladamente, animamos al lector a que los complete y/o los mejore para que le sean de mucha más utilidad.

Las uniones y las estructuras tienen propósitos muy diferentes, son objetos excluyentes.

Una unión es un tipo de datos en el que conviven varios tipos de datos dentro de un mismo espacio de memoria. Una estructura es un bloque compuesto por varios campos, cada uno ocupando su propio espacio de memoria dentro del todo.

La enumeración es un tipo definido por el usuario que consta de un conjunto de constantes enteras con nombre conocidas como enumeradores. Este artículo trata el tipo enum del lenguaje C++ estándar de ISO y el tipo de ámbito (o fuertemente tipado) enum class , que se introdujo en C++11.

[1] arrays

IX.- En este capítulo estudiamos las clases. Vemos la declaración de la clase, su nombre y establecemos sus atributos y métodos, así como la visibilidad de estos dos últimos ante el exterior.

Una clase en C++ puede pensarse como un tipo de dato abstracto (TDA), es decir, un conjunto de datos y las operaciones asociadas a esos datos. En términos de objetos, el conjunto de datos equivale a los atributos de la clase y las operaciones o algoritmos que se aplican a dichos datos serían los métodos.

X.- Este capítulo estudia la memoria. La estática tiene una duración fija, que se reserva y libera de forma automática. En contraste, la dinámica se reserva de forma explícita y continúa existiendo hasta que sea liberada, generalmente por parte del programador. La memoria dinámica afecta el rendimiento.

Todos los objetos tienen un tiempo de vida, es decir, período durante el cual se garantiza que el objeto existe. Las variables globales y las locales declaradas con el especificador static tienen duración estática.

La memoria dinámica es un espacio de almacenamiento que se puede solicitar en tiempo de ejecución. Además, de requerir espacios de almacenamiento, también podemos liberarlos (en tiempo de ejecución) cuando dejemos de necesitarlos.

Todo esto se debe a las nuevas prestaciones de la tecnología informática, a las cada vez mejores herramientas de desarrollo, y por supuesto, a los diseñadores y desarrolladores de soluciones software.

Hoy en día, hay entornos de programación que realizan está labor en un conjunto integrado, posiblemente usted trabaje con uno de ellos. En general, de todas las funcionalidades que tienen esos entornos usted sólo va a necesitar unas pocas; por lo tanto le aconsejamos que no pierda mucho el tiempo en aprender su manejo hasta que realmente no necesite hacer cosas muy potentes.

Trabaje con un único archivo .cpp y a lo sumo un archivo .h, el resto lo pone todo el entorno.

Construya y ejecute las aplicaciones, no necesita nada más. Bueno, un poco de suerte.

CAPÍTULO 1

LA HISTORIA

1.1. Inicio

Todo comienza con Dennis Ritchie en los laboratorios de AT&T Bel a principios de los años 70.

Dennis trabajó con un lenguaje llamado BCPL desarrollado por Martin Richards (que influyó para crear B de Ken Thompson), Dennis deseaba un lenguaje para manejar el hardware de la misma manera que el ensamblador pero con programación estructurada como los lenguajes de alto nivel.

Entonces, creó el lenguaje C que corría en computadoras PDP-7 y PDP-11 con el sistema operativo UNIX. Pero los verdaderos alcances de lo que sería éste, se verían poco tiempo después cuando Dennis volvió a escribir el compilador C de UNIX en el mismo C, y luego Ken Thompson (diseñador del sistema) escribió UNIX completamente en C y no en ensamblador.

Al momento que AT&T cedió (a un precio bastante bajo) el sistema operativo a varias universidades, el auge de C comenzaba. Cuando aparecieron las computadoras personales comerciales y empezaron a liberarse varias versiones de compiladores C, éste se convirtió en el lenguaje favorito para crear aplicaciones.

En 1983, el Instituto Americano de Normalización (ANSI) estandarizo el lenguaje C, tarea tardó 6 años en completarse, y con la ayuda de la Organización Internacional de Normalización (ISO), definió el C Estándar en el año de 1989.

El lenguaje de programación C++ fue desarrollado por Bjarne Stroustrup en 1980. Stroustrup trabajaba en los laboratorios Bell de AT&T y creó C++ como una extensión del lenguaje de programación C.

El nombre ”C++” se debe a que la nueva versión del lenguaje incluía características adicionales que iban más allá de las capacidades de C. ”C++” se refiere al operador de incremento en C, que agrega 1 al valor de una variable.

C++ permite la creación de clases y el uso de herencia, polimorfismo y otros conceptos de la

programación orientada a objetos. Ésta, es un paradigma que organiza el código en estructuras llamadas ”clases” que encapsulan datos y funciones relacionadas.

El primer estándar oficial de C++ fue publicado en 1998, conocido como C++98. Desde entonces, se han lanzado varias revisiones y extensiones del estándar, como C++03, C++11, C++14, C++17 y C++20. Estas revisiones han agregado nuevas características y mejoras al lenguaje.

C++ se ha convertido en uno de los lenguajes de programación más popular y ampliamente utilizado en el mundo, con una amplia variedad de aplicaciones, desde sistemas operativos y controladores de dispositivos hasta software de videojuegos y aplicaciones empresariales. C++ es conocido por su eficiencia y su capacidad para acceder directamente a los recursos de hardware, lo que lo hace adecuado para aplicaciones que requieren un alto rendimiento.

Los objetivos de C++ es ayudar a los programadores a desarrollar sistemas operativos y cuenta con tres características: escribir comportamiento complejo sin mucho esfuerzo, construcciones de alto nivel y control sobre el hardware las que hacen a C uno de los lenguajes más confiables en la actualidad.

La base del lenguaje fue creada por y para programadores, a diferencia de otros lenguajes como Basic o Cobol que fueron creados para que los usuarios resolvieran pequeños problemas de sus computadores y otros para que los no programadores pudiesen entender partes del programa.

C++ es un lenguaje de nivel medio, porque combina la programación estructurada de los lenguajes de alto nivel con la flexibilidad del ensamblador. La tabla 1.1 muestra el lugar del lenguaje respecto a otros.

Cuadro 1.1: Lugar de C++ en los lenguajes de programación

Alto nivel	Nivel medio	Nivel bajo
Ada, Modula-2, Pascal, Cobol, FORTRAN, Basic	Java, C++, C, FORTH, Macroensamblador	Ensamblador

Son ejemplos de lenguajes no estructurados y estructurados:

No estructurados	Estructurados
FORTRAN, Basic, Cobol	Pascal, Ada, C++, C, Java

En la actualidad, los lenguajes no estructurados han sido modificados para cumplir con la característica de la programación estructurada, tal es el caso de Basic, al referirnos a lenguaje estructurado pensamos en funciones y en sentencias de control (if, while, etc.).

C++, Pascal y Java también son lenguajes orientados a objetos.

Algunos sistemas operativos importantes, como Windows, Freebsd, Leopard y Linux, tienen

componentes escritos en C++ debido a su rendimiento y flexibilidad. Aquí hay algunas formas en las que C++ se utiliza en relación con los sistemas operativos:

1. Desarrollo del núcleo del sistema operativo: El núcleo o kernel de un sistema operativo administra los recursos del sistema y proporciona servicios de bajo nivel. En algunos sistemas operativos, como Linux, partes del núcleo están escritas en C++. C++ permite un control directo del hardware y la optimización de código, lo que es crucial para el rendimiento y la eficiencia del sistema operativo.

2. Bibliotecas de sistemas operativos: Los sistemas operativos proporcionan una serie de bibliotecas que ofrecen funciones y servicios a las aplicaciones. Estas bibliotecas se pueden implementar en C++ para aprovechar las ventajas de rendimiento y eficiencia del lenguaje. Un ejemplo popular es la biblioteca de Windows API (Application Programming Interface), que se utiliza para desarrollar aplicaciones de Windows y se implementa en gran medida en C++.

3. Controladores de dispositivo: Los controladores de dispositivo son programas que permiten que el sistema operativo interactúe con hardware específico, como impresoras, tarjetas gráficas o dispositivos de red. En algunos casos, los controladores de dispositivo se escriben en C++ debido a su capacidad para acceder directamente a los recursos del hardware y su eficiencia.

4. Desarrollo de aplicaciones del sistema: Además del desarrollo del núcleo del sistema operativo, C++ se utiliza ampliamente para crear aplicaciones del sistema, como herramientas de línea de comandos, utilidades del sistema y programas de administración. La potencia y la flexibilidad de C++ permiten el desarrollo de aplicaciones de alto rendimiento y bajo nivel que interactúan directamente con el sistema operativo.

C++ es un superconjunto de C, cualquier compilador de C++ es capaz de compilar un programa en C. De hecho la mayoría admite tanto código en C como en C++ en un archivo. Por esto, la mayoría de desarrolladores compilan con C++ su código escrito en C, incluso hay quienes, siendo código en C ponen la extensión CPP (extensión de los archivos de código C++) y lo compilan con C++ (hasta hace unos días yo hacía esto), lo cual no es recomendable por norma al programar.

Cuando se compila código C en un compilador C++ este debe cumplir con los estándares definidos en 1989, cualquier palabra definida en el estándar de 1999 no funcionará.

La evolución de C++ continúa, y la diversidad de compiladores contribuye a ello.

1.2. Lenguajes asociados

1.2.1. C++ Builder

Creado por una empresa de nombre Inprise, sobradamente conocida por la calidad de sus herramientas de desarrollo, que llevan la firma Borland, entre las que están Borland C++, Intrabuilder, JBuilder, Delphi, C++ Builder.

C++ Builder surgió de la fusión de dos tecnologías: Borland C++ que soporta el lenguaje C++ estándar con todas sus novedades, y el entorno RAD de Delphi.

1.2.2. Visual C++

En el año de 1992, la compañía Microsoft introduce C/C++ 7.0 y la biblioteca de clases MFC, los cuales tenían la finalidad de que el desarrollo de aplicaciones para Windows, escritas en C, fuera más fácil. Sin embargo, no resultó como esperaban, así que un año más tarde fue creado Visual C++ 1.0, que parecía más amigable a los desarrolladores, porque tenía una versión mejorada de las clases MFC.

Con Visual C++ se introdujo una tecnología de desarrollo a base de asistentes. En general es un entorno de desarrollo diseñado para crear aplicaciones gráficas orientadas a objetos. Pero si cree que se trata sólo de arrastrar componentes hacia un ”formulario”, como lo puede hacer en Visual Basic, está muy equivocado. La programación en él es más compleja que eso.

1.2.3. C#

Este lenguaje es una evolución de C y C++. Creado por Microsoft y presentado como Visual C# en el conjunto de Visual Studio .NET. Está diseñado para crear una amplia gama de aplicaciones empresariales. La biblioteca para programar en este lenguaje es .NET Framework. El lenguaje es simple, moderno, y está orientado a objetos.

El código de C# se compila como código administrado, lo que significa que se beneficia de los servicios de Common Language Runtime. Estos servicios incluyen la interoperabilidad entre lenguajes, la recolección de elementos no utilizados, mayor seguridad y mejor compatibilidad entre las versiones.

1.2.4. Java

El lenguaje de programación Java se había creado como una plataforma para desarrollo de sistemas de información para Internet y para aparatos electrodomésticos. Fue creado con base en C++. Poco tiempo después de su liberación (en 1995), se empiezan a ver las capacidades de este lenguaje. Pronto deja de ser un lenguaje que sólo se usaba en Internet, y empiezan a crearse programas completos con él.

El lenguaje es orientado a objetos y multiplataforma, esto quiere decir que el código se puede transportar a diferentes sistemas operativos y ejecutarse en ellos por medio de la maquina virtual de Java.

Al igual que con C++ existen varios compiladores para este lenguaje como lo son JBuilder y Visual J++ (ahora evolucionado a J#).

1.3. Utilidad de C

C es utilizado en el kernel de Linux, de los BSDs, de macOS y la API de Windows (WinAPI) está exportada como funciones y estructuras C. El kernel de un sistema operativo es

un espacio cercano al hardware que se programa mediante el uso de ensamblador y lenguaje C.

Así tenemos: Linux, Minix, Hurd, FreeBSD, OpenBSD, y Darwing, el kernel de Mac OS X. Linux sigue gobernando el 100 % de las 500 supercomputadoras más rápidas del mundo, un monopolio que muestra el poder de este sistema operativo a la hora de ser utilizado en estos entornos.

Proyectos modernos implementados en C: a) PostgreSQL, Mariadb, Mysql, SQLite3 b) GIMP, c) Versionado: Git, Subversion, d) Servidores Web: Apache, Nginx, e) Editores: Emacs, VIM, Geany, f) Matemáticas: R, MATLAB, Octave, g) Aplicaciones: redis, memcached, GTK+, Gnome, xfce, cURL, Grandes partes de ReactOS y h) muchos virus informáticos se crean en C porque aprovecha vulnerabilidades en los programas para crear sus exploits.

C además, tiene otras características que lo hacen único entre todos los lenguajes de programación, ya que existe un compilador de C para todas las plataformas actuales y para muchas legadas. C es la lengua franca de todos los lenguajes de alto nivel.

Además de todo eso, existen millones de líneas de código escritas en C desde hace décadas. Reemplazar todo eso por código escrito en un lenguaje actual (sin ninguna o muy poca ventaja aparente) no tiene sentido ni es rentable económicamente.

1.4. El futuro

Es un lenguaje especial para aprender cómo funcionan las computadoras y cómo funcionan otros lenguajes de alto nivel. Una buena parte de los intérpretes de los lenguajes más utilizados en la actualidad, como Python y Java, están escritos en C/C++. Sólo por esto, ya vale la pena aprenderlo.

1.5. Estructura básica de un programa

```
1  \#include <iostream.h>
2  int main(void){
3    cout<<"hola mundo"<<endl;
4    return 0;
5  }
```

Analicemos cada parte de nuestro primer programa.

#include < iostream.h > se refiere a la biblioteca de funciones a utilizar. Para llamar a una biblioteca en particular hacemos lo siguiente:

#include < librera_solicitada >. Esta línea incluye la biblioteca iostream, que proporciona las funciones de entrada y salida estándar en C++.

El C++ incluye varias bibliotecas de funciones, y dependiendo del compilador usado, aumenta el número.

intmain(void) todo programa C++ es una función main() y sólo puede haber una. Es la función principal del programa y es el punto de entrada para la ejecución del programa.

En C++, main() regresa un entero, por eso se antepone ”int” a la palabra ”main”. Los paréntesis que le siguen delimitan el bloque de instrucciones que forman la función. Si usamos la palabra ”void” significa vacío, es decir que la función main no recibe parámetros, si se omite, el compilador asume que no se envía nada. La llave que abre significa que se inicia un bloque de instrucciones.

La instrucción *cout* << ”*holamundo*” << *endl* donde cout está definida en la biblioteca iostream.h, que previamente declaramos. Una función, main(), siempre comienza su ejecución con una instrucción (la primera), y continúa así hasta que se llegue a la última instrucción. Cada instrucción termina en ”;”.

CAPÍTULO 2

TIPOS DE DATOS

En el capítulo anterior mostramos la forma general de un programa C++. En el siguiente programa:

```
#include <iostream.h>
int main( ){
  int variable;
  variable=5;
  cout<<variable;
  return 0;
}
```

Notése la parte: int variable y a esta sección le denominamos declaración, donde declaramos la(s) variables a utilizar en la función. En esta caso se trate de una variable de nombre ”variable”.

Una variable es una posición de memoria con un nombre que se usa para mantener un valor a ser modificado por el programa. Las variables son declaradas, usadas y liberadas. Una declaración se encuentra ligada a un tipo, a un nombre y a un valor.

Básicamente, la declaración de una variable presenta el siguiente aspecto: tipo nombre [=valor];

Los corchetes significan que esa parte es opcional.

Sea $intmi_variable = 5$; que declara una variable tipo entero de nombre ”mi_variable” y le asigna el valor ”5”.

C++ es sensible a mayúsculas y minúsculas, así que si el nombre de la variable empieza con una letra en mayúscula, durante el resto del código nos referirnos a ella exactamente como la escribimos. Los nombres de las variables NO pueden usar signos de puntuación, sólo caracteres ”A-Z”, ”a-z”, ”_”, ”0-9”, NO debe comenzar con un número (0-9). Y NO se deben de repetir en el mismo contexto.

Además de las restricciones anteriores, existe otra, NO podemos usar como nombre de variables a las palabras reservadas del lenguaje C++.

La lista 2.1 presenta las palabras reservadas que no pueden ocuparse como nombres de

variables, funciones, u otras instrucciones definidas por el programador.

Cuadro 2.1: Palabras reservadas de C++

and	and_eq	asm	auto
bitand	bitor	bool	break
case	match	char	class
compl	const	const_cast	continue
default	delete	do	double
dynamic_cast	else	enum	explicit
export	extern	false	float
for	friend	goto	if
inline	int	long	mutable
namespace	new	not	not_eq
operator	or	or_eq	private
protected	public	register	reinterpret_cast
return	short	signed	sizeof
static	static_cast	struct	switch
template	this	throw	true
try	typedef	typeid	typename
union	unsigned	using	virtual
void	volatile	wchar_t	while
xor	xor_eq		

Las variables se pueden declarar en tres sitios básicos: dentro de las funciones (ya sea la función main u otras creadas por el programador), variables llamadas locales; en la definición de parámetros de una función y fuera de ella, variables llamadas globales.

Ejemplo:

```
#include <iostream.h>
int variable_global=10;
int main(){
  int variable_local=20;
  cout<<"\nprograma que muestra los usos de variables "\
  "globales y locales\n"<<endl;
  cout<<"la variable global tiene asignado un: "\
  <<variable_global<<endl;
  cout<<"\nla variable local tiene asignado un: "\
  <<variable_local<<endl;
  return 0;
}
```

Una variable global puede ser modificada en cualquier parte del programa, mientras que una variable local sólo puede ser modificada y utilizada dentro del ámbito de su función. Recuerde, antes de utilizar una variable y hacer operaciones con ella, debe declararla.

Por lo general, evite el utilizar la declaración de variables globales. El siguiente caso muestra que las variables se pueden en cualquier parte del programa, siempre y cuando se declaren antes de usarlas.

```
#include <iostream.h>
int main( ){
  int variable1=10;
  cout<<"la variable 1 local tiene almacenado un: "\
  <<variable1<<endl;
  variable1=50;
  int variable2=variable1+30;
  cout<<"\nla variable 2 almacena un: "\
  <<variable2<<endl;
  return 0;
}
```

En un programa declaramos una variable y asignarle un valor, que no pueda ser modificado. En este caso, declaramos una constante.

El siguiente programa calcula el área de un círculo.

```
#include <iostream.h>
int main( ){
  const float pi=3.141592;
  int radio=5;
  float area;
  area=pi*radio*radio;
  cout<<"el area del circulo es: "<<area<<endl;
  return 0;
}
```

Declaramos una constante del tipo float, le asignamos el nombre ”pi” y el valor 3.141592. Este valor NO podrá ser modificado en el programa.

La declaración de una constante presenta el siguiente aspecto: $const tipo nombre = valor$;

2.1. Tipos de datos

Las variables utilizadas en los programas presentados han sido de tipo entero (int), pero es ilógico pensar que éste sea el único a utilizar. Además del tipo entero existen otros.

Los tipos de datos atómicos son los tipos de datos más sencillos a partir de los cuales se pueden construir otros más complejos. La tabla 2.2 muestra estos tipos con sus intervalos de valores posibles y el número de bytes que ocupan.

Cuadro 2.2: Tipos de datos de C++

Tipo	Tamaño en bytes	Intervalo
short	2	-32768 a 32767
unsigned short	2	0 a 65535
long	4	-2147483648 a 2147483647
long unsigned	4	0 a 4294967295
int	dependiendo del compilador utilizado podría ser 2 o 4	-32768 a 32767
unsigned int	2 o 4	0 a 65535
float	4	1.17549535e-38 a 3.402823466e+38 con 8 cifras decimales
double	8	2.2250738585072014e-308 a 1.7976931348623158e+308 con 16 cifras decimales
long double	10	
char	1	-128 a 127
unsigned char	1	0 a 255
bool	1	Valor lógico o booleano que puede ser trae (cierto) o false (falso).

En la biblioteca $< limits.h >$ encuentra constantes definidas para los intervalos que almacenan algunos tipos de variables.

Compile y ejecute el siguiente programa, después consulte la biblioteca $< limits.h >$ para ver más constantes definidas.

```
#include <limits.h>
#include <iostream.h>
int main( ){
  cout<<"PROGRAMA QUE MUESTRA LOS VALORES MAXIMOS Y "\
  "MINIMOS \n DE ALGUNOS DE LOS TIPOS DE DATOS "\
  "ATOMICOS"<<endl;
  cout<<"\n int maximo: "<<INT_MAX<<" int minimo: "\
  <<INT_MIN<<endl;
  cout<<"\n char maximo: "<<CHAR_MAX<<" char minimo: "\
  <<CHAR_MIN<<" tamaño en bits: "<<CHAR_BIT<<endl;
  cout<<"\n long maximo: "<<LONG_MAX<<" long minimo: "\
  <<LONG_MIN<<endl;
  cout<<"\n short maximo: "<<SHRT_MAX<<" short minimo: "\
  <<SHRT_MIN<<endl;
  return 0;
}
```

Por medio de los programas presentados damos una idea de la forma en que se asignan valores a los tipos de datos numéricos. Sólo falta mostrar como se pueden asignar valores en formato octal y hexadecimal. Las siguientes instrucciones lo aclaran.

```
int
variable1=022;
int variable2=0x12;
```

Ambas variables almacenan el número 18, la primera en octal y la segunda en hexadecimal. También, se pueden asignar números exponenciales.

$floatvariable3 = 2e6$;

Pero no sabemos nada sobre los de tipo char.

Para declarar y asignar un carácter a una variable se hace de la siguiente forma.

char nombre = ”[carácter]”;

Y si es una constante sólo se agrega la palabra const al principio de la declaración.

Note que el carácter asignado está encerrado entre comillas simples. Así, la siguiente declaración asigna la letra ”A” mayúscula a una constante de nombre ”inicial”.

const char inicial=”A”;

El valor (carácter) que se almacena en una variable o constante de tipo char, es el valor entero del carácter en el juego de caracteres ASCII. Entonces a la constante ”inicial” se le asignó un valor de 65.

Para representar caracteres de control es necesario usar secuencias de escape. Vea la tabla 2.3.

Cuadro 2.3: Secuencias de escape C++

Carácter	Significado
\n	salto de línea
\t	tabulador horizontal
\v	tabulador vertical
\a	alerta sonora
\0	carácter nulo
\b	mueve el cursor hacia atrás

Cualquier carácter se representa con ”\” seguida del propio carácter o del código octal o hexadecimal. Para las comillas dobles es ”\” o ”\042” o ”\x22”.

2.2. Operaciones básicas

En casos anteriores, usamos de algunos operadores, ”+” y ”*”, suma y multiplicación respectivamente. La tabla 2.4 muestra los operadores aritméticos.

Cuadro 2.4: Operadores aritméticos de C++

Operador	Significado
+	suma
-	resta
*	multiplicación
/	división
%	módulo o resto de la división entera

El siguiente programa lo muestra.

```
#include <iostream.h>
int main( ){
  int a=5, b=10, c=20, r;
  r=a+b;
  a=c % r;
  c=b-a;
  a = a*2;
  cout<<"a="<<a<<" b="<<b<<" c="<<c<<" r="<<r<<endl;
  cout<<"la suma de a y b es: "<<a+b<<endl;
  return 0;
}
```

Estos operadores no son exclusivos de los tipos de datos numéricos. Un dato tipo char también es modificable mediante operadores. Recuerde que almacena el código ASCII.

```
#include <iostream.h>
int main( ){
  char a='a', b;
  b=a+15;
  cout<<"b="<<b<<endl;
  cout<<"a="<<b-10<<endl;
  cout<<"la suma de los dos es:"<<a+b<<endl;
  cout<<"a="<<a-b<<endl;
  return 0;
}
```

En estos dos últimos casos, note las siguientes instrucciones: $a = a * 2$;

$car1 = car1car2$;

Ambas pueden escribirse de una mejor forma:

$a* = 2$;

$car1 = car2$;

Podría decirse que es una abreviatura de código. El operador seguido de la asignación.

Ahora, conozcamos a otros operadores útiles. ”++” y ”–”. Éstos, aumentan y disminuye en uno al dato que opera. Pero su comportamiento depende de si sea en prefijo (antes del operando) o sufijo (después de).

2.3. Tipos de datos

En el capítulo anterior vimos la forma general de un programa C++. En el siguiente programa:

```
1  #include <iostream.h>
2  int main( ){
3    int variable;
4    variable=5;
5    cout<<variable;
6    return 0;
7  }
```

notamos la parte: int variable y a esta sección le denominamos declaración, donde declaramos la(s) variables a utilizar en la función. En esta caso se trate de una variable de nombre ”variable”.

Una variable es una posición de memoria con nombre que se usa para mantener un valor que puede ser modificado por el programa. Las variables son declaradas, usadas y liberadas. Una declaración se encuentra ligada a un tipo, a un nombre y a un valor.

Básicamente, la declaración de una variable presenta el siguiente aspecto: tipo nombre [=valor];

Los corchetes significan que esa parte es opcional.

Sea $intmi_variable = 5$; que declara una variable tipo entero de nombre ”mi_variable” y le asigna el valor ”5”.

C++ es sensible a mayúsculas y minúsculas, así que si el nombre de la variable empieza con una letra en mayúsculas, debemos de asegurarnos que durante el resto del código nos refiramos a ella exactamente como la escribimos. Los nombres de las variables no pueden usar signos de puntuación, sólo caracteres ”A-Z”, ”a-z”, ”_”, ”0-9”, un nombre NO debe comenzar con un número (0-9). Además, los nombres de variables no se deben de repetir en el mismo contexto.

Además de las restricciones anteriores, existe otra, NO podemos usar como nombre de variables a las palabras reservadas del lenguaje C++. La lista 2.5 presenta las palabras reservadas que no pueden ocuparse como nombres de variables, funciones, u otras instrucciones definidas por el programador.

Cuadro 2.5: Palabras reservadas de C++

and	and_eq	asm	auto
bitand	bitor	bool	break
case	match	char	class
compl	const	const_cast	continue
default	delete	do	double
dynamic_cast	else	enum	explicit
export	extern	false	float
for	friend	goto	if
inline	int	long	mutable
namespace	new	not	not_eq
operator	or	or_eq	private
protected	public	register	reinterpret_cast
return	short	signed	sizeof
static	static_cast	struct	switch
template	this	throw	true
try	typedef	typeid	typename
union	unsigned	using	virtual
void	volatile	wchar_t	while
xor	xor_eq		

Las variables se pueden declarar en tres sitios básicos: dentro de las funciones (ya sea la función main u otras creadas por el programador), variables llamadas locales; en la definición de parámetros de una función y fuera de ella, variables llamadas globales.

El siguiente programa muestra los usos de variables globales y locales:

```
#include <iostream.h>
int variable_global=10;
int main(){
  int variable_local=20;
  cout<<"\nprograma que muestra los usos de variables "\
  "globales y locales\n"<<endl;
  cout<<"la variable global tiene asignado un: "\
  <<variable_global<<endl;
  cout<<"\nla variable local tiene asignado un: "\
  <<variable_local<<endl;
  return 0;
}
```

Una variable global puede ser modificada en cualquier parte del programa, mientras que una variable local sólo puede ser modificada y utilizada dentro del ámbito de su función.

Recuerde, antes de utilizar una variable y hacer operaciones con ella, debe declararla.

Por lo general, evite el utilizar la declaración de variables globales.

El siguiente caso muestra que las variables se utilizan en cualquier parte del programa, siempre y cuando se declaren antes de usarlas.

```
#include <iostream.h>
int main( ){
  int variable1=10;
  cout<<"la variable 1 local tiene almacenado un: "\
  <<variable1<<endl;
  variable1=50;
  int variable2=variable1+30;
  cout<<"\nla variable 2 almacena un: "\
  <<variable2<<endl;
  return 0;
}
```

En un programa, declaramos una variable y le asignamos un valor, que no pueda ser modificado. En este caso, declaramos una constante.

El siguiente programa calcula el área de un círculo.

```
#include <iostream.h>
int main( ){
  const float pi=3.141592;
  int radio=5;
  float area;
  area=pi*radio*radio;
  cout<<"el area del circulo es: "<<area<<endl;
  return 0;
}
```

Declaramos una constante del tipo float, le asignamos el nombre ”pi” y el valor 3.141592. Este valor NO podrá ser modificado en el programa.

La declaración de una constante presenta el siguiente aspecto: *consttiponombre* = *valor*;

```
#include <iostream.h>
int main( ){
  int a,b,c;
  a = 17 * 5 + 2;
  cout<<"a="<<a<<endl;
  b =a++ - 7;
  cout<<"b="<<b<<endl;
  c=++b * --a;
  cout<<"c="<<c<<endl;
  return 0;
}
```

También, puede utilizar los paréntesis (como propiedad asociativa) para indicar al programa hacia qué operaciones debe de hacer primero. Hasta aquí, podría considerarse que se han visto las operaciones básicas (suma, resta, multiplicación y división). Pero no son las únicas. Aunque los siguientes operadores no le parezcan importantes o no los comprenda por completo, pronto se dará cuenta de sus usos.

2.4. Operadores de desplazamiento

Cuadro 2.6: Operadores de desplazamiento C++

Operador	Significado
<	Desplazamiento de bits a la izquierda
>	Desplazamiento de bits a la derecha

Desplazar los bits de un número entero hacia la derecha equivale a dividirlo por 2. Veamos por qué.

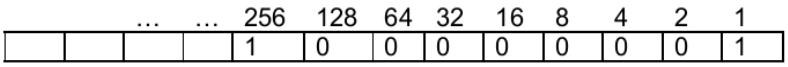


Figura 2.1: Desplazamiento de bits.

CAPÍTULO 3

ENTRADA Y SALIDA

En los programas anteriores, utilizamos la instrucción *cout* << para enviar mensajes a la pantalla.

La mayoría de los programas en C++ incluyen el archivo de encabezado < *iostream.h* >, el cual contiene la información básica requerida para todas las operaciones de entrada y salida (E/S) de flujo.

Cuando usamos la instrucción: *cout* << "*Mensajealapantalla*" << *endl*; estamos enviando una cadena de caracteres ("Mensaje a la pantalla") al dispositivo de salida estándar (la pantalla). Luego, el manipulador de flujo endl da el efecto de la secuencia de escape '\n'.

Pruebe el siguiente programa:

```
#include <iostream.h>
int main(){
  cout<<"cadena de caracteres"<<endl;
  cout<<2+2<<endl;
  cout<<9/2<<endl;
  cout<<(int)(3.141592+2)<<endl;
  return 0;
}
```

La instrucción *cout* << imprime tanto números enteros como reales sin especificar el tipo de dato del que se trata, pero una cadena de caracteres debe estar entre comillas. En casos anteriores, enviamos a la pantalla el valor de una variable, así que no hace falta más explicación al respecto.

La interacción con el usuario es muy importante en la programación, si nos solicitan hacer un programa que calcule la distancia de vuelo de un proyectil lanzado a determinada velocidad y ángulo, o un programa que calcule las raíces de una ecuación cuadrática. Es molesto estar cambiando los valores de las variables directamente en el código para cada caso a calcular, o que el usuario no conozca que dato introducir. Por eso, mostramos con un mensaje el dato a ingresar y su tipo antes de leerlo el teclado.

La función para leer desde el teclado es *cin* >>, veamos el siguiente programa.

```
#include <iostream.h>
int main(){
  int numero;
  char car;
  float otroNum;
  cout<<"escribe un numero:"<<endl;
  cin>>numero;
  cout<<"\nel numero que tecleaste es: "<<numero<<endl;
  cout<<"dame una letra"<<endl;
  cin>>car;
  cout<<"\ntecleaste: "<<car<<endl;
  cout<<"escribe un numero flotante"<<endl;
  cin>>otroNum;
  cout<<"\neste es: "<<otroNum;
  return 0;
}
```

En resumen, cin es el flujo de entrada asociado al teclado, cout es el flujo de salida estándar asociado a la pantalla, y existe otro, que aunque a lo largo de este trabajo casi no lo utilicemos, es necesario nombrarlo, cerr, que es el flujo de error estándar asociado a la pantalla.

Los operadores « y » de inserción y extracción de flujo respectivamente, y no deben confundirse con los de desplazamiento de bits. Estos operadores son muy eficaces porque no es necesario especificar formatos para presentación o lectura, ellos los presentan en función al tipo de datos de la variable.

Para obtener resultados específicos están los modificadores de formato.

3.1. Modificadores de formato

En el primer caso del presente capítulo quizá no entendió del todo lo qué hacía la instrucción: $cout << (int)(3{,}141592+2) << endl$;

Si probó el código y lo ejecutó seguramente esperaba que apareciese en la pantalla 5.141592, esto habría pasado si no hubiésemos hecho una conversión de tipo. La inclusión de ”(int)” dentro de la instrucción provocó que el resultado de la operación, que debía ser 5.141592, se transforma a un entero y por resultado pierde sus valores decimales.

A esto se le llama hacer un cast, y es muy común hacerlo cuando no queremos que se pierdan determinadas propiedades en los resultados de las operaciones.

Las siguientes líneas de código, que son equivalentes tienen por efecto convertir el tipo de la variable de origen al de la destino:

```
int B;
double A= (double) B;
int B;
double A = double(B);
```

Otra de las herramientas para la especificación de formatos son los manipuladores de flujos, así como el manipulador de flujo ”endl” da una secuencia de escape, los manipuladores dec,

oct y hex, hacen que la variable a la que le anteceden sea presentada en formato decimal, octal o hexadecimal respectivamente.

```
#include <iostream.h>
int main(){
  int numero=25;
  int leido;
  cout<<"numero es en octal: "<<oct<<numero<<endl;
  cout<<"en hexadecimal: "<<hex<<numero<<endl;
  cout<<"en decimal: "<<dec<<numero<<endl;
  cout<<"ahora teclea un numero"<<endl;
  cin>>hex>>leido;
  cout<<"el leido vale: "<<hex<<leido<<endl;
  cout<<"y en decimal: "<<dec<<leido<<endl;
  cout<<"y en octal: "<<oct<<leido<<endl;
  return 0;
}
```

Estos manipuladores de flujo no son exclusivos de la salida, también funcionan con la entrada de datos, por defecto, la entrada desde el teclado se lee en decimal y también la presentación de datos se hace en decimal, así que si se sabe que el usuario dará a la máquina un dato en hexadecimal, sería buena idea anticiparlo y poner el formato en que será leído.

En cuanto a la precisión, al imprimir en pantalla un número de tipo flotante, no importa cuantas cifras significativas tenga (mientras esté dentro de los límites de almacenamiento de flotantes), sólo aparecerán 6 números después del punto, y la ultima cifra será redondeada en caso de pasar los 6 dígitos. Es decir, que si queremos mandar a la pantalla el número 1.23456789, en su lugar aparecerá 1.234568.

Cuando se quieren resultados más exactos, nos es muy útil la librería *iomanip.h* que contiene el manipulador setprecision() con la que determinamos la precisión con la que queremos que aparezcan esos datos en la pantalla.

Existe la función miembro precision(), que puede resultar un poco más cómoda en algunos casos porque el efecto de precisión funciona para todas las instrucciones cout que le sigan.

A ambas se le pasa como parámetro el número de cifras significativas deseadas.

Veamos un caso.

```
#include <iostream.h>
#include <iomanip.h>
int main(){
  cout<<1.23456789<<endl;
  cout.precision(4);
  cout<<1.23456789<<endl;
  cout<<setprecision(5)<<1.23456789<<endl;
  cout.precision(7);
  cout<<1.23456789<<endl;
  cout<<setprecision(9)<<1.23456789<<endl;
  return 0;
}
```

En ocasiones es útil dar una presentación a nuestros datos en la pantalla, tan sólo el dejar unos

cuantos espacios de margen en la pantalla puede ser un poco tedioso para quienes no están enterados de las siguientes funciones: setw que se encarga de hacer aparecer el texto alineado determinados espacios a la derecha, si el numero de espacios solicitados para su alineación es menor que el numero de caracteres que se imprimirán, no tendrá un efecto.

setfill se ocupa del carácter de relleno, el carácter por defecto es el espacio en blanco, pero nosotros podemos especificar el que queramos.

Vea el siguiente ejemplo.

```
#include <iostream.h>
#include <iomanip.h>
int main(){
  cout<<setw(5)<<setfill('%')<<"hola"<<endl;
  cout<<setw(5)<<89;
  cout<<setw(8)<<"centavo"<<endl;
  return 0;
}
```

En la primera instrucción cout, hacemos aparecer el texto alineado 5 espacios y en ellos aparece el carácter '%'. Es necesario el manipulador setw para cada operación de salida en pantalla, mientras que setfill funciona para todas las que se incluyan. Estas funciones, junto con las de alineación son una gran herramienta para la presentación.

Las funciones de alineación se incluyen como sigue, siendo la predeterminada la alineación a la derecha.

```
cout.setf(ios::right, ios::adjustfield);
cout.setf(ios::right, ios::adjustield);
```

3.2. Otras funciones muy útiles

La función de lectura desde el teclado cin no es la única que existe. Hay otros métodos de lectura desde el teclado para la lectura de caracteres, la tabla 2.6 resume principales características.

Cuadro 3.1: Funciones de lectura sin formato

Función	Sin argumentos	Un argumento	2 argumentos	3 argumentos
get	Lee un carácter y devuelve su valor númerico	Lee un carácter y lo guarda en la variable de argumento	Lee una serie de caracteres y los guarda en el primer argumento	Lee una serie de caracteres y los almacena en el primer argumento hasta llegar a la cantidad máxima de almacenamiento o igual a la de dos argumentos, pero se detiene si encuentra el caracter especificado en el tercer argumento
getline	* * * * * * * *	* * * * * * * *	Funciona igual al de dos argumentos, pero no almacena el delimitador de fuljo	* * * * * * * *
read	* * * * * * * *	* * * * * * * *	Almacena en el primer argumento un número determinado de caracteres	* * * * * * * *

Continúa en la siguiente página

Continuación de la tabla

Función	Sin argumentos	Un argumento	2 argumentos	3 argumentos
write	* * * * * * * *	* * * * * * * *	Muestra, de la cadena enviada como primer argumento, determinado numero de caracteres (segundo argumento)	* * * * * * * *
ignore	Ignora un carácter del flujo (opción predeterminada)	Ignora el número de caracteres enviados como argumento	* * * * * * * *	* * * * * * * *
gcount	Devuelve el número de caracteres leídos en la últimoa lectura sin formato	* * * * * * * *	* * * * * * * *	* * * * * * * *

Ahora veamos un ejemplo.

```
#include <iostream.h>
int main(){
  char caracter,nombre(20), apellido(20);
  char seg_apellido(20);
  cout<<"Teclea tu nombre por favor"<<endl;
  cin.get(caracter);
  38cin.get(nombre,20,' ');
  cin.get();
  cin.get(apellido,20,' ');
  cin.get();
  cin.get(seg_apellido,20);
  cout<<caracter<<"Tu letra inicial es:"<<caracter<<endl;
  cout<<"tu nombre es: "<<caracter<<nombre<<endl;
  cout<<"tu apellido paterno es: "<<apellido<<endl;
  cout<<"tu apellido materno es: "<<seg_apellido<<endl;
  cout<<"Escribe el nombre de tu empresa: "<<endl;
  cin.ignore();
  cin.getline(nombre,20);
  cout<<"el nombre de tu empresa: "<<nombre<<" tiene "
  <<cin.gcount()<<" caracteres"<<endl;
  cin.get();
```

```
22   return 0;
23 }
```

Es una ”tira” o serie de caracteres (char) con 20 elementos.

Pide al usuario que teclee su nombre completo, para luego leer inmediatamente el primer carácter que teclea y lo almacena en la variable ”caracter”. Sigue leyendo y almacena lo que escribe en la variable ”nombre” hasta que encuentre un espacio o llegue a 20 caracteres. Luego, otra instrucción get se encarga de leer y desechar el carácter espacio, para nuevamente leer hasta encontrar otro espacio.

Deja de leer hasta que se presione enter, hayas escrito o no tus dos apellidos. Así qpera que oprimas enter para terminar.

Puede que esta no sea la mejor manera para hacer un programa de este tipo, pero para nuestro objetivo creo que esta bien. Analicemos.

Primero declaramos una variable char, y 3 variables de cadenas de caracteres, por el momento no profundizaremos en que es exactamente, pero podemos decir que es ue cuando oprimes enter, ”carecer” tiene tu letra inicial y la presenta en pantalla, para aparecer tu nombre imprime ”caracter” seguido de ”nombre”, porque a ésta última no almacenó el primer carácter. Las siguientes instrucciones son parecidas hasta que pide el nombre de la empresa.

La función ignore descarta del flujo el carácter de terminación, para luego permitir a getline() leer el nombre de tu empresa con un máximo de 20 caracteres o hasta que oprimas enter (pero no lo guarda). Se escribe el nombre y con gcount() presenta los caracteres leídos. Finalmente espera que oprimas enter para terminar el programa.

CAPÍTULO 4

ESTRUCTURAS DE CONTROL

En nuestra vida cotidiana, todos tenemos una lógica a seguir y continuamente tomamos decisiones, y éstas repercuten en nuestra acción siguiente. Supongamos el caso de un estudiante de nivel preparatoria que cursa el sexto semestre, él está pensando en presentar el examen para la universidad, sin embargo, sus calificaciones no han sido buenas, y está en riesgo de tener que repetir ese semestre, si ocurre eso, el resultado que tenga en el examen no importará. Lo que vaya a pasar marcará el camino a seguir en su vida.

Analicemos de una forma general este caso:

```
curso el sexto semestre
  presento el examen de admisión
  si apruebo el examen y apruebo el semestre
    estaré en la universidad
  si apruebo el semestre y no apruebo el examen
    estaré trabajando
  si no apruebo el semestre
    curso el sexto semestre
(es decir, regresa al principio)
```

Estas son las opciones posibles, aunque podrían otras más.

Puede estudiar, trabajar o repetir todos los pasos anteriores.

En la programación se toman decisiones que influyen en el comportamiento del programa, y se pueden repetir series de pasos hasta obtener un resultado. En este capítulo aprenderemos eso.

4.1. Bloques de sentencias

El concepto de bloque en C++ se utilizar para agrupar un conjunto de sentencias dentro de un ámbito concreto del programa. Un bloque de sentencias es un conjunto de instrucciones delimitado por llaves: { y }.

Hay diferentes lugares, en un programa escrito en C++, donde podemos usar bloques de código. Como en la sintaxis de la función main(), las instrucciones que pertenecen a esta función principal se escriben dentro de una llave de apertura y una llave de cierre. Lo que hay entre esas dos llaves es el código de la función principal.

Por otro lado, en C++ son posibles otros bloques de código que están asociados a estructuras de control y que engloban un conjunto de instrucciones que se ejecutan bajo las condiciones que controlan estas estructuras.

4.2. Operadores relacionales y lógicos

ANSI C++ define el tipo bool que tiene uno de dos literales, false y true. Una expresión booleana o lógica es una secuencia de operandos y operadores que se combinan para producir un false o true.

Operadores como == o <= que comprueban una relación entre dos operandos se llaman operadores relacionales y se utilizan en expresiones de la forma:

```
1 expresion1 operador_relacional expresion2
```

Los operadores relacionales se usan en sentencias de selección y de iteración para comprobar una condición, vea la tabla 4.1.

Cuadro 4.1: Operadores relacionales en C++

Operador	Significado	Ejemplo
==	Igual a	$a == b$
>	Mayor que	$a > b$
<	Menor que	$a < b$
>=	Mayor o igual	$a >= b$
<=	Menor o igual	$a <= b$
!=	No igual a	$a != b$

Ejemplos:

```
1 x < 5.75
2
3 b * b >= 5.0 * a * c
```

Los operadores lógicos se utilizan con expresiones para devolver un valor verdadero o falso (true o false).

Se denominan también operadores booleanos. Los operadores lógicos de C++ son: not (!), and (&&) y or (||). El operador not produce falso si su operando es verdadero, y viceversa.

El operador and produce verdadero sólo si ambos operandos son verdaderos; si cualquiera de los operando es falso, produce falso.

El operador or produce verdadero si cualquiera de los operandos es verdadero, y falso en caso de que los dos operandos sean falsos.

Ejemplos:

```
!(7 == 5)

(aNum > 5) && (letra == ''a'')

a >= 5 || b == 3 && c <= 8
```

4.3. Jerarquía de operadores

La precedencia de operadores determina el orden en que se evalúan los operadores en una expresión.
Seguiremos como referencia la siguiente lista, donde los operadores de cada grupo tiene prioridad sobre los del grupo siguiente:

1. !(*not*) - (menos unario) + (más unario)
2. operadores multiplicativos: * / %
3. operadores aditivos: + - (binarios)
4. operadores relacionales de diferencia: $<<=>>=$
5. operadores relacionales de igualdad y desigualdad: $==!=$
6. operador lógico de conjunción: &&
7. operador lógico de disyunción: ||

   Además de esta lista, tenemos que tener en cuenta los siguientes puntos:

   - Si dos operadores se aplican al mismo operando, el operador con más prioridad se aplica primero.
   - Todos los operadores del mismo grupo tienen igual prioridad y asociatividad (se expresan de izquierda a derecha).
   - Los paréntesis tienen la máxima prioridad.

4.3.1. ?

Este se trata de un operador del tipo condicional al estilo if/else. Se utiliza para condicionales cortos (de una sola línea).
Por ejemplo:

```
#include <iostream.h>
int main(){
  int calificacion;
  cout<<"Cual fue tu calificacion del semestre?"<<endl;
  cin>>calificacion;
  cout<<(calificacion>=6 ? "aprobaste" : "reprobaste");
  cin.ignore();
  cin.get();
  return 0;
}
```

La forma de este operador es la siguiente:

condición ? instrucción a ejecutar en caso verdadero : instrucción a ejecutar en caso falso ; La condición es: calificación>=6. En caso de ser verdadera se envia ”aprobaste” como argumento a la instrucción cout, en caso contrario se envia reprobaste.

4.4. Estructuras de selección

C++ tiene dos estructuras de control para la selección, if (selección simple y binaria) y switch (selección múltiple).

4.4.1. if

Se trata de una estructura de selección. Si el condicional se cumple, se ejecutan una o más instrucciones.

En el caso anterior, observamos que: si apruebo el examen y el semestre estaré en la universidad ”si” podemos interpretarlo como el if de código c++, ”apruebo el examen y apruebo el semestre” es la condición ”estaré en la universidad” es la acción a ejecutar.

Escribamos el código:

```
#include <iostream.h>
int main(){
  char examen(2),semestre(2);
  cout<<"Contesta si o no a las preguntas"<<endl;
  cout<<"Pasaste el examen?"<<endl;
  cin.get(examen,2);
  cin.ignore(20,'\n');
  cout<<"Pasaste el semestre?"<<endl;
  cin.get(semestre,2);
  cin.ignore(20,'\n');
  if((examen(0)=='s')&&(semestre(0)=='s')){
    cout<<"Bienvenido a la universidad"<<endl;
  }
  cout<<"fin del programa"<<endl;
  cin.get();
  return 0;
}
```

Analicemos la sentencia if.

La condición a cumplir es: $(examen[0] ==' s')\&\&(semestre[0] ==' s')$, en realidad se trata de dos condiciones a cumplir: que examen tenga el carácter ”s” y semestre tenga ”s”. Si la primera condición se cumple, entonces comprueba la segunda (conjunción lógica &&).

Un condicional regresa 1 en caso de ser verdadero y un 0 para falso, estos dos valores en c++ están definidos como tipos de datos booleanos, true y false respectivamente, así que si el condicional resulta true (verdadero), se evalúa la siguiente instrucción dentro del bloque if, en caso contrario no hace nada.

En el caso anterior, el bloque de instrucciones a ejecutar en caso de ser verdadera la condición se encuentra dentro de llaves, pero no es necesario cuando sólo se trata de una sola línea de código.

Cuando queremos controlar las opciones que la condición resulte falsa, incluimos las instrucciones necesarias en un bloque else.

En el código anterior, podemos añadir un else a la sentencia if, tal que quede de la siguiente forma:

```
if((examen[0]=='s')&&(semestre[0]=='s')){
  cout<<"estas en la universidad"<<endl;
} else {
  cout<<"no estas en la universidad"<<endl;
}
```

Igualmente el bloque de instrucciones a ejecutar se delimita por llaves, NO es necesario cuando se trata de una sola instrucción.

En ocasiones, necesitamos construir bloques de instrucciones if y else complejos, es decir anidamientos. El caso del estudiante también ayuda a visualizar este concepto.

si apruebo el examen y el semestre estaré en la universidad si apruebo el semestre pero no el examen estaré trabajando, si no apruebo el semestre curso el sexto semestre.

Construyamos esta serie de decisiones de la siguiente forma:

```
if apruebo el examen y el semestre
estaré en la universidad
  else if apruebo el semestre pero no el examen
    estaré trabajando
  else
    curso el sexto semestre
```

De esta manera controlamos las posibles respuestas. En caso de aprobar el examen y el semestre estará en la universidad, si aprueba el semestre pero no el examen estará trabajando, y en otro caso, cursará el semestre.

Nuestro código ya terminado queda de la siguiente manera:

```
#include <iostream.h>
int main(){
  char examen(2),semestre(2);
  cout<<"Contesta si o no a las preguntas"<<endl;
  cout<<"Pasaste el examen?"<<endl;
  cin.get(examen,2);
```

```
7    cin.ignore(20,'\n');
8    cout<<"Pasaste el semestre?"<<endl;
9    cin.get(semestre,2);
10   cin.ignore(20,'\n');
11   //ignora el enter
12   if((examen(0)=='s')&&(semestre(0)=='s'))
13   cout<<"estas en la universidad"<<endl;
14   else
15   if((examen(0)=='n')&&(semestre(0)=='s'))
16   cout<<"estaras trabajando"<<endl;
17   else
18   cout<<"cursa el sexto semestre"<<endl;
19   cout<<"fin del programa"<<endl;
20   cin.get();
21   //espera a que oprimas enter
22   return 0;
23 }
```

En esta ocasión se eliminaron las llaves que no eran necesarias.

4.4.2. switch

Muchas veces necesitamos tener el control sobre las opciones que pudiese tomar el usuario, porque es complicado pensar en varios if/else anidados, para esos casos tenemos otra herramienta la estructura de selección múltiple switch.

La forma general es:

```
1   switch (parámetro a evaluar o comparar){
2     case a :
3     //cuando el parámetro tiene un valor a
4     Acciones a ejecutar;
5     case b:
6     //cuando el parámetro tiene un valor b
7     Acciones a ejecutar
8     .
9     .
10    .
11    caso por default;
12  }
```

```
1   #include <iostream.h>
2   int main(){
3     int opcion;
4     cout<<"Menu de opciones"<<endl;
5
6     cout<<"1.- Opcion 1"<<endl;
7     cout<<"2.- Opcion 2"<<endl;
8     cout<<"3.- Opcion 3"<<endl;
9     cout<<"elige una opcion"<<endl;
10    cin>>opcion;
11    switch(opcion){
```

```
12        case 1:
13        cout<<"ejecucion 1"<<endl;
14        break;
15        case 2:
16        cout<<"ejecucion 2"<<endl;
17        break;
18        case 3:
19        cout<<"ejecucion 3"<<endl;
20        break;
21        default:
22        cout<<"es una opcion no valida"<<endl;
23        break;
24      }
25      cout<<"presiona enter para salir"<<endl;
26      cin.ignore();
27      cin.get();
28      return 0;
29    }
```

Notemos que ”opcion” es una variable de tipo entero, así que case 1 , se refiere a que la variable ”opcion” tiene un valor de 1. Si se tratase de una variable de tipo carácter, entonces debería de escribirse case ‘1’.

Encontramos una nueva instrucción, break. Esta se encarga de alterar el flujo de control. Cuando se encuentra un break, inmediatamente se salta el resto de la estructura. No es exclusiva de la estructura switch, también se puede colocar en bloques if, for, while o do while (que se verán a continuación). Sin embargo, es mejor no depender demasiado del break [1].

4.5. Estructuras de iteración

C++ nos ofrece 3 esquemas de iteración diferentes.

4.5.1. while

La sentencia de control while se encarga de repetir un bloque de código mientras se cumpla una condición. El bloque de código se debe de encontrar entre llaves, excepto si es una sola línea. La forma general de esta instrucción es la siguiente:

```
1  while (condición a cumplir) {
2    acciones a ejecutar;
3  }
```

Un ejemplo de éste lo podemos ver en el cálculo del promedio de 10 números, pensemos que son las calificaciones del semestre. Mirando rápidamente la forma de estructurar nuestro programa (algo que se debe de hacer siempre, y antes de sentarse frente a nuestro computador), podemos pensar en la siguiente solución:

[1] Romper

```
Declarar variables e inicializarlas
Mientras (no se hagan 10 iteraciones){
  Leer una calificación
  Sumarla al total
  Determinar que se ha hecho una iteración
}
```

calcular el promedio

mostrar el promedio

El programa queda como sigue:

```
#include <iostream.h>
int main(){
  int calificacion, suma=0, iteracion=1;
  float promedio;
  while(iteracion<=10){
    cout<<"teclea tu calificacion "<<iteracion<<endl;
    cin>>calificacion;
    suma+=calificacion;
    ++iteracion;
  }
  promedio=(float)suma/itercion1;
  cout<<"el promedio de calificaciones es: "<<promedio<<endl;
  cin.ignore();
  cin.get();
  return 0;
}
```

El programa es sencillo, quizás la parte que causa duda es el cálculo del promedio. Se realiza un cast para convertir temporalmente la variable suma de int a float, así se hace la división y se tiene un resultado decimal cuando sea necesario.

Hasta aquí nuestro programa va bien, pero podría mejorar, tener un mejor control del mismo y hacer que el usuario decida cuantas calificaciones va a introducir. Hacemos que el usuario teclee un ”-1” cuando acabe de escribir sus calificaciones. Así, en lugar de comparar el número de iteraciones, comprobará si la calificación es igual a ”-1” para salir.

Aquí el programa:

```
#include <iostream.h>
int main(){
  int calificacion=0, suma=0, iteracion=0;
  float promedio=0;
  while (calificacion<>1) {
    ++iteracion;
    suma+=calificacion;
    cout<<"teclea tu califiaccion "<<iteracion<<endl;
    cin>>calificacion;
  }
  promedio=(float)suma/(iteracion1);
  cout<<"el promedio de calificaciones es: "<<promedio<<endl;
```

```
13      cin.ignore();
14      cin.get();
15      return 0;
16  }
```

Vemos como cambiando unas instrucciones de lugar y un valor obtenemos un mejor resultado.

La variable ”iteracion” la inicializamos con un valor de 0, al inicio del bucle le aumentamos una unidad, luego a ”suma” le añadimos la calificación que en este caso aún es 0, luego pide y lee la calificación tecleada.

Al siguiente ciclo comprueba si se escribió ”-1”, si no entonces aumenta el contador ”iteracion”, suma la calificación introducida, por ejemplo un 8, y pide la siguiente, el usuario teclea, de nuevo, 8.

Hace de nuevo una comprobación y entra de nuevo al ciclo, aumenta el valor de ”iteracion” a hora tiene un 3, ”suma” ahora vale 8 que era lo que tenía antes más el 8 recientemente leído, es decir 16, pide la siguiente calificación, y teclea la persona ”-1”.

Otra comprobación da como resultado un falso (escribió -1) y por lo tanto no entra al ciclo, entonces calcula el promedio, donde notemos que a ”iteracion” se le resta uno para tener el verdadero número de calificaciones introducidas. Por último presenta el resultado.

Hay otras formas de hacer este programa, por ejemplo cuando no esta inicializada la variable de calificaciones o iteraciones, y necesitamos que ese ciclo se ejecute por lo menos una vez. En este caso podemos usar algo muy parecido, se trata de la sentencia do while.

4.5.2. do while

Cuando necesitamos que un ciclo se ejecute por lo menos una vez, es necesaria esta sentencia.

La forma general es:

```
1  do{
2    Acciones a ejecutar;
3  }
4  while(condicion a cumplir);
```

Pongamos como ejemplo el mismo de la estructura switch, el menú de opciones.

Necesitamos que por lo menos una vez lea la opción que elige, y en caso de que elija una no disponible, en lugar de aparecer el mensaje ”es una opción no válida” y termine el programa, espere a que el usuario escoja una de la lista.

El programa de caso:

```
1  #include <iostream.h>
2  int main(){
3    int opcion;
4    do {
5      cout<<"elije una opcion de la lista"<<endl;
6      cout<<"1.- opcion 1"<<endl;
7      cout<<"2.- opcion 2"<<endl;
8      cout<<"3.- opcion 3"<<endl;
```

```
9         cin>>opcion;
10      } while ((opcion<1) || (opcion>3));
11      cout<<"esta opcion si fue valida"<<endl;
12      cin.ignore();
13      cin.get();
14      return 0;
15    }
```

Nunca debemos olvidar el punto y coma después del while. Usada en combinación con el bloque switch, tenemos buenos resultados y encontramos varias aplicaciones de ellos.

4.5.3. for

El ciclo *for* es al que se recurre para realizar operaciones secuenciales, en donde se conoce el número de iteraciones o la condición a comprobar.

La forma general de esta sentencia es:

```
1  for(asignación inicial ; condición ; instrucción ) {
2    bloque de instrucciones;
3  }
```

Si se trata de una sola instrucción a ejecutar no es necesario ponerla entre llaves, incluso podría ponerse dentro del paréntesis el *for*, separado por una coma.

```
1  for(asignación inicial ; condicion ; instrucción1)
2  unica instrucción;
```

o, si se trata de una instrucción muy corta podría ponerse:

$for(asignacininicial; condicion; instruccin1, instruccin2)$

Pensemos en una aplicación. Hagamos un programa que calcule las coordenadas de un proyectil que se mueve en movimiento parabólico. Al menos determinaremos 10 puntos por los que pasa. Si se tienen dudas acerca de las fórmulas recomiendo que consulten un libro sobre física.

Las formulas son las siguientes:

```
1  x = V_{i} * sin(0) * t;
2  y = V_{i} * sin(0) * t \frac{1}{2} g * t
```

Donde V_i es la velocidad inicial.

θ es el ángulo c de lanzamiento.

t es el tiempo.

g es la gravedad.

La forma en que pensaría hacer este programa es la siguiente:

```
1  Declarar e inicializar variables
2  Pedir datos
3  for (contador=1; hasta que el contador llegue a 10; aumentar contador){
4    Calcular coordenada x
```

```
    Calcular coordenada y
    Imprimir las coordenadas
  }
  Termina el programa.
```

No parece tan complicado ¿verdad? Pues en realidad no lo es tanto. El programa terminado aquí está.

```
#include <iostream.h>
#include <math.h>
int main (){
  int t;
  float x=0,y=0,ang,vi, grav=9.81;
  cout<<"cual es el angulo de lanzamiento?"<<endl;
  cin>>ang;
  cout<<"cual es la velocidad inicial?"<<endl;
  cin>>vi;
  ang = ang * 3.141592 / 180;
  for(t=0;t<=10;t++){
    x = vi * t * cos(ang);
    y = (vi*t*sin(ang)) - (0.5*grav*t*t);
    cout<<"t= "<<t<<" x= "<<x<<" y= "<<y<<endl;
  }
  cout<<"fin del programa"<<endl;
  cin.ignore();
  cin.get();
  return (0);
}
```

El ciclo for comienza inicializando la variable tiempo en 0. La condición a cumplir para entrar en el ciclo es que t sea menor o igual a 10.

Cuando entra en el ciclo calcula las coordenadas (x, y). Hacemos uso de las funciones *cos* y *sin* de la librería *math.h*. Las que calculan el coseno y el seno de un ángulo determinado en radianes, es por eso que la variable ”ang” es transformada a radianes. Luego de haber calculado las coordenadas, las muestra en pantalla. Y finalmente ejecuta la última instrucción del *for* aumenta en una unidad a $t(t++)$.

En el momento en que la variable t sea mayor que 10, sale del ciclo y llega el fin del programa.

CAPÍTULO 5

FUNCIONES

Cuando tratamos de resolver un problema, resulta útil utilizar la filosofía de ”divide y vencerás”. Esta estrategia consiste en dividir nuestro problema en otros más sencillos.

Cuando realizamos un programa en el que se repetirán varias instrucciones pero con distintos valores que definan los resultados, podemos construir el programa a base de funciones. Una función es un bloque de instrucciones a las que se les asigna un nombre. Entonces, cada que necesitemos que se ejecuten esa serie de instrucciones, haremos una invocación a la función.

Ya hemos hecho uso de algunas funciones, en el capítulo anterior usamos cos y sin, que están presentes en la librería math.h. Así no nos preocupamos por todo lo que haga esa función para devolver un resultado, lo único que nos preocupó fue que datos teníamos que mandar y los que íbamos a recibir.

5.1. Proptotipos

a ejecutar.

Escribamos el código: El prototipo de una función se refiere a la información contenida en la declaración de una función. Una función debe de estar definida o al menos declarada antes de hacer uso de ella.

Cuando se declara una función debe de especificarse el tipo de dato que va a devolver, el nombre de la función, y los parámetros. La siguiente declaración:

$intsuma(inta, intb)$;

Especifica una función que devuelve un tipo de dato entero, tiene por nombre suma, y al invocarla, recibe 2 parámetros de tipo entero.

Esta declaración debe de escribirse antes de la función main, y su definición puede escribirse después de ésta.

Por ejemplo:

```
1 #include <iostream.h>
```

```
int suma(int x, int y);
int main(){
  int dato1,dato2;
  cout<<"calcular la suma de 2 numeros"<<endl;
  cout<<"escribe el dato 1 y luego el dos"<<endl;
  cin>>dato1;
  cin>>dato2;
  cout<<"la suma es: "<<suma(dato1,dato2)<<endl;
  cin.ignore();
  cin.get();
  return 0;
}
int suma(int a, int b){
  return a+b;
}
```

Observamos que primeramente declaramos la función suma para usarla en el bloque main, y al final del código la definimos. Podíamos haber escrito la función completa antes del main.

La función main también tiene un prototipo, y por lo tanto puede recibir datos.

$intmain(intargc, char*argv[], char*envp[])$

El parámetro argc es un entero que contiene el número de argumentos pasados a la función desde la línea de comandos. Es decir, almacena el número de argumentos en la tabla argv.

El parámetro argv es una tabla de cadenas de caracteres (se verá más adelante), cada cadena de caracteres contiene un argumento pasado en la linea de comandos, la primera cadena contiene el nombre con el que fue invocado y las siguientes son los demás argumentos.

El parámetro envp es una tabla de cadenas, que pasa información sobre una variable de entorno del sistema operativo.

5.2. Paso de argumentos

Cuando hablamos de argumentos, nos referimos a los valores que aparecen en la llamada a la función. En el ejemplo anterior los argumentos fueron ”dato1” y ”dato2”.

Y los parámetros son ”a” y ”b”, los que reciben los datos.

Notemos que el prototipo de la función tiene los nombres de los parámetros distintos a los de la definición, esto no afectará el comportamiento del programa, pero se vería mejor si fueran iguales. Lo que no puede cambiar es el tipo de datos que va a recibir.

En el caso del main, el paso de argumentos se realiza desde la línea de comandos, como ejemplo ponemos el comando dir. Si lo invocamos, escribimos dir /w /p; el argumento argc tiene el valor 3, y argv contiene 3 cadenas de caracteres: dir, /w, /p.

5.3. Valores de retorno

Una función puede regresar cualquier tipo de valor excepto tablas u otras funciones.

Esta limitación podría resolverse utilizando estructuras o punteros, pero se verá más adelante.

5.4. Macros

Una macro es una parte del código que puede parecer y actuar como una función, se define después de las librerías mediante un #define.

Se denominan instrucciones de preproceso, porque se ejecutan al comienzo de la compilación.

Sin embargo, no hay que abusar de ellas, porque podrían entorpecer el código y hacer lento el programa.

```
#include <iostream.h>
#define mayor(a,b)
(a>b)? a: b
int main(){
  int a,b;
  cout<<"teclea 2 numeros distintos"<<endl;
  cin>>a;
  cin>>b;
  cout<<"el mayor de esos numeros es: "
  <<(mayor(a,b))<<endl;
  cin.ignore();
  cin.get();
  return 0;
}
```

Debemos tener cuidado si vamos a usar macros, las macros, al compilar el programa, se sustituyen directamente en donde se invocan, es decir, que en este ejemplo, es como si se introdujera directamente los condicionales dentro del cout.

Así, debemos tener cuidado en introducir bloques grandes de código en una macro (algo nada recomendable), sobre todo, cuidado con los puntos y comas.

Podemos ver que las macros al igual que las funciones pueden tener parámetros, sin embargo, tienen que escribirse con cuidado.

Al hacer una macro de la siguiente manera: $\#definecurva(a)1+2*a+a*a$

Si en el código colocamos una instrucción $curva(1+3)$, se sustituye por $1+2*1+3+1+3*1+3$ contrario a lo esperado si invocamos curva(4), 13 en lugar de 25. Habrá que definir la macro como $1+2*(a)+(a)*(a)$.

El uso de macros hace más difícil la depuración de un programa, porque en caso de haber errores en la macro, el compilador avisa que hay errores, pero en la implementación de la macro. Además de que no realizará comprobaciones de tipo de datos ni conversiones.

Ejemplo del libro Programación en C/C++, Alejandro Sierra Urrecho

En el lenguaje C, se acostumbra usar macros para definir constantes (porque a diferencia de C++, ahí no existe el tipo const. Sea: $\#definePI3{,}141592$ que cambia "PI" por el valor 3.141592.

5.5. Recursividad

La recursividad se presenta cuando una función se invoca a si misma. Distintamente a las iteraciones (bucles), las funciones recursivas consumen muchos recursos de memoria y tiempo.

Una función recursiva se programa simplemente para resolver los casos más sencillos, cuando se llama a una función con un caso más complicado, se divide el problema en dos partes, la parte que se resuelve inmediatamente y la que necesita de más pasos, ésta última se manda de nuevo a la función, que a su ves la divide de nuevo, y así sucesivamente hasta que se llegue al caso base. Cuando se llega al final de la serie de llamadas, va recorriendo el camino de regreso, hasta que por fin, presenta el resultado.

El problema clásico es calcular el factorial de un número, $(n!)$.

Conocemos los casos base, $0! = 1, 1! = 1$. La función factorial se define como $n! = n * (n-1!)$.

Entonces, $5! = 5*4! = 5*4*3! = 5*4*3*2! = 5*4*3*2*1! = 120$.

Nuestro programa queda:

```
#include <iostream.h>
long factorial(long numero);
int main(){
  long numero;
  cout<<"número para calcular el factorial:"<<endl;
  cin>>numero;
  cout<<"el factorial es: "<<factorial(numero)<<endl;
  cin.ignore();
  cin.get();
  return 0;
}
long factorial(long numero){
  if(numero<=1)
    return 1;
  else
    return numero*factorial(numero1);
}
```

Nuestro programa llama a la función factorial con un número inicial, luego la función se llama a sí misma cada vez con un número menor en uno hasta que llega al caso base ($numero <= 1$).

La recursividad es un tema importante en la programación, la utilidad de este tipo de funciones se aprovecha en el diseño de algoritmos de criptografía, de búsqueda, entre otros. La implementación de la recursividad en nuestros programas debe de evaluarse con mucho cuidado, debemos de tratar de evitar que se usen funciones de complejidad exponencial, que se llamen a sí mismas una y otra vez sin que se tenga un control claro.

CAPÍTULO 6

ARREGLOS

Cuando declaramos una variable estamos apartando en memoria espacio para guardar sus posibles valores dependiendo del tipo de dato que se trata. Un array o arreglo son una serie de localidades en memoria consecutivas que están asignadas a un sólo nombre y un mismo tipo de datos.

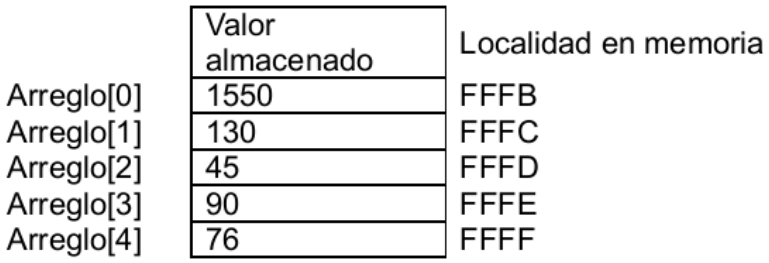


Figura 6.1: Mapa de memoria para arreglos

Ya anteriormente hemos presentado ejemplos en los que se hace uso de los arreglos, y aunque no comprendimos exactamente que significado tenía la aplicación de los arreglos nos dio una idea de sus utilidades.

6.1. Declaración

La forma de declarar un arreglo de cualquier tipo de datos es la siguiente: tipo nombre [tamaño];

Podemos declarar un arreglo de enteros con 12 elementos: int MiArreglo [12];

El compilador se encarga de asignar la memoria requerida para almacenar determinados

valores.

Cuando se declara un arreglo de caracteres se trata entonces de una cadena: char nombre[20];

6.2. Asignación de valores

Al momento de declarar un arreglo de cualquier tipo, podemos inicializarlo con los valores que queramos.

Para inicializar un arreglo de enteros:

$intMiArreglo[5] = 2,34,78,1,9;$

Así, estos valores estarán almacenados en cada elemento del array. Es muy importante hacer notar que el primer elemento de un arreglo es el elemento 0, entonces, MiArreglo[0] contendrá el número 2, el segundo (MiArreglo[1]) contendrá el número 34 y así sucesivamente hasta MiArreglo[4] que es el último elemento del array. Si un arreglo cuenta con menos inicalizadores que elementos entonces el resto se inicializará a 0.

Y en caso de que se trate de una cadena de caracteres podemos hacerlo de 2 formas: $charMiCadena[13] = "holaatodos"$;

o bien, declarar cada elemento $charMiArray[5] = 'h','o','l','a',''$;

Cuando se inicializa una cadena por el primer método, automáticamente se coloca un carácter de terminación de cadena (el carácter \0), en cambio, de la segunda forma debemos de ponerlo nosotros. También se puede excluir el tamaño del arreglo, el compilador lo determinará en la compilación.

Para acceder a cada elemento del arreglo debemos especificar su posición (la primera es la posición 0). En tiempo de ejecución podemos asignar valores a cada elemento cuidando siempre de no sobrepasar el tamaño del arreglo, si sobrepasamos ese tamaño se escribirán datos en un área de la memoria que no está asignada para ese array, puede escribir datos en un área en donde se almacenaba otra variable del programa o un componente del sistema, esto ocasiona resultados no deseados.

```
#include <iostream.h>
#include <iomanip.h>
#include <stdlib.h>
#include <time.h>
int main(){
  const int tam_max=20;
  int aleatorios(tam_max);
  int i,suma=0;
  float promedio;
  srand((unsigned)time(NULL));
  for(i=0;i<tam_max;i++){
    aleatorios(i)=rand()%128;
    cout<<"aleatorio generado: "<<setw(5)<<aleatorios(i)<<endl;
  }
  for(i=0;i<tam_max;)
  suma+=aleatorios(i++);
  promedio=(float)suma/tam_max;
```

```
18    cout<<endl<<"el promedio es: "<<promedio<<endl;
19    cin.get();
20    return 0;
21 }
```

El programa anterior muestra el uso de los arreglos. Ya anteriormente habíamos hecho un ejercicio que calculaba el promedio de una serie de números introducidos por el usuario, en este caso el programa guarda los números generados por la máquina en un arreglo para luego calcular el promedio.

Para asignar cada valor aleatorio generado, se utiliza un ciclo for con un contador ”i” que va de 0 hasta el tamaño máximo que pudiera tener el array. El tamaño del array fue declarado como una constante al principio de la función principal, esto ayudará en caso de que se quiera aumentar el tamaño del arreglo, así no tendremos que modificar cada parte de nuestro código en la que se refiera al tamaño, bastará con cambiar el valor de la constante.

Dentro del ciclo se genera un número aleatorio con la función rand(), usamos el operador para obtener el resto de la división entera, así nos dará un número menor de 128. Previamente se utilizó la función srand() para establecer una semilla para generar los números aleatorios, esa semilla la tomamos llamando al reloj del sistema con la función time(). Al mismo tiempo imprimimos el número que fue generado.

Posteriormente, se hace de nuevo un ciclo para acceder a los valores almacenados y sumarlos en una variable, aunque esto se pudo haber hecho dentro del ciclo anterior sirve para mostrar las formas de asignación y consulta de valores. Además del uso del incremento después de la asignación.

Finalmente se calcula el promedio.

En el caso de las cadenas de caracteres también podemos acceder sus elementos de la misma manera, para la asignar valores es mejor usar las funciones para la lectura de caracteres, tales como getline(), porque nos permite tener un control de los caracteres que se introducirán y también introduce el carácter de terminación de cadena, cosa que no es posible con *cin* >>directamente y además de que puede escribir en sectores de memoria no adecuados.

El siguiente programa es una muestra del uso de arreglos de caracteres.

```
1  #include <iostream.h>
2  int main(){
3    const int tam_cad=20;
4    char cadena(tam_cad);
5    cin.getline(cadena,tam_cad,'\n');
6    for(int i=0;(i<tam_cad)&&(cadena(i)!='\0');i++)
7    cadena(i)+=5;
8    cout<<cadena<<endl;
9    cin.get();
10   return 0;
11 }
```

¿Qué hace este programa?, sencillamente, almacena una cadena introducida por el usuario, para luego modificar cada carácter por medio de un ciclo hasta que encuentre el carácter de terminación (\0) o llegue al limite de caracteres.

Quizá no parezca muy interesante el ejemplo, pero con cosas pequeñas se construyen grandes cosas, adornemos un poco nuestro programa, supongamos que la cadena del usuario es un mensaje que debe ser secreto, en lugar de sumar 5 a cada carácter le sumamos el número que quiera el usuario, ¿por qué no una clave?.

El mismo programa ”adornado” o ”presuntuoso” queda de la siguiente forma.

```
#include <iostream.h>
int main(){
  const int tam_cad=20;
  char cadena(tam_cad);
  int clave;
  67cout<<"introduce el texto a cifrar"<<endl;
  cin.getline(cadena,tam_cad,'\n');
  cout<<"ahora dame tu clave (no mas de 3 digitos)"<<endl;
  cin>>clave;
  for(int i=0;(i<tam_cad)&&(cadena(i)!='\0');i++)
  cadena(i)+=clave;
  cout<<"el texto cifrado es: "<<endl;
  cout<<cadena<<endl;
  cin.ignore();
  cin.get();
  return 0;
}
```

Como ejercicio sugiero que haga un programa que realice la acción contraria, de un texto cifrado saque el mensaje oculto, y ¿por qué no? intente descifrar un mensaje sin la clave.

La manipulación de cadenas es uno de los temas más extensos en cuestiones de programación y de los más importantes. Desde las acciones más básicas como la concatenación de cadenas, la ordenación y búsqueda en arreglos, hasta los complicados algoritmos de criptografía o la búsqueda a velocidades increíbles.

6.3. Paso de argumentos

Así como en ejemplos anteriores pasamos una variable a una función, también se puede pasar como argumento un arreglo. Para pasarlo es necesario hacer una llamada como la siguiente: $MiFuncion(arreglo)$;

Y el prototipo de la función debe ser de la siguiente forma:

$tipoRetornoMiFuncion(tipoArreglonombre[])$;

No se requiere incluir el tamaño del arreglo dentro de los corchetes, si se escribiera el compilador lo ignora.

Cuando pasamos como argumento a una función una variable de tipo int, float, char, etc., estamos pasando el valor de esta variable a otra variable (podría decirse una copia) que se define en la función, esta forma de pasar argumentos se denomina ”por valor”. Cualquier modificación de valor a esta variable no tendrá efecto en la variable ”original” que pasó su valor a la función. Esta última variable, al ser local, será destruida cuando termine de ejecutarse la función que la contiene.

En el caso del paso de arreglos, este no se hace por valor, sino que se hace un paso por referencia simulado. Cuando se declara un arreglo, el nombre de éste es una variable que apunta al primer elemento del arreglo, es decir que contiene su dirección en memoria.



Figura 6.2: Desplazamiento de punteros

Cuando pasamos esta dirección a una función, cualquier modificación que se haga dentro de esa función de nuestro arreglo tendrá efectos en la variable original.

El siguiente programa da un ejemplo:

```
#include <iostream.h>
void modifica(int arreglo(), int tamano);
int main(){
  const int tam_arr=5;
  int MiArreglo(tam_arr)={5,10,15,20,25};
  int i;
  modifica(MiArreglo, tam_arr);
  for(i=0;i<tam_arr;++i){
    cout<<"elemento "<<i<<"es "
    <<MiArreglo(i)<<endl;
  }
  cout<<"presiona enter para terminar"<<endl;
  cin.get();
  return 0;
}
void modifica(int arreglo(), int tam_arr){
  for(int i=0;i<tam_arr;++i)
  arreglo(i)*=2;
}
```

Se ha declarado e inicializado un arreglo de enteros de 5 elementos. Posteriormente se llama a la función modifica, la cual multiplica cada elemento del arreglo por dos, estos cambios de valor en los elementos del arreglo tienen efecto en ”MiArreglo” aunque el nombre del parámetro de la función modifica sea diferente. También se puede pasar a una función el valor de un elemento del array, sólo se pasará el valor del elemento, no se puede modificar el elemento original.

FuncionInvocada (array [elemento]); En ocasiones puede resultar poco conveniente que alguna de las funciones que reciben un arreglo tenga derecho a modificarlo, a veces puede que cambie sus valores sin que nosotros nos percatemos inmediatamente. Para eso podemos incluir el calificador const dentro del prototipo y definición de la función, de esta manera no

podrá modificarse nada de este arreglo.

tipoRetorno nombreFuncion (const tipoArreglo nombre [ ]);

6.4. Arreglos multidimensionales

Como sabemos, se pueden crear arreglos de cualquier tipo de datos, ahora veremos que también se pueden crear arreglos de arreglos. La forma de declararlos es con múltiples subíndices, cuantos se quieran tener, nosotros trabajaremos a lo más con dobles subíndices.
tipoArreglo nombre [subíndice1] [subíndice2] ;

La forma en que queda estructurado un arreglo de dos dimensiones es más o menos como se muestra en la figura 6.3.

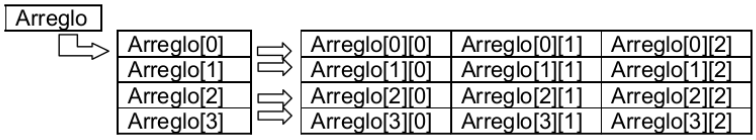


Figura 6.3: Arreglo multidimensional

El nombre del arreglo tiene la dirección del primer elemento (Arreglo[0]), y éste a su vez tiene la dirección del otro arreglo (Arreglo[0][0]), así para cada elemento de arreglo(0, 1, 2, 3).

Cuando se inicializa un arreglo multidimensional se inicializa entre llaves cada arreglo o bien, por medio de elemento seguidos:

```
{Array1 (2) (3) = {1, 2, 3, 4, 5, 6};
Array2 (2) (3) = {{2, 4, 6}, {8, 10, 12}};
}
```

Al mandar un arreglo de dos dimensiones, debemos de especificar en el prototipo y definición de la función el subíndice2 de nuestro array. El siguiente ejemplo muestra varios aspectos de ellos.

```
#include <iostream.h>
#include <iomanip.h>
#include <stdlib.h>
#include <time.h>
const int vendedores=5, meses=4;
void inicializa(int ventas()(meses));
void calculaTotal(const int ventas()(meses));
int main(){
  int ventas(vendedores)(meses);
  int i;
  cout<<"ventas:"<<setw(23);
  for(i=1;i<=meses;++i)
  cout<<"mes"<<i;
```

```
14    cout<<endl;
15    inicializa(ventas);
16    calculaTotal(ventas);
17    cout<<endl<<endl<<"presiona enter para terminar"<<endl;
18    cin.get();
19    return 0;
20  }
21  void inicializa(int ventas[][meses]){
22    int i,j;
23    srand((unsigned)time(NULL));
24    for(i=0;i<vendedores;++i){
25      cout<<"ventas del vendedor "<<i+1<<": ";
26      for(j=0;j<meses;++j){
27        ventas[i][j]=rand()%50;
28        cout<<setw(7)<<ventas[i][j];
29      }
30      cout<<endl;
31    }
32  }
33  void calculaTotal(const int ventas[][meses]){
34    int suma=0,total=0,i,j;
35    cout<<endl<<endl;
36    for(i=0;i<vendedores;++i){
37      suma=0;
38      for(j=0;j<meses;++j){
39        suma+=ventas[i][j];
40      }
41      cout<<"el vendedor "<<i+1<<" vendio "
42      <<suma<<" productos"<<endl;
43      total+=suma;
44    }
45    cout<<endl<<endl<<"en total se vendieron: "
46    <<total<<" productos en "<<meses<<" meses"<<endl;
47  }
```

Notemos primero cómo el prototipo de las funciones ”inicializa” y ”calculaTotal” debe incluir el segundo subíndice del arreglo. La función ”calculaTotal” muestra un ejemplo del uso del calificador const para evitar que en esta función se modifique algún valor del arreglo.

La función ”inicializa” llena el arreglo con valores aleatorios menores a 50, se utilizan dos contadores para dos ciclos for, ”i” para el primer subíndice, ”j” para el segundo. El ciclo recorre la primera ”fila” (viendo el arreglo como una tabla de valores) usando el contador i y a cada celda, con ayuda de ”j”, asigna valores, así cuando ”j” llega al límite de celdas (meses) se aumenta una unidad el contador ”i” para continuar con la otra fila y recorrerla de nuevo.

La función ”calculaTotal” tiene una forma muy similar, pero en ésta no se modificarán valores, sino que se hará la suma de los elementos de cada fila para presentarlos en pantalla. Una variable ”total” se encarga de ir sumando los productos vendidos cada que se cambia de fila en la tabla, para después, al término de los ciclos presentar el total de artículos vendidos en esos meses.

Puede aumentarse el número de filas y columnas de la tabla modificando los valores de las

constantes ”vendedores” y ”meses” respectivamente, definidas al comienzo del programa.

En cuanto al uso de arreglos cadenas de caracteres se dejará para el siguiente capítulo, porque es necesario considerar conceptos de direcciones de memoria, el uso de los apuntadores.

CAPÍTULO 7

ESTRUCTURAS

En muchos lenguajes, se le permite al programador definir sus propios tipos de datos a partir de los tipos atómicos. En C y C++ pasa lo mismo, y además existen de antemano los tipos union, enumeración y estructura. Estos tipos definidos forman parte del estandar de C, y por consiguiente C++, en éste último existe también el tipo class.

Para los objetivos de este trabajo excluiremos el tipo class (clase), pero a lo largo del capítulo mencionaremos en varias ocasiones este concepto para tener una idea de lo que se trata.

7.1. Enumeraciones

En ocasiones, declaramos constantes enteras una tras otra con valores consecutivos:

$constinta = 0, b = 1, c = 2, d = 3;$

para evitar este tipo de declaraciones, existe el tipo enumerativo. Tla que la declaración anterior se hace: $enuma, b, c, d;$

Una enumeración puede ser global o local. Además, debe tener un nombre que los identifique, podemos indicar desde donde empieza la numeración.

$enummesesenero = 1, febrero, marzo, abril, mayo, junio, julio, agosto, septiembre, octubre, noviembre, diciem$

Los valores que se asignan empiezan por 1 a enero, 2 a febrero y así sucesivamente. Si asignamos un 5 a marzo, entonces abril tendría un 6 y así continuaría la numeración.

Después de hacer esta enumeración, se puede hacer una nueva a partir del nombre.

Un programa sencillo lo muestra.

```
1 #include<iostream.h>
2 enum colores{rojo,amarillo,verde};
3 int main(){
4   int i=0,contador=0;
5   //unos contadores
6   colores semaforo;
7   cout<<"este programa mostrara el tipo enumerativo,"
```

```
8     <<" por medio de un semaforo de crucero"
9     <<" simulado"<<endl;
10    while(contador<3){
11      semaforo=verde;
12      cout<<"semaforo en verde, el peaton no pasa"<<endl;
13      cout<<"el peaton pide el paso con un boton"
14      <<"(presione enter)"<<endl;
15      cin.get();
16      semaforo=amarillo;
17      cout<<"semaforo en amarillo, precaucion"<<endl;
18      for(i=0;i<500000000;i++);
19      semaforo=rojo;
20      cout<<"semaforo en rojo, ALTO, pasa el peaton"<<endl;
21      for(i=0;i<500000000;i++);
22      87contador++;
23    }
24    return 0;
25  }
```

Tomamos en cuenta que al declarar un tipo de variable en base a una enumeración, como el semaforo de tipo colores, sólo se pueden asignar valores declarados dentro de esta, en este caso sólo toma valores rojo, amarillo o verde.

7.2. Uniones

El tipo de datos union puede contener datos de tipos y tamaños diferentes, esta variable almacena cualquier tipo de dato en una única localidad en memoria: la *unionedificio*

```
union edificio{
  int habitaciones;
  int despachos;
  char empresa(10);
};
```

declara un nuevo tipo de dato llamado ”edificio”. Para usar este tipo, hacemos la declaración: *edificiocasa*;

la variable casa tiene suficiente espacio en memoria para almacenar una cadena (el tipo de dato de mayor longitud dentro de la union). Es responsabilidad del programador extraer adecuadamente el tipo de dato deseado.

Nuestra variable está estructurada en la memoria como:

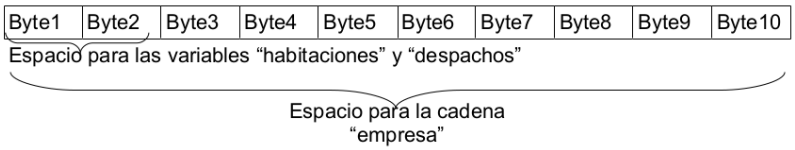


Figura 7.1: Espacio de memoria para Empresa

Cuando se almacena un entero y posteriormente una cadena, ésta última se escribe sobre el espacio de memoria asignado al entero, y por lo tanto, el contenido se pierde.

Ejecute el siguiente programa.

```
#include<iostream.h>
#include<iomanip.h>
union edificio{
  int habitaciones;
  int despachos;
  char empresa(10);
};

int main(){
  edificio casa, agencia, gobierno;
  cout<<"cuantas habitaciones tiene tu casa?"<<endl;
  cin>>casa.habitaciones;
  cout<<"escribe el nombre de la agencia en la\
  que trabajas"<<endl;
  cin.ignore();
  cin.getline(agencia.empresa,10);
  cout<<"cuantos despachos tiene la oficina de gobierno \
  de tu localidad?"<<endl;
  cin>>gobierno.despachos;
  cout<<"cuantos despachos tiene el edificio de la agencia "
  <<agencia.empresa<<"?"<<endl;
  89cin>>agencia.despachos;
  cout<<"edificio"<<setw(25)<<"habitaciones/despachos"
  <<endl;
  cout<<"casa"<<setw(18)<<casa.habitaciones<<endl;
  cout<<"agencia"<<setw(15)<<agencia.despachos<<endl;
  cout<<"gobierno"<<setw(14)<<gobierno.despachos<<endl;
  cin.ignore();
  cin.get();
  return 0;
}
```

Para acceder a los miembros de una variable de tipo union utilizamos el operador punto (.), seguido de la variable a extraer. Así, extraemos un entero de la variable ”casa” por medio de *casa.habitaciones*, si hacemos una extracción como casa.empresa no sabemos lo que aparecerá porque pueden existir otros datos en esa área de la memoria asignada a la cadena; y si se hiciese lo contrario, asignar una cadena y luego extraer un entero, no obtendremos lo que deseamos. Como con cualquier otro tipo de datos, realizamos operaciones con el contenido de los miembros de la union, operaciones básicas, envío como argumentos, conversiones, etc.

El uso de las variables de tipo union no es muy común.

7.3. Estructuras

En C++ se utiliza para definir una estructura. Que es un tipo de dato definido por el usuario que agrupa diferentes tipos de datos en una única entidad. Al igual que una clase, una

estructura puede contener miembros de datos y miembros de funciones, pero, por defecto, todos los miembros en una estructura son públicos.

Las estructuras son colecciones de elementos, los cuales pueden ser de diferente tipo y no comparten la misma área de memoria para los elementos.

La forma de declarar una estructura es más o menos como sigue:

```
struct nombre_estructura{
  tipo nombre_elemento1;
  tipo nombre_elemento2;
};
```

Es entonces cuando podemos declarar una variable del tipo de la estructura, funcionando el nombre de ella como un nuevo tipo de datos, de esta manera, la nueva variable tendrá cada uno de los elementos declarados en ella.

```
nombre_estructura
nueva_variable;
```

Para acceder a cada uno de sus elementos usamos el operador punto (.), seguido del nombre del elemento, de la misma forma que con las uniones.

Una estructura puede ser declarada de forma global o local, pero es más común hacerlo fuera de toda función.

El siguiente es un caso de su uso.

```
#include<iostream.h>
#include<math.h>
struct proyectil{
  float VelocidadInicial, angulo;
  float PosX, PosY;
};
float calculaCaida(proyectil cosa);
int main(){
  proyectil misil;
  misil.PosX=misil.PosY=0;
  float distancia;
  cout<<"Este programa calculará la distancia, en linea\
  recta,en la que caerá un proyectil"<<endl;
  cout<<"escribe la velocidad inicial para lanzarlo: ";
  cin>>misil.VelocidadInicial;
  cout<<"escribe ahora el angulo de lanzamiento: ";
  cin>>misil.angulo;
  distancia=calculaCaida(misil);
  cout<<"la distancia que recorre el proyectil es: "
  <<distancia<<" metros aproximadamente"<<endl;
  cin.ignore();
  cin.get();
}
float calculaCaida(proyectil cosa){
  int t=1;
  //tiempo
```

```
27    const float grav=9.81; //gravedad
28    cosa.angulo/=57.2957;
29    //transformamos a radianes
30    do{
31       cosa.PosX=(cosa.VelocidadInicial*cos(cosa.angulo)*t);
32       cosa.PosY=(cosa.VelocidadInicial*sin(cosa.angulo)*t)\(0.5*grav*t*t);
33       ++t;
34    }while(cosa.PosY>0);
35    return(cosa.PosX);
36 }
```

Debemos hacer notar que las estructuras no se pueden inicializar en su declaración.

Aunque las estructuras son una buena manera de mandar bloques de datos a una función, y es posible regresar estructuras (por la instrucción return), el coste es alto, el programa se hace lento, gasta CPU y memoria. Para ello, y gracias a que se trata como un nuevo tipo de variable, es posible hacer una paso por referencia, así evitamos que se hiciese una copia de toda la estructura, y sólo pasa la dirección.

La forma de hacer un paso por referencia es igual que con cualquier otro tipo de variables, sin embargo, al accesar a sus miembros debemos de hacerlo por medio del operador flecha (->). En nuestro ejemplo, cambiamos unas lineas:

```
1  .
2  .
3  cosa>angulo=57.2957;
4  //transformamos a radianes
5  do{
6     cosa>PosX=(cosa>VelocidadInicial*cos(cosa>angulo)*t);
7     cosa>PosY=(cosa>VelocidadInicial*sin(cosa>angulo)*t)\(0.5*grav*t*t);
8     ++t;
9  }while(cosa>PosY>0);
10 return(cosa>PosX);
11 .
12 .
```

Por las nuevas referencias de C++ ya descritas, incluimos un carácter '&' más en nuestro código para utilizarlo como referencia, en nuestra función (y la declaración por supuesto):

```
1  float calculaCaida(proyectil &cosa) {
2     .
3     .
4  }
```

basta para que se trate como una referencia sin necesidad de usar el operador flecha.

Las estructuras pueden tener por miembros a funciones, en cuyo caso se invoca de la misma forma como se accede a una variable. Le sugiero animarse a usarlas, intente hacer unos ejercicios con ellas.

Sin embargo, para esos casos es mejor y más útil utilizar otro tipo de datos definido en C++, se trata de las clases (class).

Veamos un programa que muestra la utilización de funciones como miembros de una estructura, y también es un ejemplo de las referencias C++.

```
#include<iostream.h>
#include<iomanip.h>
const int NumMaterias=11;
struct alumno {
  int boleta, semestre;
  int calificaciones(NumMaterias);
  void constancia(alumno & este){
    cout<<"constancia impresa"<<endl;
    cout<<"alumno con boleta: "<<este.boleta;
    cout<<setw(50)<<"del semestre numero: "
    <<este.semestre<<endl;
    cout<<endl<<"CALIFICACIONES DEL SEMESTRE: "
    <<endl;
    for(int i=0;i<NumMaterias;++i){
      cout<<"asignatura "<<i+1<<" : "
      <<este.calificaciones(i)<<endl;
    }
  }
  void inicia(alumno & este){
    for(int i=0;i<NumMaterias;++i)
    este.calificaciones(i)=0;
    este.boleta=0;
    este.semestre=0;
  }
};
int main(){
  alumno alan;
  alan.inicia(alan);
  cout<<"alan creado"<<endl;
  alan.boleta=2005630170;
  alan.semestre=5;
  alan.constancia(alan);
  return 0;
}
```

Ejemplo básico de declaración de una estructura en C++:

```
  struct MiEstructura {
    int miVariable;  // Miembro de datos
    void miFuncion() {
      // Código de la función
    }
  };
```

Definimos la estructura ”MiEstructura” que tiene un miembro de datos público ”miVariable” y un miembro de función ”miFuncion”.

Al igual que con una clase, puede crear objetos de una estructura con la palabra clave ”struct” seguida del nombre de la estructura y los paréntesis:

```
  MiEstructura objeto;
```

lo que queda como:

```
objeto.miVariable = 42;
objeto.miFuncion();
```

”struct” en C++ se utiliza para definir una estructura, que es un tipo de dato definido por el usuario que permite agrupar diferentes tipos de datos en una única entidad. A diferencia de una clase, los miembros de una estructura son públicos por defecto.

CAPÍTULO 8

CLASES

En C++, ”class” es la palabra clave para definir una clase, un tipo de dato definido por el usuario. Una clase es una plantilla para crear objetos que encapsulan datos y funciones relacionadas. Un mecanismo para modelar entidades del mundo real y organizar su comportamiento y propiedades en un lugar.

Las variables de los tipos fundamentales de datos no son suficientes para modelar objetos del mundo real. No se modela una caja mediante un int, pero se definen las variables largo, ancho y alto para representar las dimensiones de una caja. En una estructura de datos llamada Caja.

A continuación es posible definir variables de este nuevo tipo de la misma manera que con variables de tipos básicos. Se pueden crear, manipular y destruir tantos objetos de tipo Caja como se quiera. De esta manera, C++ incorpora la clase del paradigma de Orientación a Objetos y es clave para implementar este concepto.

Definamos una clase que instancia cajas:

```
1  class Caja {
2    double largo;
3    double ancho;
4    double alto;
5  };
```

Las variables de la clase se llaman datos miembro de la clase.

Declaremos la variable cajaGrande como instancia de la clase Caja.

Una vez definida la clase Caja, las declaraciones de variables de este tipo son estándares. Son instancias de la clase y se llaman objetos.

```
1  class MiClase {
2    private:
3    int miVariable;
4    public:
5    void setVariable(int valor) {
```

```
6         miVariable = valor;
7       }
8
9       int getVariable() {
10        return miVariable;
11      }
12    };
```

Aquí, definimos una clase llamada ”MiClase” que tiene un miembro de datos privado ”miVariable” y dos miembros de funciones públicos, ”setVariable” y ”getVariable”, para establecer y obtener el valor de ”miVariable” respectivamente.

Los datos y funciones de una clase son llamados miembros de la clase. Las funciones miembro también son llamadas métodos. A los datos miembro se les llama campos.

Según el especificador se establece el acceso a los miembros de la clase:

- private: los miembros privados son accesibles solamente desde adentro de la clase, únicamente los otros miembros podrán verlos.
- protected: los miembros protegidos únicamente pueden ser visibles por los demás miembros de la clase y de sus clases hijas o derivadas.
- public: los miembros públicos son accesibles desde cualquier lugar donde el objeto al que pertenecen sea visible.

Una vez que se ha definido una clase, se crean un objeto de esa clase utilizando la palabra clave ”class” seguida del nombre de la clase y los paréntesis:

```
1  MiClase objeto;
```

El objeto ”objeto” se crea según la estructura y el comportamiento definidos en ”MiClase”. Accede a los miembros de la clase utilizando el operador de punto (”.”):

```
1  objeto.setVariable(42);
2  int valor = objeto.getVariable();
```

”class” en C++ se utiliza para definir una plantilla para crear objetos que encapsulan datos y funciones relacionadas, lo que modela entidades del mundo real y organiza su comportamiento y propiedades en un sólo lugar.

8.1. Funciones miembro de una clase

Una función miembro de una clase tiene su definición o su prototipo dentro de la clase.

Las funciones miembro tienen acceso a todos los campos de la clase. Si proveemos a la clase Caja de una función que retorne el volumen de la caja, escribimos:

```
1  class Caja {
2    double largo;
3    double ancho;
4    double alto;
```

```
5     public:
6     double Volumen() {
7       return largo * ancho * alto;
8     }
9   };
```

En la definición de la clase, contamos con miembros privados (*private*), que es el acceso por defecto, y miembros públicos (*public*), que aparecen después de la etiqueta indicadora.

- Sólo las funciones dentro de la clase acceden a los miembros de la clase declarados como privados. No hay acceso directo a ellos por algún objeto o función fuera de la clase. Sólo las funciones miembro o friend (amigas) acceden a los datos privados de los miembros de una clase.

- Todos los miembros de la clase declarados como públicos estarán disponibles para todos. Otras clases también acceden a los miembros de datos y las funciones miembro declaradas públicas. Se accede a los miembros públicos de una clase desde cualquier parte del programa utilizando el operador de acceso directo a miembros (.) con el objeto de esa clase.

- Cuando un miembro de una clase es declarado como 'protected', no es accesible desde otros elementos no-miembros de la clase en el programa.

Si la definición de una clase miembro se realiza fuera de la clase, debe poner el prototipo de la función dentro de la clase. Como la definición de la función aparece afuera de la clase, hay una manera de decirle al compilador que esa función pertenece a la clase. Esto se realiza anteponiendo al nombre de la función el nombre de la clase, y separando ambos por el operador de resolución de alcance, '::'. Lo mostramos en el siguiente caso:

```
1   class Caja {
2     double largo;
3     double ancho;
4     double alto;
5     public:
6     double Volumen();
7   };
8   Caja::Volumen() {
9     return largo * ancho * alto;
10  }
```

Esto produce el mismo efecto que el caso anterior, pero no genera exactamente el mismo programa. En el segundo caso, todos los llamados a la función son tratados de manera habitual. Sin embargo, si la definición de la función está dentro de la clase, el compilador considera implícitamente a la misma como una función inline. Con ella, el compilador expande el código del cuerpo de la función en el lugar del llamado a la misma. Esto hace que el código sea más rápido.

8.2. Constructores

En algunos casos, inicializar todos los campos de una clase, demanda código. Además, como los campos son privados, no tiene acceso a ellos desde fuera de la clase.

Un constructor de clase es una función especial que crea nuevos objetos. Un constructor provee la oportunidad de inicializar los objetos al momento de ser creados, asignándoles valores válidos. Una clase puede contener varios constructores para crear objetos de varias maneras.

Los constructores siempre tienen el mismo nombre que la clase contenedora. Además, los constructores no tienen valor de retorno. A continuación, vemos un caso con un constructor para la clase Caja. Veamos el programa completo:

```
#include <iostream>
using std::cout;
using std::endl;
class Caja {
  double largo;
  double ancho;
  double alto;
  public:
  Caja (double lar, double an, double al) {
    largo = lar;
    ancho = an;
    alto = al;
  }
  double Volumen();
};
int main() {
  Caja unaCaja(0.58, 0.25, 0.55);
  cout << "Volúmen de la caja: " << unaCaja.Volumen();
}
```

Para invocar la función Volumen para la instancia de Caja unaCaja utilizamos el operador punto ('.') llamado de selección directa de miembros.

Un constructor que requiere un único argumento, no necesita ser llamado explícitamente. Caja v = 0.8; es lo mismo que Caja c(0.8);

8.3. El constructor por defecto

Si no requiere inicializar los campos en un constructor, utilice: Caja () ;

Es definido por el programador, pero en caso de no proveer un constructor en la clase, el compilador genera uno por defecto.

En ambos casos, el constructor se invoca en una sentencia: Caja caja1;

8.4. Destructores

Un destructor es una función miembro que destruye un objeto cuando ya no se lo necesita o cuando queda fuera de alcance. Tiene el mismo nombre que la clase pero precedido con tilde ( ). No retorna ningún valor y tampoco toma ningún parámetro, por lo que sólo puede existir un único destructor por clase.

Si no se define un destructor para una clase, el compilador siempre genera un destructor por defecto, pero éste no elimina objetos que han alocado memoria con el operador new. Por eso se debe definir el destructor con el correspondiente operador delete.

Por caso, si un constructor de una clase ClaseA es:

```
ClaseA(const char * texto = "Un mensaje") {
  mensaje = new char(strlen(texto) + 1);
  strcpy(mensaje, texto);
}
```

Entonces el destructor puede ser:

```
~ClaseA() {
  delete() mensaje;
}
```

El nombre de la clase.

8.5. Asignación de parámetros por defecto

De la misma manera que con las funciones ordinarias, se establecen valores por defecto en las funciones miembro y constructores:

```
Caja (double lar, double an, double al = 0.8) {
  largo = lar;
  ancho = an;
  alto = al;
}
```

8.6. Lista de inicialización

Para inicializar campos en un constructor, utilizamos una técnica alternativa llamada Lista de inicialización, cuyo formato es:

```
Caja (double lar, double an, double al = 0.8): alto(al), ancho(an),
longitud(logt) {}
```

Veamos:

```
Caja unaCaja(0.58, 0.25, 0.55);
Caja otraCaja = unaCaja;
```

Hemos inicializado un nuevo objeto llamado otraCaja idéntico al objeto unaCaja.

En las clases que tienen punteros o arreglos como campos miembro, el constructor por defecto no funciona como se espera, pues los punteros se copian y los del nuevo objeto quedan apuntando a los miembros del primero. En este caso, el programador debe proveer un constructor a la instancia.

Cuando se implementa un constructor a la instancia, el parámetro debe ser una referencia constante:

```
1  Caja(const Caja &unaCaja);
```

8.7. Acceso a campos miembro privados

Como mencionamos, fuera de la clase no accedemos a sus miembros privados. En un diseño orientado a objetos, el encapsulamiento es una característica esencial.[1] Debemos proteger los campos de la clase en la medida de lo posible, pero esto no significa que deban ser secretos. En la sección pública de la clase, definimos funciones para controlar el acceso a los datos privados. Éstas, regresan elementos del mismo tipo del campo miembro, las funciones se identifican con el prefijo get y no reciben ningún argumento. El objetivo simplemente devuelve el valor actual del atributo. En el siguiente caso se muestra cómo conocer el estado del campo alto desde fuera de la clase:

```
1  double getAlto() {
2    return alto;
3  }
```

En general, se definen funciones similares para cada campo que se quiera hacer disponible desde fuera de la clase sin perjudicar la seguridad de la misma.

8.8. Modificación de campos miembro privados

Si requerimos modificar valores de campos privados, se procede de manera similar, mediante la definición de funciones. Estas últimas no retornan valor, se las llama con el prefijo set y reciben como argumento el valor asignado al campo miembro o una referencia del mismo tipo.

El cuerpo consiste simplemente en asignar al campo en cuestión, el valor recibido como argumento.

En el siguiente caso se muestra cómo modificar el estado del campo alto desde fuera de la clase:

```
1  void setAlto(double al) {
2    alto = al;
```

[1] La encapsulación en C++ es un mecanismo que combina datos y funciones en una sola unidad, la clase, y proporciona un nivel de protección y control sobre los datos internos de la misma. Construimos programas modulares, seguros y fáciles de mantener.

```
3 }
```

8.9. Funciones amigas

Existen circunstancias donde se requiere que algunas funciones que no pertenecen a una clase, tengan acceso a los miembros privados de la misma. Tales funciones son llamadas amigas de la clase. Se las declara anteponiendo la palabra clave friend.

Tenga en cuenta que si bien el prototipo de las funciones amigas se declara dentro de la definición de la clase, éstas no son miembros de la misma, por tanto, los atributos de acceso no se aplican sobre ellas.

8.10. El puntero this

Cuando se ejecuta una función miembro, ésta contiene un puntero oculto llamado this, que apunta al objeto que hizo la llamada a la función. Cuando durante la ejecución se accede a un campo miembro de una clase, como la variable alto, la referencia es $this->alto$. El nombre del puntero this es agregado por el compilador a los miembros en las funciones. También, se hace mención explícita a este puntero.

8.11. Miembros estáticos de una clase

Los datos y las funciones miembro de una clase pueden ser declarados static.

Cuando se declara como estático un campo miembro, éste se crea una única vez y es compartido por todos los objetos de la clase. Cada objeto posee su propia copia de cada uno de los campos miembro ordinarios, pero existe sólo una instancia de los campos miembro estáticos, independientemente de la cantidad de objetos de la clase creados. Un uso común de un dato miembro estático es para contar cuántos objetos de una clase existen. Se declara en la sección pública de una clase: $staticintcantCajas$;

No puede inicializar un dato miembro estático en la definición de una clase. Por ello, la sentencia debe estar fuera de ella, según: $intCaja :: cantCajas = 0$;

Se puede referenciar un campo miembro estático desde la instancia de la clase o desde la clase misma:

```
1 cout << unaCaja.cantCajas;
2 cout << Caja::cantCajas;
```

Los campos estáticos de una clase se crean automáticamente cuando el programa comienza y se inicializan con el valor 0, es por ello que existen a pesar de que no se instancie algún objeto de la clase.

Cuando se declara como estática una función, se hace independiente de cualquier objeto de la clase. La ventaja es que existe y puede ser invocada antes de que se instancie cualquier objeto.

Estas funciones, al igual que los campos miembro estáticos, pueden llamarse desde un objeto instanciado o a partir del objeto.

8.12. Sobrecarga de operadores

La sobrecarga de operadores permite que los operadores estándares de C++ puedan programarse para operar con los tipos de datos o clases definidas por el usuario. No desarrollar nuevos operadores ni cambiar la precedencia de los existentes.

Obviamente, se espera que el comportamiento de un operador sobrecargado sea consistente con el uso normal que se le da al mismo y razonablemente intuitivo.

8.13. Funciones de sobrecarga miembros de la clase

Sólo algunos operadores no pueden sobrecargarse: (::, ?:, ., sizeof y .*)

Sobrecargamos el operador <(menor) para la clase Caja, considerando que caja es menor que la otra si tiene menor volumen.

```
1  bool Caja::operator< (const Caja &unaCaja) const {
2    return this->Volumen() < unaCaja.Volumen();
3  }
```

Aquí operator es una palabra clave, que junto con el operador (separado por espacio o no) definen el nombre de la función. Notése que la función define const porque no modifica los datos miembro de la clase. También el argumento debe ser una referencia constante.

El operador <es binario infijo, es decir, toma un operando a la izquierda y un operando a la derecha. En este caso, escribimos:

```
1    if (caja1 < caja2) cout << "La caja1 es menor que la caja2 ";
```

cuya expresión entre paréntesis es equivalente a:

```
1  caja1.operator<(caja2);
```

La sobrecarga de un operador binario mediante una función miembro, como en el caso anterior, determina implícitamente el operando de la izquierda como el objeto que hace la llamada (this).

Casos de estas sobrecargas son: asignación, comparaciones contra sí, incremento/decremento, etc.

8.14. Funciones de sobrecarga no miembros de la clase

Si se requiere sobrecargar un operador en el cual el primer operando no es el puntero this, la función de sobrecarga debe ser una función ordinaria o una función amiga. Esta última es

de utilidad cuando se necesita acceder a los miembros privados de la clase, por caso cuando se sobrecarga $<<$.

8.15. Clases derivadas

C++ proporciona construcciones del lenguaje que soportan directamente la idea del diseño de sistemas que indica: las clases deben usarse para modelar conceptos en el mundo del programador y de la aplicación.

Un concepto no existe en forma aislada. Coexiste con otros próximos a él y, gran parte de su fuerza radica en las relaciones entre ellos. Si se piensa en un auto, rápidamente se lo asocia con las nociones de: rueda, motor, conductor, peatón, camión, ambulancia, carreteras, gasolina, exceso de velocidad, etc. Puesto que, usamos clases para representar los conceptos, la cuestión es cómo representar las relaciones entre ellos.

Clasificamos los conceptos en dos dimensiones: pertenencia y variedad.

Puede decirse que un taxi es un tipo especial de auto (relación de variedad o, una relación de tipo ***es un***) y que, una rueda es parte de un auto (relación de pertenencia o, una relación de tipo ***tiene un***).

C++ implementa ambos tipos de relaciones. La combinación de este tipo de relaciones es potente: la relación de pertenencia facilita el agrupamiento físico de estructuras relacionadas lógicamente y la de variedad o herencia permite que estos grupos de aparición frecuente se reutilicen con facilidad en distintas abstracciones.

La noción de clase derivada y sus mecanismos del lenguaje asociados expresan relaciones jerárquicas, es decir, se utiliza para caracterizar aspectos comunes entre las clases. Los conceptos de círculo y triángulo están relacionados por cuanto ambos son formas, tienen en común el concepto de forma. As, debemos definir explícitamente las clases Circulo y Triangulo de modo que tengan en común una clase FormaGeometricaPlana.

Si representamos, en un programa, un círculo y un triángulo sin la forma geométrica plana se pierde algo esencial, la herencia. Lo que implica una relación de generalización/especialización en la que, una clase derivada especializa el comportamiento o estructura más general de sus clases bases.

Realmente esta es la herencia: si B no es un tipo de A, entonces B no debería heredar de A.

Las clases bases representan abstracciones generalizadas y, las clases derivadas son especializaciones en las que, los datos y funciones miembros de la clase base, tienen añadidos, modificaciones o incluso ocultaciones.

Una de las ventajas del mecanismo de derivación de clases es reutilizar código sin tener que escribirlo nuevamente; las clases derivadas utilizan código de la clase base de la jerarquía, sin tener que volver a definirlo en cada una de ellas.

La herencia declara abstracciones con economía de expresión. Ignorar las jerarquías de clases conduce a diseños mal formados y poco elegantes; sin herencia cada clase es una unidad independiente, desarrollada partiendo de cero; no existe relación entre sí puesto que, el desarrollador de cada una de ellas, proporciona funciones miembros según se le viniese en ganas.

Toda consistencia entre clases es el resultado de una disciplina por parte de los programadores. La herencia posibilita la definición de nuevo software de la misma forma en que se presenta un concepto a un recién llegado: comparándolo con algo que le resulta familiar.

Esto, conduce a una sencilla idea, que es una de las bases de la programación orientada a objetos.

8.16. Archivo .h y .cpp

Cada clase irá programada en dos archivos:

- Un archivo de cabecera (.h) en donde se declara la clase y se definen sus atributos y métodos, en este archivo solamente se declaran los miembros de la clase, para los métodos se escriben los prototipos de función y si es pertinente, para aquellas funciones cortas su implementación como inline.
- Un archivo fuente (.cpp) en donde se realiza la implementación de aquellos métodos de los que declaramos su prototipo en la definición de la clase.

De esta manera podremos tener a la vista, en el archivo de cabecera, las características de la clase y saber qué es lo que tenemos en un objeto de dicha clase; así mismo dejamos de lado los detalles más profundos de la implementación en el archivo fuente.

8.17. Clases derivadas

Considere la construcción de una aplicación que maneje cajas de distintos tipos. La clase Caja , define una caja en términos de sus dimensiones más un conjunto de funciones públicas que podrían aplicarse a objetos de tipo Caja para resolver problemas asociados a las mismas. La misma tendría este aspecto:

```
1  //Archivo cabecera Caja.h
2  class Caja {
3    public:
4    Caja(double l=1.0, double an=1.0, double al=1.0);
5    double volumen(void);
6    ~Caja(void);
7    private:
8    double largo;
9    double ancho;
10   double alto;
11 };
12 //Archivo Caja.cpp
13 #include "caja.h"
14 #include <iostream>
15 using std::cout;
16 using std::endl;
17 Caja::Caja(double l, double an, double al) {
18   largo=l;
```

```
19      ancho=an;
20      alto=al;
21    }
22    double Caja::volumen(void) {
23      return largo*ancho*alto;
24    }
25    Caja::~Caja(void) {
26      cout << "Se invoca al destructor de Caja" << endl;
27    }
```

Existen muchos tipos de cajas de base rectangular; se las podría diferenciar por la clase de material que contienen, del que están hechas y muchas otras formas. Definimos una clase particular de caja de base rectangular según características genéricas: alto, ancho, largo y, añadir otras características al tipo de caja básica para diferenciarlas del resto.

Habrá nuevas cosas para hacer con este tipo de cajas o redefinir algunas acciones.

Para representar todo esto definimos un tipo genérico de cajas de base rectangular con las características básicas y especificar otras clases de cajas como especializaciones de ellas. La figura 8.1 muestra un caso de estas relaciones que podrían definirse entre diferentes clases de cajas de base rectangular.

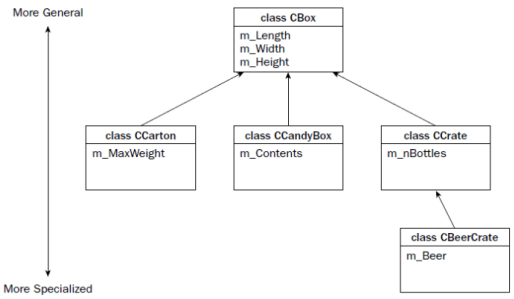


Figura 8.1: Relaciones entre cajas.

La clase CCrate (caja de botellas) deriva de la clase CBox (caja genérica) y, a la inversa, CBox es la clase base de CCrate. La clase CCrate tendrá los miembros propios de la clase CBox (alto, ancho, largo) además de los propios (nrobotellas, representando la cantidad de botellas que puede contener la caja). A menudo se representa gráficamente la derivación mediante un puntero desde la clase derivada (vea figura 8.1) hasta su clase base, para indicar que la clase derivada se refiere a su base (y no al contrario). A medida que nos movemos hacia abajo en el diagrama, las cajas son más especializadas.

Con frecuencia, una clase derivada hereda las propiedades de su clase base (herencia), lo mismo ocurre con las funciones miembros, con/sin restricciones, pero la clase derivada no hereda el constructor y destructor de la clase base como tampoco el operador de asignación sobrecargado en dicha clase ni funciones y/o datos miembros estáticos de la clase base; las clases derivadas tendrán sus propias versiones de constructor, destructor y operador de asignación sobrecargado.

Cuando se dice que estas funciones no se heredan no significa que no existen como miembros

en un objeto de la clase derivada; ellas existen aún para la parte de la clase base que conforma un objeto de la clase derivada.

A veces se denomina a la clase base superclase y a la clase derivada subclase. Sin embargo, esta terminología induce a error a quien observa que los datos de un objeto de clase derivada son un superconjunto de los datos de un objeto de la clase base. Una clase derivada es mayor que su clase base en el sentido que, usualmente contiene más datos y proporciona más funciones que esta última. En el caso de las cajas se invocaba automáticamente el constructor por defecto de la clase base (con los argumentos por defecto).

Para hacer utilizable la clase derivada CajaBotellas se contempla la posibilidad de especificar las dimensiones de la caja de botellas, además del número de botellas que pueda contener. Dicho constructor invoca explícitamente al constructor de la clase base para dar valores iniciales a los datos miembros que heredó de la clase base Caja; la invocación se realiza al final de la signatura del constructor de la clase derivada CajaBotellas luego de añadir :, puede observarse que esto coincide con la forma que puede utilizarse para inicializar datos miembros en un constructor, es decir, al respecto la clase base actúa exactamente igual que un miembro de la clase derivada.

```
1  #include "caja.h"
2  class CajaBotellas :
3  public Caja {
4    public:
5    CajaBotellas(int nro=1);
6    CajaBotellas(double l, double an, double al, int nro=1);
7    double volumen(void);
8    ~CajaBotellas(void);
9    private:
10   int nrobotellas;
11 };
12
13 #include "CajaBotellas.h"
14 #include <iostream>
15 using std::cout;
16 using std::endl;
17 CajaBotellas::CajaBotellas(int nro) {
18   cout << "Se invoca al constructor 1 de CajaBotellas" << endl;
19   nrobotellas=nro;
20 }
21 CajaBotellas::CajaBotellas(
22 double l, double an, double al, int nro):Caja(l, an, al) {
23   cout << "Se invoca al constructor 2 de CajaBotellas" << endl;
24   nrobotellas=nro;
25 }
26 double CajaBotellas::volumen(void) {
27   return 0.85*Caja::volumen();
28 }
29
30 CajaBotellas::~CajaBotellas(void) {
31   cout<<"Invoca al destructor de CajaBotellas" << endl;
32 }
```

Sino se invoca explícitamente al constructor de la clase base, el compilador dispone que al ejecutarse el código, se invoque al constructor por defecto de la clase base (no-arg); sino existe este tipo de constructor, el compilador indica un error.

Un constructor de la clase derivada puede especificar inicializadores sólo para sus propios miembros, no puede inicializar directamente miembros de su clase base directa, eso es responsabilidad de la invocación explicita al constructor de la misma.

Los objetos de una clase se construyen de abajo arriba: primero la base, luego los miembros y a continuación la clase derivada. Se destruyen en el orden contrario: primero la clase derivada, luego los miembros y a continuación la base.

Se añaden mensajes por pantalla a los constructores y destructores para poder observar esta secuencia al ejecutar la aplicación de prueba de dichas clases.

```
#include <iostream>
#include "CajaBotellas.h"
using std::cout;
using std::endl;
int main() {
  Caja caja1(4.0, 3.0, 2.0);
  CajaBotellas cajab1;
  CajaBotellas cajab2(6);
  CajaBotellas cajab3(1.0, 2.0, 3.0);
  cout<<"Volumen de caja1: "<<caja1.volumen()<<endl
  <<"Volumen de cajab1: "<<cajab1.volumen()<<endl
  <<"Volumen de cajab2: "<<cajab2.volumen()<<endl
  <<"Volumen de cajab3: "<<cajab3.volumen()<<endl;
  return 0;
}
```

8.17.1. Constructor de instancia

La copia de los objetos de una clase la definen el constructor de copia y las asignaciones.

Recuerde al definir un constructor de copia en cualquier clase que, debe recibir como parámetro una referencia a un objeto de dicha clase, para evitar un número infinito de invocaciones al mismo; lo que ocurriría si el argumento se pasa por valor y, por tanto, debe construirse una copia del mismo.

En la clase CajaBotellas se incluye un constructor de copia que se encarga en primer lugar de invocar explícitamente al constructor de copia de la clase base Caja.

```
class Caja {
  public:
  Caja(double l=1.0, double an=1.0, double al=1.0);
  Caja(const Caja &c);
  double volumen(void);
  ~Caja(void);
  private:
  double largo;
  double ancho;
  double alto;
```

```
};
```

```
#include <iostream>
#include "caja.h"
using std::cout;
using std::endl;
Caja::Caja(double l, double an, double al) {
  largo=l;
  ancho=an;
  alto=al;
  cout <<
  "Se invoca al constructor de Caja"
  << endl;
}

Caja::Caja(const Caja& c) {
  largo=c.largo;
  ancho=c.ancho;
  alto=c.alto;
  cout<<"Invocado el constructor por copia de Caja"<<endl;
}
double Caja::volumen(void) {
  return largo*ancho*alto;
}
Caja::~Caja(void) {
  cout << "Se invoca al destructor de Caja" << endl;
}

#include "caja.h"
class CajaBotellas : public Caja {
  public:
  CajaBotellas(int nro=1);
  CajaBotellas(double l, double an, double al, int nro=1);
  CajaBotellas(const CajaBotellas& cb);
  double volumen(void);
  ~CajaBotellas(void);
  private:
  int nrobotellas;
};

#include "CajaBotellas.h"
#include <iostream>
using std::cout;
using std::endl;
CajaBotellas::CajaBotellas(int nro) {
  cout << "Se invoca al constructor 1 de CajaBotellas" << endl;
  nrobotellas=nro;
}
CajaBotellas::CajaBotellas(
  double l, double an, double al, int nro):Caja(l, an, al) {
  cout<<"Se invoca al constructor 2 de CajaBotellas"<<endl;
  nrobotellas=nro;
}
```

```
52 CajaBotellas::CajaBotellas(const CajaBotellas& cb):Caja(cb) {
53   cout<<"Invocado constructor por copia de CajaBotellas"<<endl;
54   nrobotellas=cb.nrobotellas;
55 }
56 double CajaBotellas::volumen(void) {
57   return 0.85*Caja::volumen();
58 }
59 CajaBotellas::~CajaBotellas(void) {
60   cout << "Se invoca al destructor de CajaBotellas" << endl;
61 }
62 #include <iostream>
63 #include "CajaBotellas.h"
64 using std::cout;
65 using std::endl;
66 int main() {
67   Caja caja1(4.0, 3.0, 2.0);
68   CajaBotellas cajab1;
69   CajaBotellas cajab2(6);
70   CajaBotellas cajab3(1.0, 2.0, 3.0);
71   CajaBotellas cajab4(cajab3);
72   cout<<"Volumen de caja1: "<<caja1.volumen()<<endl
73   <<"Volumen de cajab1: "<<cajab1.volumen()<<endl
74   <<"Volumen de cajab2: "<<cajab2.volumen()<<endl
75   <<"Volumen de cajab3: "<<cajab3.volumen()<<endl;
76   return 0;
77 }
```

8.18. Jerarquía de clases

La jerarquía de clases conectadas mediante relaciones de herencia forma estructuras de apariencia arborescente.

Una clase puede heredar datos y funciones miembros de una o más clases base, aquí sólo consideraremos la herencia simple (heredar de una única clase base) y no la múltiple.

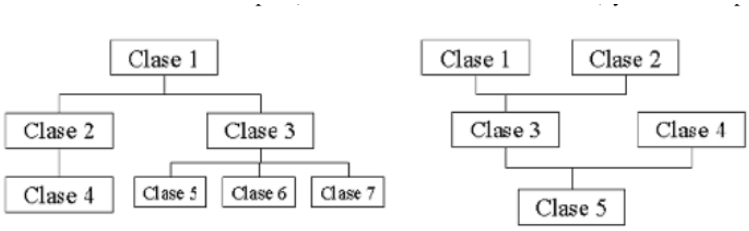


Figura 8.2: Herencia simple y múltiple

8.19. Tipos de campos

Una forma de determinar que un tipo de un objeto pertenece a una jerarquía de clases mediante herencia es incorporar una sentencia switch que luego invoque la acción apropiada. Esta solución expone a una variedad de problemas potenciales: el programador podría olvidarse de incluir una prueba de tipos cuando es obligatoria hacerla o de evaluar todos los casos posibles; al modificar un programa basado en este tipo de solución, agregando nuevos tipos, el programador podría olvidar insertar los nuevos casos en dicha sentencia. Cada adición o eliminación de una clase requiere la modificación de todos los switch del programa; rastrear estas instrucciones puede consumir tiempo y ser propenso a errores.

8.20. Funciones virtuales

Se añade a la clase Caja una función miembro mostrarVolumen( ) que se encarga de invocar a la función volumen( ) y mostrar en pantalla el valor calculado por dicha función. Esta función, por pertenecer a la interfaz pública de esta clase base es heredada por las clases derivadas de la misma. En la funcion main() de una clase de prueba, se invoca dicha función para mostrar los datos de una Caja y de una CajaBotellas, de las mismas dimensiones (en caso de CajaBotellas el volumen debería ser 85 % menor que la Caja de mismas dimensiones). Se observa que al ser invocada produce los mismos resultados.

```
1  void Caja::mostrarVolumen(void) const {
2    cout<<endl<<"El volumen de la caja es: "<<volumen();
3  }
4
5  #include <iostream>
6  #include "CajaBotellas.h"
7  using std::cout;
8  using std::endl;
9
10 int main() {
11   Caja caja1(4.0, 3.0, 2.0);
12   CajaBotellas cajab1(4.0, 3.0, 2.0);
13   Caja* cajap=0; //puntero nulo a la clase base Caja
14   caja1.mostrarVolumen();
15   cajab1.mostrarVolumen();
16   //si no es virtual se llama volumen de la clase Caja
17   cajap=&caja1; //puntero a objeto de la clase base
18   cajap->mostrarVolumen();
19   cajap=&cajab1; //puntero a objeto de la clase derivada
20   cajap->mostrarVolumen();
21   cout<<endl;
22   return 0;
23 }
```

La función mostrarVolumen() es miembro de la clase base y, en tiempo de compilación, se resuelve la invocación a volumen() que se realiza en el cuerpo de dicha función (enlace estático o temprano) y siempre se invoca a la versión de la clase base.

Para obtener el comportamiento esperado, la invocación correcta a la función volumen() de la clase base o de alguna clase derivada, determinada por el tipo de objeto asociado, debe resolverse en tiempo de ejecución, es decir, usar un enlace dinámico (o tardío) y no, arbitrariamente fijado por el compilador antes de ejecutar la aplicación donde figura este código.

C++ resuelve este tema con: las funciones virtuales.

En una clase base, una función virtual se declara usando la palabra reservada virtual.

Si existe, en alguna clase derivada otra definición para dicha función declarada como virtual en la clase base, le indica al compilador que no use enlace estático para la misma, sino determinará en tiempo de ejecución qué versión se invoca. A menudo a las funciones virtuales se les llama métodos.

Si se agrega la palabra virtual a la definición de la función volumen( ) la aplicación funciona correctamente. No es esencial agregar la palabra virtual a la versión de la función volumen() que está en la clase derivada CajaBotellas; es obligatorio hacerlo en la clase en la que se declara por primera vez. Se recomienda hacerlo para facilitar la lectura de la definición de las clases derivadas e informar cuáles funciones son virtuales y por tanto serán seleccionadas dinámicamente en tiempo de ejecución.

```
class Caja {
  public:
  Caja(double l=1.0, double an=1.0, double al=1.0);
  Caja(const Caja& c);
  virtual double volumen(void) const;
  //se usa virtual en la declaración de la clase
  void mostrarVolumen(void) const
  ~Caja(void);
  protected:
  double largo;
  double ancho;
  double alto;
};

#include <iostream>
#include "caja.h"
using std::cout;
using std::endl;

Caja::Caja(double l, double an, double al) {
  largo=l;
  ancho=an;
  alto=al;
  cout << "Se invoca al constructor de Caja" << endl;

}

Caja::Caja(const Caja& c) {
  largo=c.largo;
  ancho=c.ancho;
  alto=c.alto;
```

```
32    cout<<"Invocado el constructor por copia de Caja"<<endl;
33  }
34
35  double Caja::volumen(void) const {
36    return largo*ancho*alto;
37  }
38
39  void Caja::mostrarVolumen(void) const {
40    cout<<endl
41    <<"El volumen de la caja es: "<<volumen();
42  }
43
44  Caja::~Caja(void) {
45    cout << "Se invoca al destructor de Caja" << endl;
46  }
47
48  #include "caja.h"
49  class CajaBotellas :
50  public Caja {
51    public:
52    CajaBotellas(int nro=1);
53    CajaBotellas(double l, double an, double al, int nro=1);
54    CajaBotellas(const CajaBotellas& cb);
55    virtual double volumen(void) const;
56    ~CajaBotellas(void);
57    private:
58    int nrobotellas;
59  };
60
61  #include "CajaBotellas.h"
62  #include <iostream>
63  using std::cout;
64  using std::endl;
65  CajaBotellas::CajaBotellas(int nro) {
66    cout << "Se invoca al constructor 1 de CajaBotellas" << endl;
67    nrobotellas=nro;
68  }
69
70  CajaBotellas::CajaBotellas(
71    double l, double an, double al, int nro):Caja(l, an, al) {
72    cout << "Se invoca al constructor 2 de CajaBotellas" << endl;
73    nrobotellas=nro;
74  }
75
76  CajaBotellas::CajaBotellas(const CajaBotellas& cb):Caja(cb) {
77    cout<<"Invocado constructor por copia de CajaBotellas"<<endl;
78    nrobotellas=cb.nrobotellas;
79  }
80
81  double CajaBotellas::volumen(void) const {
82    return 0.85*largo*ancho*alto;
83  }
84
```

```
85 CajaBotellas::~CajaBotellas(void) {
86   cout << "Se invoca al destructor de CajaBotellas" << endl;
87 }
```

8.21. Constructores y destructores

Algunas clases derivadas necesitan constructores. Si la clase base tiene un constructor hay que invocarlo y si, dicho constructor necesita argumentos, hay que proporcionarlos.

Aunque los constructores de la clase base no se heredan, son usados para crear la parte heredada de la clase base, de un objeto de la clase derivada y, esta tarea es responsabilidad del constructor de la clase base.

Para que una función se comporte como virtual en una clase derivada, debe tener el mismo nombre, lista de parámetros y tipo de retorno que la versión de la clase base. Si la función en la clase base es const, en la clase derivada también lo deberá ser. Si estos requisitos no se cumplen, el mecanismo de función virtual no funcionará y se usará enlace estático en tiempo de compilación.

Se puede usar una función virtual en una clase aunque no se derive otra clase de ella. Tampoco es necesario que una clase derivada proporcione una versión de una función definida como virtual en su clase base, sólo debe proporcionarse una versión adecuada si es necesaria sino, se usa la versión de la clase base.

Las funciones virtuales constituyen un mecanismo extraordinariamente poderoso; cuando se usan de manera adecuada proporcionan un buen grado de estabilidad a un programa en evolución. El término polimorfismo es una de las características más importantes de la programación orientada a objetos.

Mediante el polimorfismo, un nombre (tal como la declaración de una variable) puede denotar objetos de muchas clases diferentes, relacionadas por una clase base común; cualquier objeto denotado por este nombre es, por tanto, capaz de responder a algún conjunto común de operaciones de distintas formas.

Existe polimorfismo cuando interactúan la herencia y el enlace dinámico.

Para obtener polimorfismo en C++ debe usarse funciones virtuales, invocadas a través de punteros o referencias. Cuando se manipulan los objetos directamente (en lugar de hacerlo de esta forma indirecta) su tipo exacto es conocido en tiempo de compilación y no es necesario polimorfismo en tiempo de ejecución.

La invocación a funciones que están redefinidas en clases derivadas, pero usando el operador de resolución de ámbito (::) asegura que no se usa el mecanismo de las funciones virtuales, siempre se invoca a la versión de la clase base.

Cada clase que utiliza funciones virtuales tiene un vector de punteros, uno por cada función virtual, llamado v-table. En el caso que una clase derivada no tenga versión propia de una función definida como virtual, el puntero apuntará a la versión de la clase más próxima en la jerarquía de clases, que tenga una definición propia de dicha función virtual. Es decir se busca primero, al invocar una función virtual, en la propia clase, luego en la clase anterior en

la jerarquía y así, subiendo hasta encontrar una clase que tenga la definición de la función buscada. Cada objeto que se crea, de alguna clase que tenga funciones virtuales, contiene un puntero oculto a la v-table de su clase.

Las funciones virtuales no pueden declararse static puesto que, carecen del puntero this y, las funciones virtuales lo necesitan para la mecánica de su funcionamiento.

8.22. Punteros a objetos y funciones virtuales

A un puntero a un objeto de una clase base puede también asignársele un objeto de la clase derivada. Por tanto puede usarse, punteros a objetos de la clase base conjuntamente con funciones virtuales, para obtener distintos comportamientos dependiendo del tipo real de objeto al que se apunta.

En el caso de las cajas ahora el método mostrarvolumen() se invoca mediante punteros a objetos de la clase base Caja:

```
#include <iostream>
#include "CajaBotellas.h"
using std::cout;
using std::endl;
int main() {
  Caja caja1(4.0, 3.0, 2.0);
  CajaBotellas cajab1(4.0, 3.0, 2.0);
  Caja* cajap=0; //puntero nulo a la clase base Caja
  caja1.mostrarVolumen();
  cajab1.mostrarVolumen();//sino es virtual se llama volumen de la clase Caja
  cajap=&caja1; //puntero a objeto de la clase base
  cajap->mostrarVolumen();
  cajap=&cajab1; //puntero a objeto de la clase derivada
  cajap->mostrarVolumen();
  cout<<endl;
  return 0;
}
```

Ahora el mecanismo de funciones virtuales garantiza que se invoque la versión correcta de volumen().

8.23. Referencias a objetos y funciones virtuales

Análogamente a los punteros, las referencias a objetos de la clase base pueden utilizarse para referirse a objetos de clases derivadas. Este hecho combinado con el uso de funciones virtuales hará que, en tiempo de ejecución, se invoque la versión correcta de dicha función virtual.

Con respecto al caso de las cajas, se añade en el archivo de prueba de las clases creadas, aparte de main( ) otra función Output( ) que toma como argumento una referencia a objetos de la clase base Caja, dicha función utiliza esta referencia para invocar a la función virtual volumen( )

```
#include <iostream>
#include "CajaBotellas.h"
using std::cout;
using std::endl;
void Output(const Caja& c);
//prototipo de función para ver referencia +func.virtuales
int main() {
  Caja caja1(4.0, 3.0, 2.0);
  CajaBotellas cajab1(4.0, 3.0, 2.0);
  Caja* cajap=0;
  //puntero nulo a la clase base Caja
  caja1.mostrarVolumen();
  cajab1.mostrarVolumen();
  //sino es virtual se llama volumen de la clase Caja
  cajap=&caja1; //puntero a objeto de la clase base
  cajap->mostrarVolumen();
  cajap=&cajab1; //puntero a objeto de la clase derivada
  cajap->mostrarVolumen();
  Output(caja1);
  Output(cajab1);
  cout<<endl;
  return 0;
}

void Output(const Caja& c){
  c.mostrarVolumen();
}
```

8.24. Funciones virtuales puras y clases abstractas

Muchas clases tales como FormaGeometricaPlana representan conceptos abstractos para los cuales no pueden existir objetos, sólo tienen sentido para derivar de ellas formas geométricas planas concretas, tales como círculos. Determinadas operaciones, como calcular área, sólo tienen sentido para formas geométricas concretas. Para este tipo de funciones la solución es usar funciones virtuales puras. Una función virtual pura se logra mediante el inicializador = 0.

Una clase con, al menos una función virtual pura, se denomina clase abstracta, no pudiéndose crear objetos de la misma; sólo puede utilizarse como interfaz y clase base para otras derivadas de ellas. No puede utilizarse una clase abstracta como tipo a pasar como parámetro o como tipo devuelto por una función. Sí, está permitido definir punteros o referencias que, posteriormente serán inicializados con objetos de clases derivadas concretas.

Una función virtual pura que no esté redefinida en una clase derivada, permanece como función virtual pura por lo que, dicha clase derivada se convierte también en clase abstracta. Un uso importante de las clases abstractas es proporcionar una interfaz sin exponer ningún detalle de implementación.

Al caso de las cajas se le añade una clase base abstracta Contenedor puesto que contiene el método virtual puro volumen( ). Ahora la clase Caja, un tipo particular de contenedor, se

deriva de dicha clase base abstracta; en Caja el método volumen( ) está perfectamente definido puesto que se crearán objetos de dicha clase. Se define también una clase Lata, que es otro tipo de Contenedor y, también se define el volumen( ) de acuerdo a la fórmula:

$$h * \pi * r^2$$

```
class Contenedor {
//contenedor genérico, clase abstract

  public:
  virtual double volumen() const=0;
  //virtual sólo en declaración
  void mostrarVolumen() const;
  //punteros a la clase base no se invocan a los destructores correctos
  virtual ~Contenedor(void);
  //para invocar el destructor correcto agrego virtual
};

#include "Contenedor.h"
#include <iostream>
using std::cout;
using std::endl;
void Contenedor::mostrarVolumen() const{
  cout<<endl
  <<"El volumen de la caja es: "<<volumen()<<endl;
}
Contenedor::~Contenedor(void) {
  cout<<"Invocado el destructor de Contenedor"<<endl;
}
#include "Contenedor.h"
class Caja: public Contenedor {
  public:
  Caja(double l=1.0, double an=1.0, double al=1.0);
  Caja(const Caja& c);
  virtual double volumen(void) const;
  ~Caja(void);
  protected:
  double largo;
  double ancho;
  double alto;
};
#include <iostream>
#include "Caja.h"
using std::cout;
using std::endl;
Caja::Caja(double l, double an, double al) {
  largo=l;
  ancho=an;
  alto=al;
  cout << "Se invoca al constructor de Caja" << endl;
}
Caja::Caja(const Caja& c) {
```

```
47    largo=c.largo;
48    ancho=c.ancho;
49    alto=c.alto;
50    cout<<"Invocado el constructor por copia de Caja"<<endl;
51  }
52  double Caja::volumen(void) const {
53    return largo*ancho*alto;
54  }
55  Caja::~Caja(void) {
56    cout << "Se invoca al destructor de Caja" << endl;
57  }
58  #include "contenedor.h"
59  class Lata :
60  public Contenedor {
61    public:
62    Lata(double al=4.0, double d=2.0);
63    virtual double volumen() const;
64    ~Lata(void);
65    protected:
66    double diametro;
67    double alto;
68  };
69  #include "Lata.h"
70  #include <iostream>
71  using std::cout;
72  using std::endl;
73  const double PI = std::atan(1.0)*4;
74  Lata::Lata(double al, double d):alto(al), diametro(d){
75    cout<<"Invocado constructor de Lata"<<endl;
76  }
77  double Lata::volumen() const {
78    return PI* diametro*diametro*alto;
79  }
80  Lata::~Lata(void) {
81    cout<<"Invocado destructor de Lata"<<endl;
82  }
83  #include "caja.h"
84  class CajaBotellas :
85  public Caja {
86    public:
87    CajaBotellas(int nro=1);
88    CajaBotellas(double l, double an, double al, int nro=1);
89    CajaBotellas(const CajaBotellas& cb);
90    virtual double volumen(void) const;
91    //agrego para usar ref constante en Output
92    ~CajaBotellas(void);
93    private:
94    int nrobotellas;
95  };
96  #include "CajaBotellas.h"
97  #include <iostream>
98  using std::cout;
99  using std::endl;
```

```
100 CajaBotellas::CajaBotellas(int nro) {
101   cout << "Se invoca al constructor 1 de CajaBotellas" << endl;
102   nrobotellas=nro;
103 }
104 CajaBotellas::CajaBotellas(
105   double l, double an, double al, int nro):Caja(l, an, al) {
106   cout << "Se invoca al constructor 2 de CajaBotellas" << endl;
107   nrobotellas=nro;
108 }
109 CajaBotellas::CajaBotellas(const CajaBotellas& cb):Caja(cb) {
110   cout<<"Invocado constructor por copia de CajaBotellas"<<endl;
111   nrobotellas=cb.nrobotellas;
112 }
113 double CajaBotellas::volumen(void) const {
114   return 0.85*largo*ancho*alto;
115 }
116 CajaBotellas::~CajaBotellas(void) {
117   cout << "Se invoca al destructor de CajaBotellas" << endl;
118 }
119 #include <iostream>
120 #include "CajaBotellas.h"
121 #include "Lata.h"
122 using std::cout;
123 using std::endl;
124 int main(void) {
125   //puntero clase base abstracta Contenedor apunta a Caja nuevo
126   Contenedor* pc1=new Caja(2.0, 3.0, 4.0);
127   //puntero clase base abstracta Contenedor apunta a objeto Lata nuevo
128   Contenedor* pl1=new Lata(6.5, 3.0);
129   Lata l1(6.5, 3.0);
130   //crea otra lata igual a la anterior
131   CajaBotellas cb(2.0, 3.0, 4.0);//crea caja de botellas
132   pc1->mostrarVolumen();
133   pl1->mostrarVolumen();
134   cout<<endl;
135   //limpia el espacio asignado dinámicamente
136   delete pc1;
137   delete pl1;
138   //inicializa pc1 con la dirección de la lata l1
139   pc1=&l1;
140   pc1->mostrarVolumen();
141   //ahora el puntero pc1 apunta a la dirección de CajaBotellas cb
142   pc1=&cb;
143   pc1->mostrarVolumen();
144   return 0;
145 }
```

CAPÍTULO 9

PUNTEROS

Los punteros en C++ (o apuntadores) son quizás uno de los temas que más confusión causan al momento de aprender a programar en C++, sin embargo verá que no es para tanto y que todo depende de dos elementos: el signo & (ampersand) y el * (asterisco). En este apartado aprenderá usar punteros y que además son de gran ayuda al momento de necesitar valores y estructuras dinámicas, como crear un array dinámico, con dinámico nos referimos a que su tamaño puede ser establecido en tiempo de ejecución y lo mismo se puede hacer con las matrices (que en realidad son un array multidimensional).

Un puntero o apuntador es una variable que da referencia a una región de memoria; en otras palabras es una variable cuyo valor es una dirección de memoria. Si se tiene una variable 'p'de tipo puntero que contiene una dirección de memoria en la que se encuentra almacenado un valor 'v'se dice que 'pápunta a 'v'.

El programador utiliza los punteros para guardar datos en memoria en muchas ocasiones, de la forma que se describe a continuación.

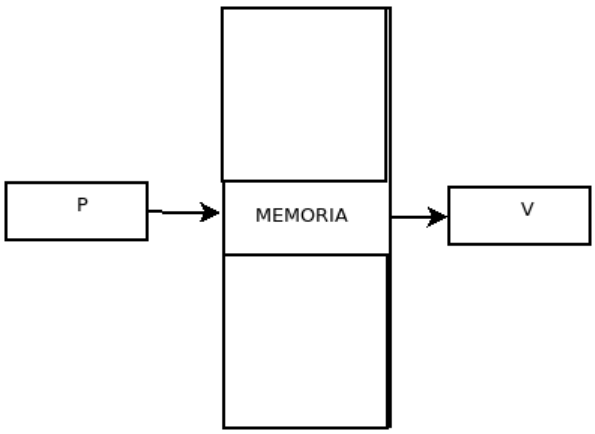


Figura 9.1: Entra - salida a memoria

Trabajar con punteros no implica la manipulación de los datos en sí, sino manejar las direcciones de memoria en la cuales estos residen.

Los punteros en C++ (o apuntadores) son quizás uno de los temas que más confusión causan al momento de aprender a programar en C++, sin embargo verá que no es para tanto y que todo depende de dos elementos: el signo & (ampersand) y el * (asterisco). En este apartado aprenderá usar punteros y que además son de gran ayuda al momento de necesitar valores y estructuras dinámicas, como crear un array dinámico, con dinámico nos referimos a que su tamaño puede ser establecido en tiempo de ejecución y lo mismo se puede hacer con las matrices (que en realidad son un array multidimensional).

Comencemos con un caso sencillo y su correspondiente explicación. Trataremos de explicar cada componente, su sintaxis y el final, cómo aprovechar dichas funcionalidades para nuestro beneficio junto con algún caso práctico.

```
1  int variable;
2  int * apuntador = &variable;
3  *apuntador = 20;
4  delete () apuntador;
5  puntero = NULL;
```

Hemos creado y usado nuestro primer puntero ¿Notó el uso del asterisco y del ampersand? espero que sí y además de eso hay otros detalles que debemos considerar, veamos:

- Creamos un entero.
- Creamos una apuntador a la posición en memoria de ”variable”
- Le asignamos un valor a esa posición de memoria.
- Después de operar con punteros es necesario liberar la memoria.

Comencemos con un caso sencillo y explicar cada componente, su sintaxis y el final, cómo aprovechar dichas funcionalidades para nuestro beneficio junto con algún caso práctico.

```
1 int variable;
2 int *apuntador = &variable;
3 *apuntador = 20;
4 delete() apuntador;
5 puntero = NULL;
```

Hemos creado y usado nuestro primer puntero ¿Notó el uso del asterisco y del ampersand? espero que sí y además de eso hay otros detalles que debemos considerar, veamos:

- Creamos un entero.
- Creamos una apuntador a la posición en memoria de ”variable”.
- Le asignamos un valor a esa posición de memoria.
- Después de operar con punteros es necesario liberar la memoria.

9.1. Detalles al crear y usar punteros

El tipo de dato del apuntador debe coincidir con el de la variable cuya posición en memoria apuntan. En el caso vemos que tanto variable como apuntador son enteros.

Siempre que usamos el puntero, anteponemos el asterisco (*) para usar el valor de la posición de memoria apuntada.

De no usar el asterisco el comportamiento será impredecible. Haremos uso de la dirección de memoria más no del valor en ella.

Después de usar un puntero, especialmente si trabajamos con arreglos o matrices, es MUY recomendable liberar la memoria utilizada con la función delete (tal como en el caso presentado).

Un puntero o apuntador puede ser de cualquier tipo de dato, inclusive los usamos con tipos complejos.

Ahora que conocemos algunos trucos y detalles sobre los apuntadores en C++, definimos formalmente la utilidad del operador & y del asterisco.

9.2. Los punteros y el ampersand

El ampersand (&) es un operador de C++ y es utilizado para los punteros. Con este operador obtenemos la dirección de memoria de una variable cualquiera, que es lo que utilizan los punteros para referenciar valores.

9.3. Los apuntadores y el asterisco

El asterisco es el operador por excelencia de los punteros. Su utilidad radica en que, si el valor de dicho apuntador corresponde a una dirección de memoria, el asterisco la resuelve y accede al valor almacenado allí. Resumiendo, un apuntador es una dirección de memoria (un número) y el asterisco obtiene el valor en dicha dirección.

Veamos otro caso con cada elemento detallado paso a paso

```
1 char *apuntador = NULL;
2 char letra;
3 apuntador = &letra;
4 *apuntador = 'x';
5 cout << letra;
```

Vemos que podemos usar cualquier tipo de dato, que un puntero se inicializa independientemente y luego se le asigna su referencia correspondiente. Nótese que al asignar, no utilizamos el asterisco, pues estamos definiendo la dirección de memoria y no el valor en ella (el * resuelve la dirección de memoria).

9.4. Parámetros por referencia

Al enviar un parámetro a una función todo lo que se haga con dicho parámetro dentro de ella, NO tiene efecto por fuera. Sí una función tiene una variable cuyo valor es diez y, al interior de la función le sumamos un cinco, después de su ejecución el valor de la variable es diez. Lo que pasó al interior de la función se quedó allí. Para solucionar esto, si queremos que el valor cambie, usamos punteros para pasar no el valor del parámetro sino una referencia a éste (paso por referencia). Veamos:

```
#include ''iostream''
#include ''stdio.h''
using namespace std;
int funcion(int valor) {
  valor = valor + 10;
  return valor;
}
int funcionPunteros(int* valor) {
  *valor = *valor + 10;
  return *valor;
}
int main() {
  int numero = 10;
  cout << ''Antes de funcion '' << numero << ''\n'';
  funcion(numero);
  cout << ''Despues de funcion '' << numero << ''\n'';
  cout << ''Antes de funcionPunteros '' << numero << ''\n'';
  funcionPunteros(&numero);
  cout << ''Despues de funcionPunteros '' << numero << ''\n''; //20 (10+10)
  system(''pause'');
  return 0;
}
```

Como podrás comprobar si ejecutas el código del ejercicio al llamar a ”funcion” sólo enviamos el valor y por ende éste no es modificado por fuera de la función; con ”funcionPunteros” estamos manipulando la posición en memoria del parámetro recibido (usamos el *) y por ende al ejecutarla, el valor de la variable se modifica. De ese modo, hicimos el primer ejercicio con punteros en C++ y comprendemos el paso por referencia.

9.5. Array dinámico

Por medio de apuntadores, creamos arreglos o vectores dinámicos, es decir, un array al cual se le define su tamaño o capacidad durante la ejecución del código y no antes, con lo cual el tamaño es impuesto por el usuario.

Para este caso, regresamos a los arreglos o vectores: desarrollamos un programa para almacenar los títulos y autores de diferentes libros. El usuario es el encargado de suministrar la información de cada libro. Usamos punteros, así que el usuario dirá cuántos libros desea ingresar, y no suponer que sólo ingresa 5 libros. Veamos:

```
#include ''iostream''
#include ''stdio.h''
#include ''string''
using namespace std;
int main() {
  string* titulos = NULL;
  string* autores = NULL;
  int tamanio ;
  cout << ''Cuantos libros desea ingresar?'';
  string entrada;
  getline(cin, entrada); //Se asigna el valor ingresado
  tamanio = stoi(entrada); //Se transforma la entrada en número
  titulos = new string(tamanio);
  autores = new string(tamanio);
  cout << ''Por favor ingrese la siguiente información de los Libros: \n'';
  for(int i = 0; i < tamanio; i++) {
    cout << ''\n******* Libro '' << i + 1 << ''********:\n'';
    cout << ''Titulo: '';
    //cin >> titulos(i);
    getline(cin, titulos(i));
    cout << ''Autor: '';
    //cin >> autores(i);
    getline(cin, autores(i));
  }
  delete () titulos;
  delete () autores;
  titulos = NULL;
  autores = NULL;
  system(''pause'');
  return 0;
}
```

Así, tenemos dos punteros, uno para los autores y otro para los títulos. Haciendo uso de ellos, el usuario define la cantidad de libros a ingresar, en tiempo de ejecución y de manera dinámica.

9.6. Matrices dinámicas

Así como lo hicimos con los array, tenemos matrices dinámicas y definimos su tamaño, número de filas o de columnas (o las dos) según sea necesario.

Para esto, siguiendo el caso de los libros, pero usamos una matriz, en vez de dos vectores, tal y como se solucionó en la sección de arreglos:

```
#include ''iostream''
#include ''stdio.h''
#include ''string''
using namespace std;
int main() {
  int cols = 2;
  string** libros;
  int tamanio ;
  cout << ''Cuantos libros desea ingresar?'';
```

```
10    string entrada;
11    getline(cin, entrada);
12    tamanio = stoi(entrada);
13    libros = new string*(tamanio);
14    cout << ''Por favor ingrese la siguiente información de los Libros: \n'';
15    string titulo ,autor;
16    for(int i = 0; i < tamanio; i++)  {
17      libros(i) = new string(cols);
18      cout << ''\n******* Libro '' << i + 1 << ''********:\n'';
19      cout << ''Titulo: '';
20      getline(cin,titulo);
21      cout << ''Autor: '';
22      getline(cin,autor);
23      libros(i)(0) = titulo;
24      libros(i)(1) = autor;
25    }
26    for (int i = 0; i < tamanio; ++i)    {
27      delete () libros(i);
28    }
29    delete () libros;
30    system(''pause'');
31    return 0;
32  }
```

Este ejercicio es el perfecto para aclarar dudas o darse cuenta si comprendemos el concepto de apuntadores y su aplicación para arrays dinámicos. Debido a que la cantidad de columnas es fija, no se lo pedimos al usuario, simplemente lo declaramos con valor dos. Luego, tenemos el puntero, en una matriz con lo que tiene otros punteros dentro, por eso se usamos doble asterisco. Luego, el usuario define la cantidad de libros y, al momento, de inicializar la fila indicamos que es un arreglo de punteros, por eso se usa el * en la línea 23. Luego, al interior del ciclo, cuando estamos llenando la matriz, indicamos que cada fila está compuesta por un array de punteros de tamaño dos (dos columnas) y construimos nuestra matriz dinámica.

Debes notar también que la liberación de la memoria es un más laboriosa, pues debemos ir fila por fila liberando la memoria de las columnas creadas y luego liberar la fila completa. Ahí podrás notar la diferencia en eficiencia y uso de memoria al usar arreglos o usar matrices.

> **Nota:** La instrucción new utilizada con los strings es necesaria para asignar el espacio en memoria para esa colección de elementos. No te preocupes por ello de momento, solo asegúrate de usarla y ya el lenguaje se hace cargo del resto.

Muy bien, es todo en esta función. En este momento, conoces qué es y cómo hacer un puntero en C++, para qué sirve el ampersand y el asterisco cuando hablamos de apuntadores en C++ y cómo crear arrays dinámicos y matrices.

9.7. Usos de los punteros inteligentes

Los punteros inteligentes se definen en el espacio de nombres std del archivo de encabezado <memory>. Son cruciales en la expresión de programación RAII o Resource Acquisition Is Initialization. El objetivo principal de esta expresión es asegurarnos que la adquisición de recursos ocurre al mismo tiempo que se inicializa el objeto, de manera que todos los recursos del objeto se creen y se dispongan en una sola línea de código.

En la práctica, el principio básico RAII consiste en proporcionar la propiedad de cualquier recurso asignado por montón (por caso, memoria asignada dinámicamente o identificadores de objetos del sistema) a un objeto asignado a la pila cuyo destructor contiene código para eliminar o liberar el recurso, además de cualquier código asociado de limpieza.

En la mayoría de los casos, cuando se inicializa un puntero o un identificador de recursos sin formato para apuntar a un recurso real, el puntero se pasa inmediatamente a un puntero inteligente. En el lenguaje C++ actual, los punteros sin formato se utilizan únicamente en pequeños bloques de código de ámbito limitado, bucles o funciones del asistente donde el rendimiento es crucial y no hay posibilidad de confusión sobre la propiedad.

En el caso siguiente comparamos una declaración de puntero sin formato con una declaración de puntero inteligente.

```
void UseRawPointer() {
  Song* pSong = new Song(''Nothing on You'', ''Bruno Mars'');
  delete pSong;
}

void UseSmartPointer() {
  unique_ptr<Song> song2(new Song(L''Nothing on You'', L''Bruno Mars''));
  wstring s = song2->duration_;
}
```

Como se muestra en el caso, un puntero inteligente es una plantilla de clase que se declara en la pila y se inicializa con un puntero sin formato que apunta a un objeto asignado por montón. Una vez que se inicializa el puntero inteligente, se convierte en propietario del puntero sin formato. Esto significa que el puntero inteligente es responsable de eliminar la memoria que el puntero sin formato especifica. El destructor del puntero inteligente contiene la llamada de eliminación y, dado que el puntero inteligente se declara en la pila, su destructor se invoca cuando el puntero inteligente sale del ámbito, incluso si se produce una excepción en alguna parte que se encuentre más arriba en la pila.

Para acceder al puntero encapsulado, utilice los conocidos operadores de puntero ->y *, que la clase del puntero inteligente sobrecarga para devolver el puntero sin formato encapsulado.

La expresión del puntero inteligente de C++ se asemeja a la creación de objetos en lenguajes como C#: se crea el objeto y después se permite al sistema que se ocupe de eliminarlo en el momento correcto. La diferencia es que ningún recolector de elementos no utilizados independiente se ejecuta en segundo plano; la memoria se administra con las reglas estándar de ámbito de C++, de modo que el entorno en tiempo de ejecución es más rápido y eficaz.

Nota: Cree siempre punteros inteligentes en una línea de código independiente,

nunca en una lista de parámetros, para que no se produzca una pérdida de recursos imperceptible debido a algunas reglas de asignación de la lista de parámetros.

En el caso siguiente se muestra cómo utilizar un tipo de puntero inteligente unique_ptr de la biblioteca estándar de C++ para encapsular un puntero a un objeto grande.

```
class LargeObject {
  public:
  void DoSomething(){}
};
void ProcessLargeObject(const LargeObject& lo){}
void SmartPointerDemo() {
  std::unique_ptr<LargeObject> pLarge(new LargeObject());
  pLarge->DoSomething();
  ProcessLargeObject(*pLarge);
}
```

En el caso se muestran los pasos básicos siguientes para utilizar punteros inteligentes.

Declare el puntero inteligente como variable automática (local). (No utilice la expresión new o malloc en el propio puntero inteligente).

En el parámetro de tipo, especifique el tipo al que apunta el puntero encapsulado.

Pase un puntero sin formato al objeto new-ed en el constructor de puntero inteligente. (Algunas funciones de utilidad o constructores de puntero inteligente hacen esto por usted.)

Utilice los operadores sobrecargados ->y * para tener acceso al objeto.

Deje que el puntero inteligente elimine el objeto.

Los punteros inteligentes están diseñados para ser lo más eficaces posible tanto en términos de memoria como de rendimiento. Por caso, el único miembro de datos de unique_ptr es el puntero encapsulado. Esto significa que unique_ptr tiene exactamente el mismo tamaño que ese puntero, cuatro u ocho bytes. El acceso al puntero encapsulado a través del puntero inteligente sobrecargado * y los operadores >no es mucho más lento que el acceso directo a los punteros sin formato.

Los punteros inteligentes tienen sus propias funciones miembro, a las que se accede mediante la notación ”punto”. Por caso, algunos punteros inteligentes de la biblioteca estándar de C++ tienen una función miembro de restablecimiento que libera la propiedad del puntero. Esto es útil cuando se quiere liberar la memoria que es propiedad del puntero inteligente antes de que el puntero inteligente salga del ámbito, tal y como se muestra en el caso siguiente.

```
void SmartPointerDemo2() {
  std::unique_ptr<LargeObject> pLarge(new LargeObject());
  pLarge->DoSomething();
  pLarge.reset();
}
```

Los punteros inteligentes proporcionan un mecanismo para acceder al puntero sin formato. Los punteros inteligentes de la biblioteca estándar de C++ tienen una función miembro get con este propósito y CComPtr tiene un miembro de clase p público. Si proporciona acceso directo al puntero subyacente, puede utilizar el puntero inteligente para administrar la memoria en el

propio código y continuar pasando el puntero sin formato en un código que no admita punteros inteligentes.

```
void SmartPointerDemo4() {
  std::unique_ptr<LargeObject> pLarge(new LargeObject());
  pLarge->DoSomething();
  LegacyLargeObjectFunction(pLarge.get());
}
```

9.8. Clases de punteros inteligentes

En la sección siguiente se resumen los distintos tipos de punteros inteligentes disponibles en el entorno de programación de Windows y se describe cuándo utilizarlos.

Utilice estos punteros inteligentes como primera opción para encapsular punteros a los objetos estándar de C++ (POCO).

unique$_p$tr

Permite exactamente un único propietario del puntero subyacente. Utilice esta opción como predeterminada para los objetos POCO, a menos que sepa con certeza que necesita un objeto shared_ptr. Puede moverse a un nuevo propietario, pero no se puede copiar ni compartir. Sustituye a auto_ptr, que está desusado. Comparado con boost::scoped_ptr, unique_ptr es pequeño y eficiente; el tamaño es un puntero y admite referencias rvalue para la inserción y recuperación rápidas de colecciones de bibliotecas estándar de C++. Archivo de encabezado: <memory>.

shared$_p$tr

Puntero inteligente con recuento de referencias. Utilícelo cuando desee asignar un puntero sin formato a varios propietarios, por caso, cuando devuelve una copia de un puntero desde un contenedor pero desea conservar el original. El puntero sin formato no se elimina hasta que todos los propietarios de shared_ptr han salido del ámbito o, de lo contrario, han renunciado a la propiedad. El tamaño es dos punteros: uno para el objeto y otro para el bloque de control compartido que contiene el recuento de referencias. Archivo de encabezado: <memory>.

weak$_p$tr

Puntero inteligente de caso especial para usarlo junto con shared_ptr. weak_ptr proporciona acceso a un objeto que pertenece a una o varias instancias de shared_ptr, pero no participa en el recuento de referencias. Utilícelo cuando desee observar un objeto, pero no quiere que permanezca activo. Es necesario en algunos casos para interrumpir las referencias circulares entre instancias de shared_ptr. Archivo de encabezado: <memory>.

9.8.1. Punteros inteligentes para objetos COM (programación clásica de Windows)

Cuando trabaje con objetos COM, encapsule los punteros de interfaz en un tipo de puntero inteligente adecuado. Active Template Library (ATL) define varios punteros inteligentes para propósitos diferentes.

También puede usar el tipo de puntero inteligente _com_ptr_t, que el compilador utiliza cuando crea clases contenedoras de archivos .tlb. Es la mejor opción si no desea incluir los archivos de encabezado ATL.

CComPtr(*clase*)

Utilice esta opción a menos que no puede usar ATL. Realiza el recuento de referencias mediante los métodos AddRef y de Release.

CComQIPtr(*clase*)

Se parece a CComPtr, pero también proporciona la sintaxis simplificada para llamar a QueryInterface en objetos COM.

CComHeapPtr(*clase*)

Puntero inteligente a objetos que utilizan CoTaskMemFree para liberar memoria.

CComGITPtr(*clase*)

Puntero inteligente para las interfaces que se obtienen de la tabla de interfaz global (GIT).

_com_ptrt(*Clase*)

Se parece a CComQIPtr en funcionalidad, pero no depende de los encabezados ATL.

9.8.2. Punteros inteligentes ATL para objetos POCO

Además de punteros inteligentes para los objetos COM, ATL también define punteros inteligentes y colecciones de punteros inteligentes para objetos estándar de C++ (POCO). En la programación clásica de Windows, estos tipos son alternativas útiles a las colecciones de bibliotecas estándar de C++, cuando no se requiere portabilidad de código o cuando no se quieren mezclar los modelos de programación de la biblioteca estándar de C++ y ATL.

CAutoPtr(*clase*)

Puntero inteligente que exige una propiedad única al transferir la propiedad en la copia. Puede compararse con la clase std::auto_ptr en desuso.

CHeapPtr(*clase*)

Puntero inteligente para objetos asignados mediante la función de C malloc.

CAutoVectorPtr(*clase*)

Puntero inteligente para matrices que se asignan mediante new[].

CAutoPtrArray(*clase*)

Clase que encapsula una matriz de elementos CAutoPtr.

CAutoPtrList(*clase*)

Clase que encapsula los métodos para manipular una lista de nodos de CAutoPtr. Debes notar

también que la liberación de la memoria es un poco más trabajosa, pues debemos ir fila por fila liberando la memoria de las columnas creadas y luego liberar la fila completa. Ahí podrás notar la diferencia en eficiencia y uso de memoria al usar arreglos o usar matrices.

> **Nota:** La instrucción new utilizada con los strings es necesaria para asignar el espacio en memoria para esa colección de elementos. No te preocupes por ello de momento, solo asegúrate de usarla y ya el lenguaje se hará cargo del resto.

9.9. Operadores referencia-desreferencia

Un apuntador se define como una variable que contiene una dirección de memoria.

Para declarar un apuntador se hace uso del operador desreferencia o indirección (*), no se debe confundir con la multiplicación.

Al declarar un apuntador debemos de hacerlo de la siguiente forma:

```
1 tipo *nombre;
2 int *MiApuntador;
```

Ya habíamos dicho anteriormente que el nombre de un arreglo apunta al primer elemento de éste, es decir contiene la dirección en memoria del elemento 0.

Tomando en cuenta esto, podemos ver que si hacemos:

```
1 int arreglo(10);
2 int *apuntador;
3 apuntador = arreglo;
```

sucede que, pasamos la dirección del primer elemento del arreglo a la variable ”apuntador” que está determinada para almacenar una dirección de memoria.

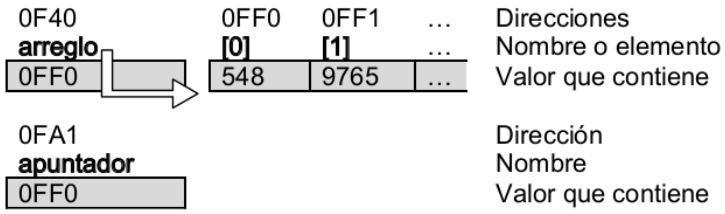


Figura 9.2: Direccionamiento de apuntadores

Las variables también tienen una dirección en memoria, y para saber cual es debemos utilizar el operador referencia o dirección (&).

```
1 int MiVariable=20;
2 int *MiApuntador;
3 MiApuntador = &MiVariable;
```

Así ”MiApuntador” contendrá la dirección de ”MiVariable”.

El uso de los apuntadores sirve para hacer uso de una variable con la cual no se tiene contacto directo, por medio del operador desreferencia. Por ejemplo, podemos asignar un nuevo valor a esta variable referenciada.

$*MiApuntador = 9$; por medio de la instrucción anterior se está asignando un valor de 9 a la variable que apunta ”MiApuntador”. Es decir que ”MiVariable” ahora contiene un 9.

Un pequeño ejemplo nos muestra lo dicho hasta el momento:

```
#include<iostream.h>
int main(){
  int variable=10;
  int *apuntador;
  cout<<"la variable contiene un "<<variable<<endl;
  apuntador=&variable;
  cout<<"variable esta en la direccion "
  <<apuntador<<endl;
  *apuntador=15;
  cout<<"la variable ahora tiene un "<<variable<<endl;
  cout<<"ahora tecle un numero"<<endl;
  cin>>*apuntador;
  cout<<"y ahora contiene un "<<*apuntador<<endl;
  cin.ignore();
  cin.get();
  return 0;
}
```

Podemos usar un apuntador para asignar cantidades (o lo que sea) a la variable referenciada ya sea por instrucción o por medio del teclado.

En la última instrucción cout se imprime el valor de ”lo que apunta el ‘apuntador’ ”.

9.10. Aritmética de operadores

Quizá ya se dio cuenta, así como la multiplicación es el inverso de la división, en el caso de los apuntadores sucede algo parecido, si se coloca una instrucción como la que sigue:

```
cout<< ''la variable contiene '' << &*variable;
```

o

```
cout<< ''la variable contiene '' <<*&variable;
```

tendrán el mismo resultado que si no se colocaran los operadores * y &.

Un arreglo se considera un apuntador (por las razones anteriormente mencionadas), y por lo tanto se aplica la aritmética de operadores en ellos, en realidad no es muy diferente de cómo lo hacíamos antes, pero ahora utilizaremos el ” * ”.

Un array puede ser operado mediante incrementos y decrementos:

$MiArreglo[7]$;

También puede hacerse:

$*(MiArreglo+7)$;

Aunque no es nada recomendable utilizar este método para tratar los arreglos, nos da la idea de que se pueden hacer operaciones de suma y resta sobre los apuntadores. Si hacemos una instrucción como: $apuntador+=6$; la dirección de memoria almacenada cambia a 3 lugares adelante. Por consiguiente, también se puede aplicar una resta.

Cuando encontramos una instrucción del tipo: $*puntero = una_variable + 6$; se está haciendo una asignación a "lo que apunta 'puntero"'. Se modificará el valor de la variable referenciada por "puntero". Igual pasa con asignaciones de otro tipo: $*puntero* = 3$; que en este caso se hace una multiplicación de la variable referenciada por 3 y luego se vuelve a asignar.

9.11. Arreglos de apuntadores (arreglos de cadenas)

Habíamos dejado pendiente un momento este tema, porque era mejor ver primero el concepto de apuntadores, ahora que ya sabemos en que consiste, podemos declarar e inicializar una cadena de caracteres de la siguiente forma: $char*cadena = "alguna_palabra"$

Si quisiéramos declarar una cadena con un determinado número de elementos, es necesario utilizar uno de los métodos ya conocidos.

Un arreglo de cadenas en realidad se trata de un arreglo de apuntadores porque una cadena es un apuntador al primer carácter. Entonces, un arreglo de cadenas se vería de la siguiente forma.

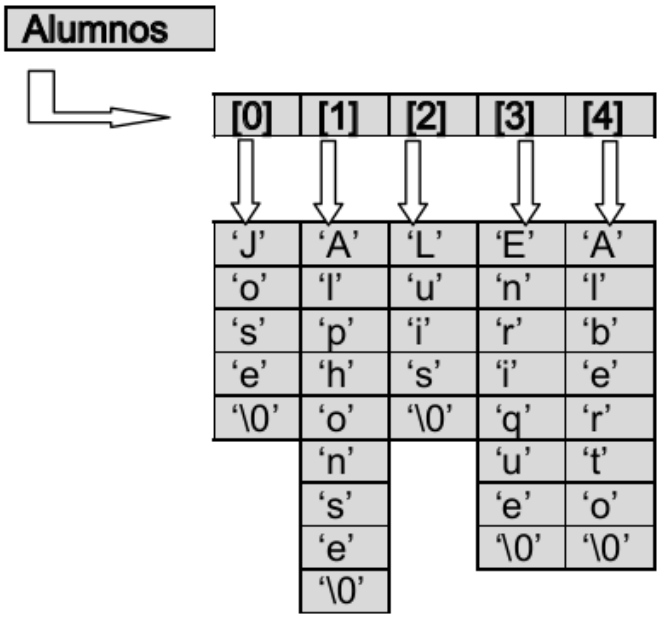


Figura 9.3: Arreglo de cadena de caracteres.

La declaración del arreglo "Alumnos" sería de la siguiente forma: $char*Alumnos[4] =$

$''Jose'',''Alphonse'',''Luis'',''Enrique'',''Alberto''$;

Notemos que de esta forma no es necesario colocar el carácter '\0' al final de cada cadena, el compilador la coloca por sí sólo. También se pueden utilizar las otras formas, que ya conocemos, de inicializar un arreglo de dos dimensiones y se puede especificar el tamaño de las cadenas.

$charMasAlumnos[4][10]$;

Veamos entonces que Alumnos[0] contiene la dirección en memoria que ocupa la letra 'J'.

Y para acceder a cada carácter del arreglo declarado de cualquier forma, se utilizan los subíndices, por ejemplo, asignaremos con la siguiente instrucción un carácter contenido en el arreglo a otra variable de tipo char: $caracter = Alumnos[1][5]$;

Así asignamos el carácter 'n' a la variable "caracter" Un ejemplo nos ayudará a visualizar su utilización. El siguiente programa se trata de un juego, o por lo menos la idea de cómo empezarlo. Consiste en teclear, lo más rápidamente, las palabras que vayan apareciendo en la pantalla, se cuenta con un determinado tiempo para hacerlo, si no se consigue dentro de ese intervalo continuará a la siguiente palabra. El programa termina cuando presionan la tecla "Esc", pero bueno, aquí está el código.

```
#include<conio.h>
#include<stdlib.h>
#include<iostream.h>
#include<time.h>
int main(){
  int cont=0,i=0,num_aleat=0;
  char car;
  char *cadena(6)={"cinco","siete","texto","poema","radio",\
    "raton"};
  while(car!=27){
    srand((unsigned)time(NULL));
    num_aleat=rand()%6;
    cout<<endl<<endl<<cadena(num_aleat)<<endl;
    for(cont=0,i=0;(cont<100000)&&(cadena(num_aleat)(i)!='\0');++cont){
      if(kbhit()){
        car=getch();
        if(car==cadena(num_aleat)(i)){
          cout<<car;
          ++i;
        }
        fflush(stdin);
      }
    }
  }
  cout<<endl<<"juego terminado"<<endl;
  cin.get();
  return 0;
}
```

El programa define primeramente una matriz de cadenas, para luego empezar el juego mediante un ciclo while. Repetirá todas las instrucciones dentro del bloque mientras no se oprima la tecla "Esc". Se calcula un número aleatorio menor a 6 y se almacena en una variable tipo int,

la cual servirá para que se mande a pantalla, mediante la instrucción cout, una cadena al azar, note que se le está enviando la dirección del primer carácter de la cadena.

Después se usa un ciclo for, en el cual se inicializa la variable que contará el tiempo (cont) y la variable de desplazamiento por los elementos de la tabla (i). Este ciclo se detendrá cuando el contador de tiempo llegue a 100000 o cuando se llegue al carácter '\0' en la cadena.

Dentro de ese ciclo for se encuentra la parte más interesante del programa, usamos una función (kbhit) que se encarga de detectar si se ha oprimido una tecla, esta tecla no aparecerá en la pantalla pero sí se quedará dentro del flujo, si detecta que se ha oprimido entonces, usando la función getch(), leerá y guardará en la variable car aquel carácter que se quedó dentro del flujo, pero sin presentarlo en pantalla, si el carácter leído corresponde con el carácter de la posición referida por "i", entonces si se presentará en pantalla y se continuará con el siguiente carácter (++i). Saliendo del condicional se limpia el flujo de entrada (función fflush() definida en stdlib.h) para evitar que se quede algún residuo o basura.

Las Funciones kbhit() y getch() no son parte de la librería estándar de C++, estas librerías son propiedad de la compañía Borland® y que se distribuyen en sus productos Borland/TurboC++. Sin embargo, debido a su amplio uso, están disponibles también versiones para otros productos como lo son el Visual C++ de Microsoft®, el Dev-C++ de Bloodshed, y por supuesto para el compilador GCC.

9.12. Paso de argumentos por referencia

Anteriormente vimos como se hacia la modificación de un arreglo mediante el paso por referencias simulado. En el caso de las variables de tipo int, float, etc., también es posible hacer uso de ellas mediante el paso por referencia.

El paso por referencia consiste en pasar la dirección en memoria que ocupa esa variable para que la función que la recibe pueda hacer uso de ella, sin necesidad de hacer una copia como ocurre con el paso por valor. Para mandar la dirección es necesario utilizar el operador & dentro de la invocación a la función. Y el prototipo y definición de la función debe especificar que será un apuntador mediante el uso del *.

$tipo_que_regresanombre_funcion(tipo_apuntador * nombre, otro_tiponombre, ...)$;

Lo anterior es la forma de una definición que hace uso de apuntadores. La función puede devolver o no un valor mediante una instrucción return(). Puede contener varios parámetros apuntador o no apuntador. Cuando se va a hacer uso de esa función, la invocación es de la siguiente forma: $nombre_funcion(\&nombre, \dots)$;

En el caso de los arreglos recordemos que estos son la dirección, y no es necesario poner el operador &.

$nombre_funcion(arreglo[numero_cadena])$;

La instrucción anterior, siendo "arreglo" un array de cadenas, envía la dirección del primer carácter de la cadena numero_cadena.

El siguiente ejemplo muestra estos aspectos.

```
1  #include<iostream.h>
```

```
2  void aMayusculas(char *cadena);
3  void cambiaNum(int *numero);
4  int main(){
5    char palabra[10];
6    int numero;
7    cout<<"teclea una cadena "<<endl;
8    cin.getline(palabra,10);
9    aMayusculas(palabra);
10   cout<<endl<<palabra<<endl;
11   cout<<"ahora teclea un numero"<<endl;
12   cin>>numero;
13   cambiaNum(&numero);
14   cout<<"este numero se cambio a: "<<numero;
15   cin.ignore();
16   cin.get();
17   return 0;
18 }
19 void aMayusculas(char *cadena){
20   for(int i=0;cadena[i]!='\0';++i)
21   cadena[i]+=('A' -'a');
22 }
23 void cambiaNum(int *numero){
24   *numero *= *numero;
25 }
```

El código no es difícil de entender, quizá la parte en la que cusa un poco de confusión podría ser la función ”cambiaNum”, simplemente lo que hace es multiplicar ”lo que apunta ‘numero”’ por sí mismo, es decir lo eleva a cuadrado. Revise la tabla de precedencia de operadores si se tienen más dudas sobre esto.

Gracias a las referencias de C++ es posible hacer una función cambiaNum() con una sintaxis más sencilla y de una manera más amigable, utilizando el ’&’.

```
1  void cambiaNum(int & numero){
2    numero *= numero;
3  }
```

de esta manera hace el mismo efecto que la que habiamos definido, cuando se invoque sólo se hará: *cambiaNum*(*numero*); como si fuese una llamada normal.

CAPÍTULO 10

MEMORIA DINÁMICA (HEAP)

La asignación dinámica de la memoria hace referencia a reservar un espacio en el montón (heap) para almacenar una información. Esto requiere usar la palabra reservada new para establecer el tamaño del puntero:

```
int *p_numero = new int;  // Espacio reservado dinámicamente
*p_numero = 55;  // Asignación en el espacio dinámico
std::cout << *p_numero << std::endl;  // 55
```

Los datos en el montón (heap) existen mientras el programador no los elimine manualmente de la memoria, por ello vaciamos el espacio con la palabra delete[1], devolviendo al programa la memoria ocupada al sistema operativo: $delete p_numero$;

También, una vez vaciado el espacio usamos el puntero nullptr para indicar en el código un reseteo[2], eso nos indica que reservamos la memoria antes de utilizar el puntero: $p_numero = nullptr$;

Por defecto, el espacio reservado dinámicamente contiene información aleatoria, pero asignamos el valor directamente en la definición:

```
int *p_numero{int(55)};
int *p_numero = new int(55);
int *p_numero = new int{55};
int *p_numero{new int{55}};
```

Por cierto, no eliminamos un puntero vacío, podría pasar cualquier cosa en la memoria y dejar nuestro programa inservible:

```
delete p_numero;
delete p_numero;
```

En resumen:

[1] Borrar
[2] Liberar

```
// Variable en la pila (stack) gestionada automáticamente
int numero = 10;

// Puntero reservado dinámicamente en el montón (heap)
int *p_numero = new int(55);
// Debemos vaciar la memoria manualmente
delete p_numero;
// Y recomendablemente asignarle nullptr para dejar un aviso
p_numero = nullptr;
```

10.1. Punteros colgantes (dangling pointers)

Un puntero colgante es uno que no apunta a una dirección de memoria válida, por lo que al intentar dereferenciarlo y utilizarlo, resulta un comportamiento indefinido.

Esto es algo muy malo, pues se pueden generar situaciones imprevistas en el código y posiblemente un error de ejecución que termine con el programa bloqueado.

Hay tres tipos de punteros colgantes:

- Punteros no inicializados.
- Punteros eliminados.
- Múltiples punteros apuntando a la misma dirección de memoria.

Veamos un caso:

```
int *p_numero;
delete p_numero;
int *ptr1 {new int{10}};
int *ptr2 {ptr1};
delete ptr1;
ptr1;
ptr2;
```

Para prevenir los punteros colgantes podemos aplicar estas directrices:

- Inicializar siempre los punteros.
- Reiniciar los punteros después de eliminarlos con el valor nullptr.
- Comprar que un puntero no es nullptr antes de utilizarlo.
- En múltiples punteros con la misma dirección, asegurarnos de que la propiedad esté clara.

Inicialización para evitar un puntero colgante

```
int *p_numero{new int{55}};
delete p_numero;
p_numero = nullptr;
if (p_numero != nullptr) std::cout << *p_numero << std::endl;
int *p_maestro{new int{77}};
int *p_esclavo{p_maestro};
p_maestro = nullptr;
if (p_maestro != nullptr) std::cout << *p_esclavo << std::endl;
```

10.2. Error al reservar la memoria dinámica

De vez en cuando, en raras ocasiones, la reserva de memoria dinámica puede fallar, por ejemplo cuando no hay suficiente memoria disponible.

De forma predeterminada, cuando se utiliza el operador new para intentar asignar memoria y la función no puede hacerlo, se produce una excepción bad_alloc. Utilizando un bloque try catch capturamos la excepción y actuar en consecuencia:

```
try {
  int* numeros {new int(100000000000000)};
} catch(std::exception& ex){
  std::cout << ''Error al reservar la memoria: '' << ex.what() << ''\n'';
}
```

También es posible utilizar un modificador *nothrow* como argumento para que en lugar de fallar se devuelva un puntero nulo. Esta constante *nothrow* es un valor de tipo $nothrow_t$ con el único propósito de no lanzar la excepción bad_alloc:

```
int *numeros{new (std::nothrow) int(100000000000000)};
if (numeros != nullptr)
  std::cout << ''Memoria reservada correctamente'' << ''\n'';
else
  std::cout << ''Error al reservar la memoria'' << ''\n'';
```

10.3. Pérdidas de memoria (memory leaks)

Un memory leak es una situación que puede ocasionar la pérdida del acceso a la memoria que hemos almacenado dinámicamente en el montón (heap).

Por ejemplo, si almacenamos una información en el montón y luego redireccionamos ese puntero a otra variable. En ese caso perdemos el acceso a la información original almacenada en la memoria, es decir, habrá ocurrido un memory leak[3]:

```
int *p_numero{new int{66}};
int numero{99};
p_numero = &numero ;
```

[3]Una fuga de memoria (memory leak) es un error de software que ocurre cuando un bloque de memoria reservada no es liberada en un programa de computación.

Esto no ocurre sólo al sobreescribir la dirección de una variable en la pila (stack), pasará lo mismo al definir una nueva variable en el montón (heap):

```
int *p_numero{new int{66}};
p_numero = new int{99};
```

Debemos prestar mucha atención a esto, sobretodo con las definiciones en bloques, pues fuera de ellos perderemos el acceso al puntero pero el valor seguirá existiendo en la memoria:

```
#include <iostream>

int main() {
  int *p_numero{new int{66}};
}
```

Recuerde eliminar y reiniciar la memoria para garantizar espacio al sistema:

```
#include <iostream>

int main() {
  int *p_numero{new int{66}};

  delete p_numero;      // Borrado
  p_numero = nullptr;   // Y reinicio
}
```

Una fuga de memoria (más conocido por el término inglés memory leak) es un error de software que ocurre cuando un bloque de memoria reservada no es liberada en un programa de computación.

10.4. Arreglos almacenados dinámicamente

Se trata de arreglos que se reservan en el montón (heap) mediante el operador new. Permiten el uso de la constante *std* :: *nothrow* para evitar el error en caso de no disponer de la memoria requerida.

Definimos estos arreglos de distintas formas:

```
// Definimos un tamaño para el arreglo
size_t size{10};

// Inicialización con sobrecarga creciente (los valores son basura)
int *p_numeros = new (std::nothrow) int(size);

// Inicialización con sobrecarga no arrojadiza (los valores son 0)
int *p_numeros = new (std::nothrow) int(size){};

// Inicialización con asignación parcial (los demás valores son 0)
int *p_numeros = new (std::nothrow) int(size){11, 22, 33, 44, 55};
```

Recordemos que mediante *std* :: *nothrow* el puntero será nullptr en caso de no haber reservado la memoria correctamente, podemos comprobarlo directamente sin la igualdad, ya que se tomará como 0:

```
if (p_numeros) {
  // Si no es un puntero nulo podemos utilizarlo sin problema
}
```

Para liberar la memoria de un arreglo dinámico debemos especificar los corchetes:

```
delete[] p_numeros;
p_numeros = nullptr;
```

Por último, recordemos que estos arreglos al ser punteros no permiten la sintaxis del for con elementos porque no almacenan propiedades internas como su longitud:

```
  for(int numero : p_numeros) { }
```

En su lugar, los recorremos con índices o con la aritmética de punteros mediante la suma de posición con un for a partir de la longitud almacenada anteriormente:

```
for (int i = 0; i < size; i++) {
  std::cout << p_numeros[i] << std::endl;
  std::cout << *(p_numeros + i) << std::endl;
}
```

10.5. Diferencias entre memoria dinámica y estática

Este no es un concepto exclusivo de C++. Forma parte de los conceptos básicos. Ocurre que en los lenguajes modernos o ”de alto nivel” (entrecomillado porque C++ se considera de alto nivel, como todos excepto el ensamblador) por lo que la gestión de la memoria no está en manos del programador. Pero incluso así, hay diferencia entre la memoria estática y la dinámica.

Dicho esto, justamente en C++ y, sobre todo, en las últimas versiones, se han introducido cosas para definir explícitamente qué puede ser tratado como memoria estática.

Por esto, usamos punteros básicos y no clases de la librería estándar del C++.

La memoria estática es aquella cuyos datos no pueden (o deben) ser modificados bajo ningún concepto por el programador.

Otra manera de verlo es que la memoria estática se crea en tiempo de compilación (cuando se crea el programa) y la dinámica en tiempo de ejecución (mientras el programa está funcionando).

Haciendo esto:

```
  char *txt = ''Hola mundo!'';
```

En el caso anterior, se crea una memoria estática. No importa el ámbito. Da igual que sea algo global, dentro de una clase, o una función o en un if. La memoria usada por el texto ”hola mundo!” no puede (o debe) ser alterada.

Si intenta algo así:

```
  txt[1] = 'O';
```

Obtiene un error en tiempo de ejecución.

En tiempo de compilación, también obtiene un error o una advertencia si el compilador considera que el ”lvalue”, es una variable estática.

La memoria dinámica es la que se reserva en tiempo de ejecución. Tanto la memoria del montón (heap) como la de pila (stack) son memorias dinámicas:

```
int main() {
  func(''hola'');
}

void func(char *str) {
  char *str2 = new char(strlen(str)+1);
  strcpy(str2, str);
  delete str2;
}
```

Donde str2 es un puntero a una memoria dinámica, reservada en el montón (heap).

Por lo tanto, esto:

```
  str2(1) = 'O';
```

es legal.

Bien, no hemos dicho nada del parámetro ’str’de dentro de la función ’func’. ¿Qué es dinámica o estática?

Una cosa es la memoria usada por el texto ”hola mundo!” y otra es la memoria usada por el puntero hacia esa memoria.

Para que su programa pueda acceder al texto ”hola mundo!” necesita un puntero (o apuntador) Este puntero es un número. Como tal, está también guardado en memoria. Y tiene las dos posibilidades: estática o dinámica. Y se le aplican las mismas normas mencionadas.

Pero, al ser controlado por el compilador, no se ve tan claro qué es.

Entonces, manipular el puntero, es legal:

```
  str++;
```

Pero manipular los datos apuntados por el puntero, ¡no!

```
  str(0) ='O';
```

Total, el texto ”Hola mundo!” estará en la RAM, como cualquier otro dato del programa.

En un principio:

- Cualquier variable definida dentro de una función será memoria de stack y dinámica.
- Cualquier dato reservado usando new (o malloc, en C) será heap y dinámico.
- Cualquier variable definida de manera global es estática y está en el heap.
- Cualquier dato definido en el propio código y que sea ”visto” por el compilador es estático y en el heap.

No confundir con los modificadores ”static”:

- Una variable ”static” es una variable que, a pesar de estar dentro de la función, no está en la pila, pero si en el montón. Y sucesivas invocaciones a la función donde esté definida, no crea nuevas copias de esa variable. O sea, sus datos son los mismos.

- Una función ”static” (en C++) dentro de una clase, sólo accede a variables ”static” o globales. Esta función, aún dentro de una clase, es una función global que podrá ser invocada sin una instanciación de la clase (por eso no puede acceder a las miembros no-estáticos: porque no hay ”objeto” [this])

Hoy, los compiladores son capaces de detectar variables que son estáticas. Así que podrías encontrarte que según qué hagas con una variable, esté en la pila o en el montón y sea estática o dinámica, independientemente de cómo y dónde se haya definido.

En las últimas versiones del C++, se ha añadido un modificador: constexp. Desconozco los detalles sobre él. Sirve para que funciones aparentemente complejas, el compilador las ejecute/evalúe en tiempo de compilación.

CAPÍTULO 11

APÉNDICE

11.1. Biblioteca estándar de C++

En el estándar C++ las bibliotecas dejan de utilizar el ".h", es decir que en lugar de utilizar $< iostream.h >$, ahora tendrá que escribirse $< iostream >$, en vez de $< iomanip.h >$ será $< iomanip >$.

Esto tiene varias ventajas, que tal vez no vea en este momento, tiene que colocar "std" antes de cada llamada a una función de la biblioteca, debido a que toda la biblioteca estándar C++ está definida en el espacio de nombres llamado std. Con esto, tenemos que hacer una llamada a escribir en pantalla con la instrucción $std :: cout$.

Para evitar esto, y por consiguiente hacer menos extenso el código a escribir, empleamos la directriz using, indicando al compilador el espacio de nombres donde está referenciado.

El siguiente programa es la nueva versión de nuestros primeros programas escritos.

```
1 #include <iostream>
2 using namespace std;
3 int main(){
4   cout<<"adios a todos"<<endl;
5   return 0;
6 }
```

Para tener una idea de las capacidades aportadas por la biblioteca estándar, vea las siguientes tablas donde se clasifican las librerías de acuerdo a su funcionalidad.

Cuadro 11.1: Entrada/Salida

Biblioteca	Descripción
<cstdio>	E/S de la biblioteca de C
<cstdlib>	Funciones de clasificación de caracteres
<cwchar>	E/S de caracteres extendidos
<fstream>	Flujos para trabajo con ficheros en disco
<iomanip>	Manipuladores
<ios>	Tipos y funciones básicos de E/S
<iosfwd>	Declaraciones adelantadas de utilidades de E/S
<iostream>	Objetos y operaciones sobre flujos estándar de E/S
<istream>	Objetos y operaciones sobre flujos de entrada
<ostream>	Objetos y operaciones sobre flujos de salida
<sstream>	Flujos para trabajar con cadenas de caracteres
<streambuf>	Búferes de flujos

Cuadro 11.2: Cadenas o Strings

Biblioteca	Descripción
<cctype>	Examinar y convertir caracteres
<cstdlib>	Funciones de cadena estilo C
<cstring>	Funciones de cadena estilo C
<cwchar>	Funciones de cadena de caracteres extendidos estilo C
<cwctype>	Clasificación de caracteres extendidos
<string>	Clases para manipular cadenas de caracteres

Cuadro 11.3: Contenedores

Biblioteca	Descripción
<bitset>	Matriz de bits
<deque>	Cola de dos extremos de elementos de tipo T
<list>	Lista doblemente enlazada de elementos de tipo T
<map>	Matriz asociativa de elementos de tipo T
<queue>	Cola de elementos de tipo T
<set>	Conjunto de elementos de tipo T (contenedor asociativo)
<stack>	Pila de elementos de tipo T
<vector>	Matriz de elementos de tipo T

Cuadro 11.4: Iteradores

Biblioteca	Descripción
<iterator>	Soporte para iteradores

Cuadro 11.5: Algoritmos

Biblioteca	Descripción
<algorithm>	Algoritmos generales (buscar, ordenar, contar, etc.)
<cstdlib>	bsearch y qsort

Cuadro 11.6: Números

Biblioteca	Descripción
<cmath>	Funciones matemáticas
<complex>	Operaciones con números complejos
<cstdlib>	Números aleatorios estilo C
<numeric>	Algoritmos numéricos generalizados
<valarray>	Operaciones con matrices numéricas

Cuadro 11.7: Diagnóticos

Biblioteca	Descripción
<cassert>	Macro ASSERT
<cerrno>	Tratamiento de errores estilo C
<exception>	Clase base para todas las excepciones
<stdexcept>	Clases estándar utilizadas para manipular excepciones

Cuadro 11.8: Utilidades generales

Biblioteca	Descripción
<ctime>	Fecha y hora estilo C
<functional>	Objetos función
<memory>	Funciones para manipular bloques de memoria
<utility>	Manipular pares de objetos

Cuadro 11.9: Localización

Biblioteca	Descripción
<clocale>	Control estilo C de las diferencias culturales
<locale>	Control de las diferencias culturales

Cuadro 11.10: Soporte del lenguaje

Biblioteca	Descripción
<cfloat>	Limites numéricos en coma flotante de estilo C
<climits>	Limites numéricos estilo C
<csetjmp>	Salvar y restaurar el estado de la pila
<csignal>	Establecimiento de manejadores para condiciones excepcionales (también conocidos como señales)
<cstdarg>	Lista de parámetros de función de longitud variable
<cstddef>	Soporte de la biblioteca al lenguaje C
<cstdlib>	Definición de funciones, variables y tipos comunes
<ctime>	Manipulación de la hora y fecha
<exception>	Tratamiento de excepciones
<limits>	Limites numéricos
<new>	Gestión de memoria dinámica>
<typeinfo>	Identificadores de tipos durante la ejecución Biblioteca estándar de C++ 5

BIBLIOGRAFÍA

[1] BJARNE STROUSTRUP (1997), *The C+ + Programming Language. Addison-Wesley Publishing Company, USA*

[2] BRIAN OVERLAND (2013), *C++ for the Impatient, Addison-Wesley. ISBN-10: 0-321- 88802-2 (alk. paper). USA*

[3] SYD LOGAN (2008), CROSS-PLATFORM DEVELOPMENT IN C++, BUILDING MAC OS X, LINUX AND WINDOWS APPLICATIONS. ADDISON-WESLEY. ISBN 0-321-24642-X. USA

[4] BRUCE ECKEL (2012), *Pensar en C++ (Volumen 1) Copyright © 2000 Bruce Eckel*

[5] BRUCE ECKEL (2012), *Pensar en C++ (Volumen 2) Copyright © 2000 Bruce Eckel*

[6] SCOTT MEYERS (2008), *Effective C++ : 55 specific ways to improve your programs and designs.– 3rd ed.– Upper Saddle River (New Jersey) : Addison-Wesley*

[7] HERBERT SCHILDT (1996), *C Manual de referencia. McGraw-Hill Interamericana*

[8] AL STEVENS (2003), *C++ Programming, Editorial. Wiley India*

[9] JORGE DOMÍNGUEZ CHÁVEZ (2023), *Problemario de Lenguaje C++, IEASS, Editores*

[10] JORGE DOMÍNGUEZ CHÁVEZ (2019), *Fundamentos de Programación en Lenguaje C, IEASS, Editores*

[11] BOOCH, G. (1999), *Análisis y Diseño Orientado a Objetos con Aplicaciones, Addison Wesley*.

[12] HORTON, IVOR, IVOR HORTON (2008), *Beginning Visual C++ 2008, Wiley Publishing / WROX Programmer to Programmer*.

[13] H.M. DEITEL, P.J. DEITEL (1995), *Como programar en C/C++ (2a ed), Prentice-Hall*.

ÍNDICE ALFABÉTICO

*, 86
->, 57
#define, 43
&, 57, 86

apuntadores, 2
Archivo .cpp, 70
Archivo .h, 70
Array, 45
arreglos, 2
ASCII, 15

Bits, 22
Bool, 30
Break, 35

cadena de texto, 2
Cast, 22
Char, 15
Cin, 21
Class, 53
Conjunción lógica, 33
Constante, 13
Constructor, 64
Cout, 21
Cpu, 57

Dec, 22
Delete, 65
Destructor, 65
Dir, 42
Do while, 35

Encapsulación, 66
Endl, 21
Esc, 98
Estructura, 55
Exponencial, 15

Float, 13
For, 35
Friend, 63
Función, 41

Gcount, 27
Get, 66
Getline, 27
Global, 18

Heap, 101
Hex, 23
Hexadecimal, 14
historia, 1

If, 32
If corto, 31
Infijo, 68
Inline, 63
Instancia, 61
Internet, 8
Iteración, 37

kernel, 7

Local, 18

Math.h, 41
Memoria dinámica, 103
memoria dinámica, 3
Memory leak, 103
mensaje, 1

New, 65
Nullptr, 104

Oct, 23
Octal, 14
Operador resolución, 79

Palabras reservadas, 11
Paso por referencia, 88
Private, 62
Programa, 9
Propiedad asociativa, 19
Protected, 62
Public, 62
puntero, 2, 85

Rand, 47
Richards, M., 5
Ritchie,D., 5

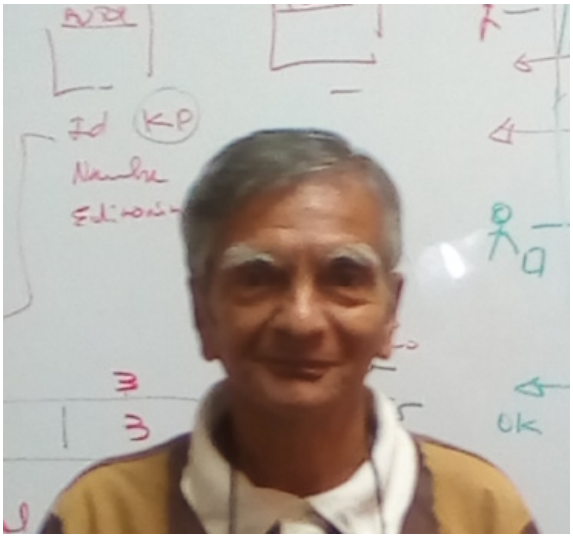
Set, 66
Setfill, 24
Setprecision, 23
Setw, 24
Srand, 47
Static, 67
Stroustrup, B., 5
Struct, 59
Switch, 37

This, 67
Thompson, K., 5
tiempo de ejecución, 2
Time(), 47
Try catch, 103

Union, 54
Unix, 5

While, 35

Acerca del autor



Graduado en Física, Facultad de Ciencias, Universidad Nacional Autónoma de México (UNAM), Doctor en Ciencias de la Computación, Instituto de Investigaciones en Matemáticas Aplicadas y Sistemas (IIMAS) UNAM.

El autor es un especialista reconocido en base de datos (Postgresql y Mariadb), desarrollo de software (aplicaciones web) y servidores en el área de LINUX DEBIAN.

Cursante del Doctorado en Minería de Datos, Modelos y Sistemas Expertos por la Universidad de Illinois en Urbana-Champaings (USA).

Auditor de sistemas informáticos certificado por Ernst & Young, Cleveland, Ohio, USA, 1985.

Ha sido profesor en la Facultad de Química y en la Facultad de Ingeniería, UNAM. Fue profesor en el Departamento de Ciencias Básicas, Universidad de Las Américas en Cholula, (UDLAP), Puebla, México y, durante seis años, profesor visitante en la Universidade Federal de Rio Grande do Sul (Brasil), profesor y jefe de sistemas postgrados de agronomía y veterinaria, Universidad Central de Venezuela (UCV), es profesor del Departamento de informática y del Departamento de Postgrado, Universidad Politécnica Territorial de Aragua (UPT Aragua), Venezuela.

Expositor y conferencista a nivel nacional e internacional.

Es asesor en mejora de procesos, gestión de proyectos, desarrollo de software corporativo en los sectores de servicios, banca, industria y e-gobierno.

El Dr. Domínguez en la actualidad orienta su trabajo a la creación y desarrollo de equipos de software de alto desempeño.

Autor de múltiples artículos y libros sobre la materia.

View publication stats